SkillBridge AG

Aarbergstrasse 46, 2503 Biel/Bienne, Schweiz

EduPlanner

Weiterbildungsplattform für Kursanbieter

arc42 Architekturdokumentation

Version	1.0
Status	Freigegeben (Lehrfassung)
Datum	März 2026
Autor	Patrick Hari
Template	arc42 Version 9 (Juli 2025) – https://arc42.org
Auftraggeber	SkillBridge AG, Biel/Bienne (fiktiv)

Hinweis – Fiktives Lehrbeispiel

Dieses Dokument dient ausschliesslich Ausbildungszwecken im Modul Software-Architektur *an der Höheren Fachschule für Technik Mittelland. Die Firma SkillBridge AG sowie das Produkt EduPlanner sind vollständig fiktiv. Jegliche Ähnlichkeit mit realen Unternehmen oder Produkten ist rein zufällig. Erstellt nach dem arc42-Template – https://arc42.org*

Begriffe, Konzepte und Entscheidungen werden teilweise ausführlicher erläutert, als es in einem reinen Projektdokument üblich wäre – insbesondere durch erklärende Fussnoten und konkrete Beispiele. In einem produktiven Projektkontext können diese Passagen zugunsten der Kürze reduziert werden.

Konzipiert und verfasst wurde das Beispiel von ***Rolf Jufer*** *(rolf.jufer@hftm.ch).*

Revisionshistorie

Version	Datum	Autor	Änderung
0.1	2026-01-15	Patrick Hari	Initiale Erstellung
0.2	2026-01-28	Patrick Hari	Kontextdiagramme
0.3	2026-02-01	Patrick Hari / Thomas Zbinden	Bounded Contexts, ADRs
0.4	2026-02-10	Patrick Hari	restliche Kapitel
1.0	2026-03-04	Sandra Meier	Review und Freigabe Version 1.0

Fiktive Personas

Alle genannten Personen (Sandra Meier, Thomas Zbinden, Patrick Hari, Andrea Senn u. a.), Budgetangaben und Entscheidungen dienen ausschliesslich der Illustration im Rahmen der HFTM Grenchen.

Inhaltsverzeichnis

Revisionshistorie 2

1 Einführung und Ziele 5
1.1 Aufgabenstellung 5
1.1.1 Geschäftskontext und Marktposition 5
1.1.2 Erwartete Nutzungszahlen (Jahr 1) 5
1.1.3 Kernfunktionen des Systems 6
1.1.4 Go-Live-Kriterien 6
1.2 Qualitätsziele 7
1.3 Stakeholder 8

2 Randbedingungen 9
2.1 Technische Randbedingungen 9
2.2 Organisatorische Randbedingungen 10
2.3 Rechtliche und regulatorische Randbedingungen 10

3 Kontextabgrenzung und Systemabgrenzung 11
3.1 Fachlicher Kontext 11
3.2 Technischer Kontext 12
3.2.1 Technische Protokolle 13
3.3 Systemabgrenzung (Out-of-Scope) 13

4 Lösungsstrategie 14
4.1 Kernarchitekturentscheidung 14
4.1.1 Microservices vs. Modularer Monolith – Faire Gegenüberstellung 14
4.2 Fachliche Strategie (Domain-Driven Design) 15
4.2.1 Bounded Contexts und Verantwortlichkeiten 16
4.2.2 DDD Context Map – Beziehungen zwischen Bounded Contexts 16
4.2.3 Zusammenfassung der Architekturentscheidungen 19
4.3 Übergreifende Architekturprinzipien 19
4.4 Qualitätsziel-Mapping 20
4.5 Skalierungsstrategie pro Bounded Context 21

5 Bausteinsicht 22
5.1 Ebene 1 – Gesamtsystem (Whitebox) 22
5.2 Ebene 2 – Bounded Context: `catalog` 23
5.2.1 CourseStatus – Zustandsübergangsdiagramm 24
5.3 Ebene 2 – Bounded Context: `scheduling` 25
5.4 Ebene 2 – Bounded Context: `registration` 26
5.4.1 Ebene 3 – CapacityChecker und OutboxPublisher (Whitebox) 27
5.5 Ebene 2 – Bounded Context: `notification` 28
5.6 Ebene 2 – Bounded Context: `identity` 29

6 Laufzeitsicht 32
6.1 Szenario 1: Teilnehmer schreibt sich ein (Happy Path) 32
6.2 Szenario 2: Kurs ausgebucht – Warteliste 32
6.3 Szenario 3: Einschreibung stornieren 33
6.4 Szenario 4: Admin veröffentlicht einen Kurs 34
6.5 Szenario 5: Cross-Context-Transaktion (Kurs archivieren) 35

7 Verteilungssicht 37
7.1 Infrastruktur-Übersicht 37
7.1.1 Cloud-Provider-Entscheid (ADR-010) 37
7.2 Deployment-Umgebungen 37
7.3 Infrastrukturkomponenten 38

7.4 Disaster Recovery Plan . . . 39
7.5 Netzwerk-Topologie . . . 39

8 Querschnittliche Konzepte 40
8.1 Sicherheitskonzept . . . 40
8.1.1 Authentifizierung . . . 40
8.1.2 Autorisierung (RBAC) . . . 40
8.2 Logging und Observability . . . 42
8.3 Datenpersistenz . . . 42
8.4 API-Design-Richtlinien . . . 42
8.4.1 Contract-First-Workflow . . . 42
8.4.2 API-Design-Regeln . . . 43
8.5 Teststrategie (Ebene 1–3) . . . 44

9 Architekturentscheidungen (ADRs) 45
9.1 ADR-001: Microservices entlang der Bounded Contexts . . . 45
9.2 ADR-002: Asynchrone Kommunikation via Message Broker . . . 46
9.3 ADR-003: Externer Identity Provider . . . 46
9.4 ADR-004: Transactional Outbox für zuverlässige Events . . . 46
9.5 ADR-005: Angular SPA als Frontend-Technologie . . . 47
9.6 ADR-006: PostgreSQL als primäre Datenbank . . . 47
9.7 ADR-007: Kein Service Mesh in v1.0 . . . 47
9.8 ADR-008: Single-Tenant-Architektur in v1.0 . . . 48
9.9 ADR-009: Outbox Relay – Technologiewahl . . . 49
9.10 ADR-010: Cloud-Provider – Microsoft Azure . . . 50
9.11 ADR-011: Contract-First mit OpenAPI 3.1 und Swagger UI . . . 51

10 Qualitätsanforderungen 53
10.1 Qualitätsbaum . . . 53
10.2 Qualitätsszenarien . . . 54

11 Risiken und technische Schulden 57
11.1 Identifizierte Risiken . . . 57
11.2 Technische Schulden . . . 59

12 Glossar 60
12.1 Fachliche Begriffe . . . 60
12.2 Domain Events – Vollständige Übersicht . . . 61

13 Betrieb und Deployment – Runbook 62
13.1 Pre-Deployment Checklist . . . 62
13.2 Incident Response . . . 62
13.3 Routine-Wartungsarbeiten . . . 63

14 Roadmap und Evolution 64
14.1 Versionsübersicht . . . 64
14.2 v1.1 – Polish und Stabilität (Q4 2026) . . . 64
14.3 v2.0 – Multi-Tenancy und SaaS (Q1–Q2 2027) . . . 65

A Anhang A: Event Storming – Ergebnisse 66
A.1 Methodik . . . 66
A.2 Vollständige Domain-Event-Liste . . . 66

B Anhang B: Diagramme in A4 Quer – Grossansicht 68

C Anhang D: API-Referenz 86

1 Einführung und Ziele

Dieses Kapitel beschreibt die wesentlichen Anforderungen und Treiber, die bei der Architekturentscheidung berücksichtigt wurden: Aufgabenstellung, Geschäftskontext, Qualitätsziele und relevante Stakeholder.

1.1 Aufgabenstellung

Die SkillBridge AG ist ein mittelgrosses Schweizer Unternehmen mit Sitz in Biel/Bienne (BE), das sich auf die Digitalisierung von Weiterbildungsangeboten für Berufsverbände, Bildungsinstitutionen und private Kursanbieter spezialisiert. Das Kernprodukt ist **EduPlanner** – eine webbasierte Weiterbildungsplattform, die Kursanbieter bei der vollständigen Verwaltung ihres Kursgeschäfts unterstützt.

EduPlanner bildet den gesamten Lebenszyklus eines Kursangebots ab – von der Erstellung und Veröffentlichung über die Planung konkreter Durchführungen bis hin zur Einschreibung von Teilnehmern und der automatischen Kommunikation relevanter Ereignisse.

1.1.1 Geschäftskontext und Marktposition

EduPlanner positioniert sich als spezialisierte SaaS-Lösung für mittelständische Kursanbieter mit 10 bis 500 Mitarbeitenden. Im Wettbewerbsumfeld konkurriert die Plattform primär mit drei Alternativen: der manuellen Verwaltung über Excel oder Google Sheets, kostenintensiven Legacy-Systemen sowie generischen Learning Management Systems (LMS), denen dedizierte Kursplanungs-Funktionalitäten fehlen.

Gegenüber diesen Alternativen differenziert sich EduPlanner durch vier Merkmale, die gleichzeitig zentrale Treiber der Architekturentscheidungen darstellen:

- **Benutzerfreundlichkeit:** Eine intuitive Oberfläche für nicht-technisches Personal erfordert klare UI-Richtlinien – konkret: maximal 3 Klicks pro Kernfunktion und Ladezeiten unter 500 ms.
- **Schnelle Implementierung:** Das Ziel, innerhalb von 2 Wochen produktiv zu gehen, macht eine vollständig automatisierte CI/CD-Pipeline zur Voraussetzung.
- **Schweizer Datenschutz:** Die Einhaltung von DSGVO und DSG erfordert einen Privacy-by-Design-Ansatz sowie eine Datenspeicherung ausschließlich in der EU bzw. der Schweiz.
- **API-First:** Um die Integration in bestehende Systemlandschaften zu ermöglichen, sind eine vollständige OpenAPI-Dokumentation und eine klare Versionierungsstrategie verbindlich.

1.1.2 Erwartete Nutzungszahlen (Jahr 1)

- 5–20 Kursanbieter (Tenants)
- 10–50 aktive Kurse im Katalog pro Anbieter
- 50–500 Einschreibungen pro Monat
- Bis zu 100 gleichzeitige Nutzer zu Spitzenzeiten

1.1.3 Kernfunktionen des Systems

Funktion	Beschreibung
Kurskatalog	Kurse anlegen, beschreiben, veröffentlichen und archivieren; Trainer zuordnen
Kursplanung	Konkrete Durchführungen (Termine, Orte, Kapazitäten) planen und verwalten
Einschreibung	Teilnehmer einschreiben, Wartelistenverwaltung, Stornierungen
Benachrichtigungen	Automatische E-Mails bei Einschreibung, Wartelistenaufnahme und Stornierung
Sicherheit	Rollenbasierte Zugriffskontrolle für alle Benutzergruppen
Reporting	Dashboard mit Auslastungsquoten, Teilnehmerstatistiken und Kursübersicht

1.1.4 Go-Live-Kriterien

MVP (Minimum Viable Product) – Monat 1–6:

- **Kurskatalog-Verwaltung:** Kurse können erstellt, bearbeitet, veröffentlicht und gelöscht werden (vollständige CRUD-Operationen)
- **Kursplanung:** Termine, Durchführungsorte und Teilnehmerkapazitäten können pro Kurs erfasst und bearbeitet werden
- **Einschreibung:** Teilnehmer können sich einschreiben; bei voller Kapazität erfolgt automatische Aufnahme in die Warteliste mit Nachrücklogik
- **E-Mail-Benachrichtigungen:** Automatischer Versand bei Einschreibung, Wartelistenaufnahme und Stornierung
- **Zugriffskontrolle:** Vier Rollen (Admin, Kursanbieter, Trainer, Teilnehmer) mit klar getrennten Berechtigungen sind vollständig implementiert und getestet
- **Verfügbarkeit:** System erreicht nachweislich 99,5 % Uptime zu Kernzeiten im Staging-Betrieb (vgl. Q2)
- **Überbuchungsschutz:** Kein einziger Fall einer Überbuchung in den Integrationstests nachgewiesen (vgl. Q1)

v1.0 (Vollständiger Release) – Monat 7–12:

- **Erweiterte Planung:** Unterstützung mehrerer Durchführungen pro Kurs (Multi-Execution) inkl. Konfliktprüfung bei Trainer- und Raumzuweisung
- **Reporting:** Auslastungsberichte, Einschreibungsstatistiken und Export-Funktion (CSV/PDF) für Kursanbieter
- **API-Dokumentation:** Vollständige OpenAPI 3.0-Spezifikation für alle öffentlichen Endpunkte, inkl. Beispielaufrufen und Fehlercode-Beschreibungen
- **Performance-Verifikation:** Lasttest mit 1 000 gleichzeitigen Nutzern bestanden; Ladezeiten unter 500 ms (Q6, Q11)
- **Security-Audit:** Penetrationstest gemäss OWASP Top 10 durchgeführt und alle kritischen Befunde behoben (Q5, Q7)

- **Datenschutz-Audit:** Nachweis der DSGVO/DSG-Konformität durch externen Audit bestätigt; Datenspeicherung ausschliesslich EU/Schweiz (Q12)

1.2 Qualitätsziele

#	ISO-25010-Attribut[1]	Priorität	Motivation
1	Functional Correctness	Kritisch	Einschreibungen und Kapazitätsprüfungen müssen stets konsistent sein. Eine Überbuchung ist geschäftskritisch und nicht tolerierbar.
2	Reliability – Availability	Hoch	Das System muss zu Kernzeiten (Mo–Fr, 07:00–22:00 Uhr) mit $\geq 99{,}5\,\%$ Verfügbarkeit betrieben werden.
3	Maintainability	Hoch	Fachliche Domänen müssen unabhängig voneinander entwickelt, getestet und deployt werden können.
4	Security	Hoch	RBAC muss sicherstellen, dass Nutzer ausschliesslich auf freigegebene Funktionen und Daten zugreifen.
5	Usability	Mittel	Kernprozesse in ≤ 3 Klicks; Ladezeiten $< 500\,\text{ms}$.
6	Performance Efficiency	Mittel	P95-Ladezeit unter 500 ms bei bis zu 1 000 gleichzeitigen Nutzern (vgl. Q6, Q11).

[1] ISO/IEC 25010 ist ein internationaler Standard, der Softwarequalität in acht übergeordnete Merkmale gliedert (u. a. *Performance Efficiency*, *Security*, *Maintainability*, *Reliability*). Jedes Merkmal ist wiederum in konkrete Teilmerkmale unterteilt, die als messbare Qualitätsziele in der Architektur verankert werden können.

1.3 Stakeholder

Rolle	Name / Funktion	Erwartungen / Interessen
Auftraggeber	Sandra Meier, CEO	Erwartet messbaren ROI durch Reduktion manueller Verwaltungsaufwände. Das System soll mit wachsender Kundenzahl skalieren, ohne proportional mehr Personal zu erfordern. Entscheidet über Budget und Go/No-Go.
Product Owner	Thomas Zbinden, Leiter Produktentwicklung	Verantwortet die termingerechte Lieferung aller vereinbarten Features. Priorisiert den Backlog, nimmt Sprints ab und stellt sicher, dass fachliche Anforderungen korrekt umgesetzt werden.
Fachbereich	Kursplanungs-Team	Nutzt das System täglich für die operative Planung. Erwartet eine intuitive Oberfläche ohne Schulungsaufwand, eine übersichtliche Kalenderansicht sowie schnelle Reaktionszeiten bei der Dateneingabe.
Endbenutzer	Teilnehmer	Benötigen einen reibungslosen Self-Service: Kurssuche, Einschreibung und Stornierung in wenigen Klicks. Erwarten zuverlässige Bestätigungs- und Erinnerungsbenachrichtigungen per E-Mail.
Endbenutzer	Trainer (Kursleiter)	Benötigen eine persönliche Übersicht ihrer zugewiesenen Durchführungen, Termine und Teilnehmerlisten. Erwarten rechtzeitige Benachrichtigung bei Änderungen oder Stornierungen.
Betrieb	Patrick Hari, IT-Leiter	Verantwortet den stabilen Betrieb der Plattform. Erwartet einfaches Deployment via CI/CD, aussagekräftiges Monitoring und Alerting sowie klar dokumentierte Konfigurationsparameter ohne Eingriffe ins Quellcode.
Entwicklung	Dev-Team	Benötigt eine klar strukturierte, gut dokumentierte Architektur mit hoher Testabdeckung. Erwartet automatisierte CI/CD-Pipelines, saubere Schnittstellendefinitionen zwischen den Services und geringe Kopplungen zwischen den Bounded Contexts.
Datenschutz	Andrea Senn, CISO	Stellt die Einhaltung von DSGVO und Schweizer DSG sicher. Erwartet Privacy-by-Design, eine revisionssichere Protokollierung von Datenzugriffen sowie eine starke Authentifizierung und rollenbasierte Zugriffskontrolle für alle personenbezogenen Daten.

2 Randbedingungen

Randbedingungen (*Constraints*) schränken den Lösungsraum ein. Im Gegensatz zu Architekturentscheidungen, die das Team bewusst trifft und in ADRs[2] festhält, sind Randbedingungen *vorgegeben* – etwa durch den Auftraggeber, die Organisation oder das regulatorische Umfeld – und können vom Architekturteam nicht einseitig geändert werden.

Historischer Entscheidungsstand

Einige technische Randbedingungen beschreiben den initialen *Entscheidungsraum, bevor konkrete Architekturentscheidungen (ADRs) getroffen wurden. Wo ADRs existieren, die eine offene Option konkretisieren (z. B. Cloud-Provider, Frontend-Framework, Message Broker), wird dies im jeweiligen Eintrag vermerkt. Die verbindliche Entscheidung liegt stets im zugehörigen ADR (Kap. 9).*

2.1 Technische Randbedingungen

Randbedingung	Hintergrund / Erläuterung
Webtechnologie	Browserbasierter Zugriff ohne lokale Installation; Unterstützung mobiler Browser ist erforderlich.
Cloud-Deployment	Public Cloud; kein On-Premise-Betrieb. *Historischer Stand zum Entscheidungszeitpunkt: Azure oder AWS präferiert.* Die finale Provider-Wahl ist in ADR-010 dokumentiert: Microsoft Azure, Region Switzerland North (Zürich).
REST / JSON	Alle Backend-Dienste stellen RESTful-HTTP-APIs bereit. GraphQL ist für eine spätere Ausbaustufe vorgesehen (vgl. Kap. 14.3).
Containerisierung	Docker-Container, orchestriert via Kubernetes (K8s 1.28+).
HTTPS / TLS	Sämtliche Client-Server-Kommunikation erfolgt über HTTPS mit TLS 1.3.
Datenbank / Messaging	PostgreSQL 15 für transaktionale Daten; RabbitMQ als Message Broker (Azure Service Bus als evaluierte Alternative – zurückgestellt, vgl. ADR-002).
Authentifizierung	OAuth 2.0 / OpenID Connect über einen externen Identity Provider (z. B. Keycloak). Kein proprietäres Auth-System.
Programmiersprache	Backend: Java 21 (Spring Boot 3) oder Kotlin. Frontend: TypeScript mit Angular (entschieden, vgl. ADR-005; Vue.js als gleichwertige Alternative evaluiert und zurückgestellt).

[2] **ADR – Architecture Decision Record.** Ein ADR ist ein kurzes, strukturiertes Dokument, das eine einzelne, architekturrelevante Entscheidung samt Kontext, betrachteten Alternativen und Begründung festhält. Das Format geht auf Michael Nygard zurück (https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions).

2.2 Organisatorische Randbedingungen

Randbedingung	Hintergrund / Erläuterung
Teamgrösse	Cross-funktionales Team von 5–8 Personen (inkl. QA-Engineering).
Entwicklungsprozess	Scrum mit 2-Wochen-Sprints. Die Definition of Done schliesst Unit- und Integrationstests ein.
Zeitrahmen	MVP innerhalb von 6 Monaten; vollständiger Rollout nach 12 Monaten.
Budget (Cloud-Betrieb)	Cloud-Betriebskosten $\leq$ CHF 800 / Monat. Ein Puffer von 10 % (CHF 80) wird empfohlen (vgl. Kap. 11, R12).
Open Source	Bevorzugte Nutzung von Komponenten unter Apache-2.0- oder MIT-Lizenz.
Source Control	Git (GitHub). Trunk-Based Development[3] mit kurzlebigen Feature-Branches und verpflichtenden Pull-Request-Reviews.

2.3 Rechtliche und regulatorische Randbedingungen

Randbedingung	Hintergrund
DSGVO / revDSG	Personenbezogene Daten unterliegen der EU-DSGVO sowie dem Schweizer revDSG. Datenspeicherung erfolgt ausschliesslich in der EU bzw. der Schweiz.
Datensparsamkeit	Es werden nur die für den Kernbetrieb notwendigen Personendaten erhoben (Grundsatz der Datenminimierung).
Recht auf Löschung	Personendaten müssen auf Anfrage innerhalb von 30 Tagen gelöscht oder anonymisiert werden (Art. 17 DSGVO; Art. 32 revDSG).
Auftragsverarbeitun	Sämtliche Cloud-Provider und externen Dienstleister sind über einen Auftragsverarbeitungsvertrag (AVV) nach Art. 28 DSGVO zu binden.
Protokollierung	Sicherheitsrelevante Aktionen müssen auditierbar protokolliert werden. Aufbewahrungsfrist: mindestens 3 Jahre.

[3] **Trunk-Based Development** bezeichnet ein Versionsverwaltungsmodell, bei dem es nur einen einzigen Hauptzweig (`main`) gibt, der jederzeit auslieferbar sein muss. Änderungen entstehen auf kurzlebigen Arbeitszweigen (*Feature-Branches*, Lebensdauer 1–3 Tage) und werden über sogenannte *Pull Requests* zurückgeführt: Mindestens ein weiteres Teammitglied prüft dabei den Code auf Korrektheit und Einhaltung der Projektstandards (*Code Review*). Die Zusammenführung erfolgt erst nach erfolgreicher Prüfung und bestandener automatisierter Testpipeline.

3 Kontextabgrenzung und Systemabgrenzung

Die Kontextabgrenzung zeigt EduPlanner in seinem Umfeld: welche Nutzer, Nachbarsysteme und externe Dienste mit dem System interagieren und über welche Schnittstellen. Die **Systemabgrenzung** (neu in arc42 v9) beschreibt explizit, was *nicht* Teil des Systems ist.

3.1 Fachlicher Kontext

EduPlanner als Blackbox mit seinen Akteuren und Nachbarsystemen (*Diagrammtyp: C4 Context Diagram*):

Akteur / System	Kanal	Interaktion
Teilnehmer	HTTPS (Browser)	Self-Service-Portal: Kursübersicht, Einschreibung, Stornierung, Einschreibungshistorie
Trainer	HTTPS (Browser)	Ansicht eigener Durchführungen, Teilnehmerlisten
Staff	HTTPS (Browser)	Kurs- und Durchführungsverwaltung, Trainer-Zuweisung
Admin	HTTPS (Browser)	Vollständige Verwaltungsoberfläche inkl. Veröffentlichung und Nutzerverwaltung
E-Mail-Dienst[4]	SMTP / HTTPS	Transaktionaler E-Mail-Versand (z. B. Einschreibungsbestätigungen, Stornierungen, Erinnerungen) über einen Cloud-E-Mail-Dienst wie Amazon SES oder SendGrid.
Identity Provider (OIDC)	OAuth 2.0 / OIDC	Authentifizierung und Token-Ausstellung; EduPlanner verwaltet keine Passwörter.
Kalender (iCal, opt.)	iCal / CalDAV	Optionaler Export in externe Kalender (Google, Outlook). Begründung "optional": kein Kernprozess; Implementierung in v1.1 geplant.

[4] **Amazon SES – Simple Email Service.** Ein verwalteter Cloud-Dienst von AWS für den skalierbaren Versand transaktionaler E-Mails. SES übernimmt Zustellung, Bounce-Handling und Reputationsmanagement. Alternativen: SendGrid, Mailgun, Azure Communication Services.

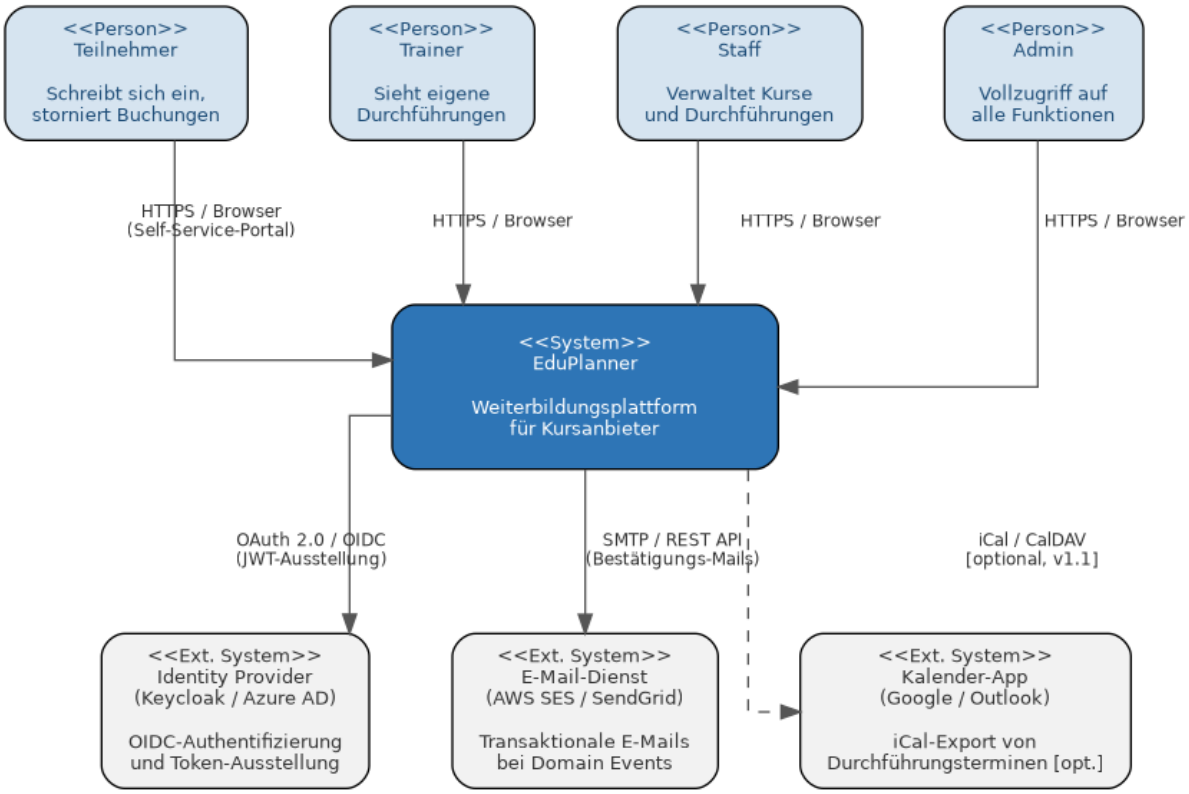


Abbildung 1: *C4 - Kontext Diagramm EduPlanner*

3.2 Technischer Kontext

Diagrammtyp: C4 Container Diagram. Schichten:

1. **Client:** Angular SPA oder Vue.js SPA (TypeScript) im Browser.
2. **API Gateway:** Spring Cloud Gateway oder NGINX – TLS-Terminierung, Rate-Limiting, JWT-Weiterleitung.
3. **Services:** catalog, scheduling, registration, notification, identity – je in eigenem Kubernetes-Pod.
4. **Datenhaltung:** Je eine PostgreSQL-Datenbank pro Service; RabbitMQ für asynchrone Events.

3.2.1 Technische Protokolle

Verbindung	Protokoll	Bemerkung
SPA → API Gateway	HTTPS / TLS	Port 443; REST APIs via JSON
Gateway → Services	HTTP / JSON	Internes K8s-Netzwerk; JWT-Header-Propagierung
Services → DB	JDBC	PostgreSQL-Protokoll; Port 5432
Services → RabbitMQ	AMQP	Port 5672; asynchrone Domain Events
System → E-Mail	SMTP/REST	Verschlüsselt via STARTTLS oder REST-API
System → IdP	OIDC/OAuth 2.0	HTTPS-Endpunkte für Token-Validation und UserInfo
API-Spezifikation	OpenAPI 3.1.0	Contract-First: Spec in `openapi/eduplanner-api.yaml`; Swagger UI je Service unter `/swagger-ui/index.html`; vollständige Übersicht in Anhang D (ADR-011).

3.3 Systemabgrenzung (Out-of-Scope)

Systemgrenze in einem Satz

*EduPlanner verwaltet Kurskatalog, Durchführungsplanung, Einschreibung, Benachrichtigung und Identitätsanbindung – **nicht** aber Lerninhalte, Zahlungsflüsse oder die Passwortverwaltung der Nutzer.*

Was EduPlanner **nicht** leistet (Explizite Abgrenzung nach arc42 v9):

Nicht im Scope	Begründung / Verweis
Lerninhalt (LMS)	EduPlanner verwaltet Kurse und Buchungen – kein Moodle-Ersatz. Inhalte (Videos, Lernmaterial) liegen extern. LMS-Integration geplant für v2.0 (Kap. 14).
Zahlungsabwicklung	Integration einer Zahlungsabwicklung wie z. B. Stripe erst in v2.0 (Kap. 14). Rechnungsstellung und Zahlungsflüsse sind kein MVP-Scope.
Identity Management	Passwortverwaltung, MFA-Setup[5] und User-Provisioning liegen vollständig beim externen IdP (ADR-003).
Videokonferenz / Streaming	Keine Integration von Zoom, MS-Teams o. ä. Durchführungen können physisch oder extern gelinkt sein.
Multi-Tenancy (v1.0)	Kursanbieter-Mandantentrennung erst ab v2.0 (ADR-008, TS4).
Mobile App	Native iOS/Android-App erst in v2.0. v1.0: Responsive Web via gegebene SPA (Angular oder Vue.js).

[5] **MFA – Multi-Factor Authentication** (Mehr-Faktor-Authentifizierung). Bei MFA muss sich ein Benutzer mit mindestens zwei unabhängigen Faktoren ausweisen, z. B. Passwort (Wissen) und Einmalcode per Authenticator-App (Besitz). **MFA-Setup** bezeichnet die erstmalige Einrichtung dieser zusätzlichen Faktoren – etwa das Verknüpfen einer Authenticator-App oder das Hinterlegen einer Mobilnummer für SMS-Codes. Im Kontext von EduPlanner liegt dieser gesamte Prozess beim externen Identity Provider und ist daher nicht Teil des eigenen Funktionsumfangs.

4 Lösungsstrategie

Dieses Kapitel beschreibt die grundlegenden Architekturentscheidungen und Lösungsansätze. Detaillierte Begründungen finden sich in den ADRs (Kap. 9).

4.1 Kernarchitekturentscheidung

EduPlanner folgt einer **Microservice-Architektur**[6], ausgerichtet an den fünf fachlichen Bounded Contexts[7] aus dem Domain-Driven Design (DDD). Jeder Bounded Context wird als eigenständiger Dienst mit eigener Datenbank (Database-per-Service-Pattern) implementiert. → Vollständige Begründung: ADR-001.

4.1.1 Microservices vs. Modularer Monolith – Faire Gegenüberstellung

Didaktische Anmerkung

Die Entscheidung für Microservices ist bei kleinem Team und engem MVP-Scope nicht selbstverständlich. Nachfolgend eine explizite Gegenüberstellung beider Optionen – denn für EduPlanner wäre ein modularer Monolith zunächst die einfachere *Wahl gewesen.*

[6] **Microservice-Architektur.** Ein Architekturstil, bei dem eine Anwendung als Sammlung kleiner, eigenständiger Dienste (*Services*) realisiert wird. Jeder Service ist unabhängig deploybar, besitzt einen eigenen Datenhaushalt und kommuniziert mit anderen Services über wohldefinierte Schnittstellen (typischerweise REST oder asynchrone Nachrichten). Im Gegensatz zum monolithischen Ansatz lassen sich einzelne Services unabhängig skalieren, aktualisieren und durch verschiedene Teams weiterentwickeln. Vgl. Sam Newman, *Building Microservices*, O'Reilly, 2021.

[7] **Bounded Context (Domain-Driven Design).** Ein *Bounded Context* ist ein zentrales Konzept aus dem Domain-Driven Design (DDD) von Eric Evans. Er definiert eine explizite Grenze, innerhalb derer ein bestimmtes Domänenmodell mit einer einheitlichen Fachsprache (*Ubiquitous Language*) gilt. Ausserhalb dieser Grenze können dieselben Begriffe eine andere Bedeutung haben – z. B. meint "Kurs" im Context `catalog` die inhaltliche Beschreibung, im Context `scheduling` hingegen eine konkrete Durchführung mit Datum und Ort. Bounded Contexts bilden in einer Microservice-Architektur die natürlichen Dienstgrenzen: ein Service pro Bounded Context. Vgl. Eric Evans, *Domain-Driven Design*, Addison-Wesley, 2003.

Kriterium	Modularer Monolith	Microservices (gewählt)
Betriebskomplexität	Gering: 1 Prozess, 1 Datenbank, 1 Deployment-Artefakt.	Hoch: 5 Services, 5 Datenbanken, Message Broker, API Gateway.
Lokale Entwicklung	Einfach: `mvn spring-boot:run` genügt.	Aufwändig: Docker Compose mit 8+ Containern.
Skalierbarkeit	Immer das gesamte System; keine Teilskalierung.	Selektiv: `registration` zur Spitzenzeit auf 10 Pods, `catalog` auf 1 Pod.
Team-Autonomie	Gemeinsames Codebase; Merge-Konflikte bei paralleler Entwicklung.	Separate Repositories; Teams deployen unabhängig.
Datenkonsistenz	ACID-Transaktionen über Modulgrenzen hinweg.	Nur Eventual Consistency; Saga-Muster für verteilte Transaktionen.
Einstiegshürde	Niedrig: Standard-Wissen reicht.	Hoch: Verteilte Systeme, Netzwerkpartitionen, Observability.
Technische Schulden	Modulgrenzen erodieren mit der Zeit ("Big Ball of Mud").	Service-Grenzen sind hart; Erosion schwerer möglich.
Passt für EduPlanner?	Ja – für MVP und kleine Teamgrösse sogar naheliegender.	Ja – wenn Skalierbarkeit und Team-Autonomie langfristig wichtiger sind als kurzfristige Einfachheit.

Warum trotzdem Microservices?

Drei konkrete Faktoren haben den Ausschlag gegeben (vollständige Begründung in ADR-001):

1. ***Unterschiedliche Skalierungsbedarfe:*** *Der `registration`-Service muss zur Einschreibungsfrist horizontal skalieren; `catalog` und `notification` haben einen Bruchteil der Last. Im Monolithen wäre nur vertikale Skalierung des Gesamtsystems möglich.*
2. ***Klare fachliche Grenzen:*** *Die fünf Bounded Contexts haben grundlegend verschiedene Änderungsraten (Katalog ändert sich selten, Einschreibungslogik häufig). Getrennte Deployments vermeiden, dass ein stabiles Modul wegen Änderungen an einem anderen neu deployt wird.*
3. ***Lernziel der Unterrichtsunterlage:*** *Microservices sind in der Praxis weit verbreitet; die Dokumentation zeigt Studierenden ein realistisches Architekturmuster mit seinen echten Trade-offs (vgl. ATAM-light, Kap. 10.1).*

Modularer Monolith als Fallback: *Falls der Betriebsaufwand das Team überfordert, ist ein Rückbau auf einen modularen Monolithen explizit als Risikomitigierung geplant (Risiko R5, Kap. 11).*

4.2 Fachliche Strategie (Domain-Driven Design)

1. **Event Storming** (nach Alberto Brandolini) als kollaborative Methode zur Domänenentdeckung. Dabei modellieren Fachexperten und Entwickler gemeinsam mit Hilfe eines Whiteboards den gesamten Geschäftsprozess – ausgehend von *Ereignissen* ("Teilnehmer hat sich eingeschrieben", "Kurs wurde veröffentlicht" etc.). Aus den entdeckten Ereignissen lassen sich natürliche fachliche Grenzen ableiten, die direkt in die Bounded-Context-Struktur einfliessen. Das Event-Storming-Ergebnis für EduPlanner ist in Anhang A dokumentiert.
2. **5 Bounded Contexts** als natürliche Dienstgrenzen. EduPlanner definiert fünf Kontexte mit

klarer fachlicher Verantwortlichkeit: `catalog`, `scheduling`, `registration`, `notification` und `identity`. Die Kontextgrenzen wurden im Event Storming identifiziert und decken sich bewusst 1:1 mit den Servicegrenzen (Details siehe Abschnitt 4.2.1).

3. **Ubiquitous Language** (allgegenwärtige Sprache) je Kontext. Jeder Bounded Context pflegt ein eigenes, verbindliches Vokabular, das sowohl im Fachgespräch als auch im Quellcode (Klassen, Methoden, Events) konsistent verwendet wird. Dadurch wird sichergestellt, dass Missverständnisse zwischen Fachseite und Entwicklung minimiert werden. Das vollständige Glossar pro Context findet sich in Kap. 12.

4. **Domain Events** als primäres Muster der kontextübergreifenden Kommunikation. Wenn in einem Bounded Context ein fachlich relevantes Ereignis eintritt (z. B. `EnrollmentConfirmed`), wird dieses als *Domain Event* über den Message Broker (RabbitMQ) veröffentlicht. Andere Contexts können dieses Event abonnieren und darauf reagieren – etwa der `notification`-Context, der daraufhin eine Bestätigungsmail versendet. Dieses Muster entkoppelt die Services: Der sendende Context muss nicht wissen, wer das Event konsumiert.

5. **Aggregate** als Konsistenzgrenzen. Ein Aggregate ist eine Gruppe fachlich zusammengehöriger Objekte, die immer gemeinsam geladen, validiert und gespeichert werden. Jedes Aggregate besitzt genau einen *Aggregate Root* – das ist der einzige Einstiegspunkt von aussen. Beispiel: Im Context `registration` bildet die Einschreibung (`Enrollment`) das Aggregate Root; die zugehörigen Wartelisteneinträge und Stornierungsinformationen sind Teil desselben Aggregates und können nur über das Root verändert werden. So wird sichergestellt, dass Geschäftsregeln (z. B. "maximale Teilnehmerzahl") bei jeder Änderung konsistent geprüft werden.

4.2.1 Bounded Contexts und Verantwortlichkeiten

Die folgende Tabelle ordnet jedem Bounded Context seine fachliche Verantwortlichkeit zu. Die Spalte *ADRs* verweist auf die Architekturentscheidungen (Kap. 9), die den jeweiligen Context massgeblich beeinflusst haben – etwa hinsichtlich Serviceschnitt, Kommunikationsmuster oder Datenhaltung.

Bounded Context	Verantwortlichkeit	Relevante ADRs
`catalog`	Kursbeschreibungen, Lernziele, Trainer-Profile. Definiert *was* angeboten wird.	ADR-001, ADR-006
`scheduling`	Durchführungen, Termine, Kapazitäten, Trainer-Zuweisung. Definiert *wann* und *wie*.	ADR-001, ADR-006
`registration`	Einschreibungen, Warteliste, Kapazitätsprüfung, Stornierungen. Verwaltet *wer*.	ADR-001, ADR-004, ADR-006
`notification`	Event-getriebener Versand von E-Mail-Benachrichtigungen.	ADR-001, ADR-002
`identity`	Authentifizierung, Rollenmanagement, OIDC-Integration.	ADR-001, ADR-003

4.2.2 DDD Context Map – Beziehungen zwischen Bounded Contexts

Die Context Map zeigt, wie die fünf Bounded Contexts miteinander interagieren. Jede Beziehung folgt einem etablierten *Beziehungsmuster* aus dem Domain-Driven Design. Diese Muster beschreiben, wie zwei Contexts miteinander kommunizieren und wer dabei die Hoheit über das Datenmodell hat.

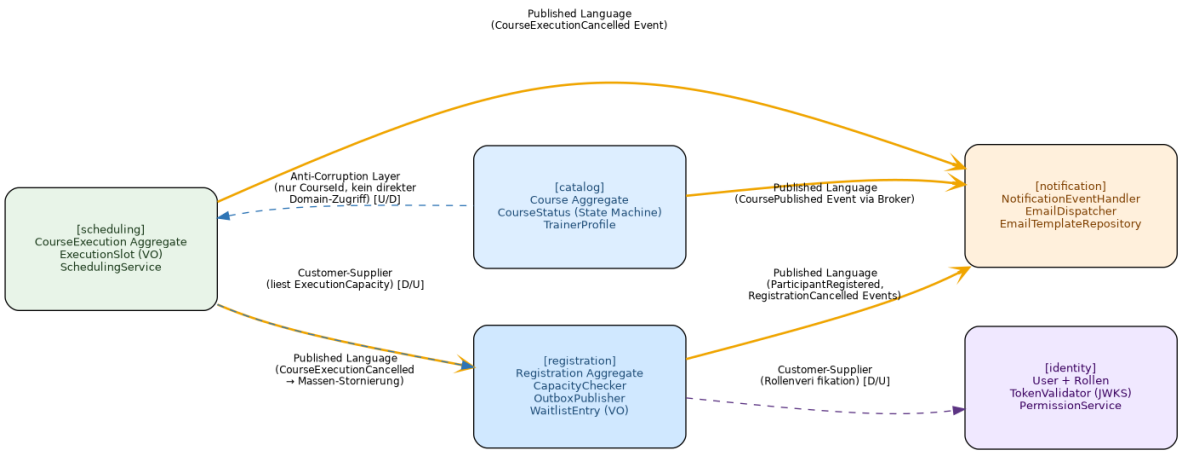


Abbildung 2: *DDD Context Map: Bounded Contexts und Beziehungen*

Beziehung	Muster (DDD)	Erläuterung
scheduling ↔ catalog	Anti-Corruption Layer (ACL)	Ein ACL ist eine Schutzschicht, die verhindert, dass das Domänenmodell eines fremden Contexts in den eigenen Code "hineinwächst". Konkret: `scheduling` übernimmt *keine* Klassen oder Strukturen aus `catalog`, sondern übersetzt eingehende Daten an der Grenze in eigene Objekte. Es wird lediglich die `CourseId` als Referenz verwendet – nie das vollständige Kursmodell. So bleibt `scheduling` unabhängig, selbst wenn sich das Modell in `catalog` ändert.
registration → scheduling	Customer-Supplier	In diesem Muster gibt es eine klare Rollenverteilung: Der *Supplier* (Lieferant) stellt eine Schnittstelle bereit, der *Customer* (Kunde) konsumiert sie. Hier liefert `scheduling` als Supplier Kapazitätsdaten (z. B. freie Plätze) über eine synchrone REST-API. `registration` als Customer ruft diese Daten ab, um bei einer Einschreibung zu prüfen, ob noch Plätze verfügbar sind. Der Supplier verpflichtet sich, die Schnittstelle stabil zu halten.
registration → identity	Customer-Supplier	Gleiche Rollenverteilung: `identity` ist Supplier und stellt das Berechtigungsmodell bereit (Rollen, Berechtigungen). `registration` als Customer fragt bei Bedarf Rollendaten ab – etwa um zu entscheiden, ob ein Nutzer eine Einschreibung stornieren darf.
catalog / reg / sched → notification	Published Language	Bei diesem Muster einigen sich die beteiligten Contexts auf ein *gemeinsames, dokumentiertes Datenformat* für die Kommunikation – hier in Form von Domain Events mit definierter JSON-Struktur (z. B. `EnrollmentConfirmed`, `CoursePublished`). `notification` kennt *ausschliesslich* dieses Event-Format und hat keinen Einblick in die internen Domänenmodelle der sendenden Contexts. Die Events werden asynchron über den Message Broker (RabbitMQ) transportiert. So kann `notification` eine Bestätigungsmail versenden, ohne zu wissen, wie eine Einschreibung intern modelliert ist.

4.2.3 Zusammenfassung der Architekturentscheidungen

Die folgende Tabelle fasst die zentralen Architekturentscheidungen zusammen, die aus der gewählten Microservice-Architektur und den oben beschriebenen Bounded-Context-Beziehungen resultieren. Jede Entscheidung ist in einem eigenen ADR (Kap. 9) ausführlich dokumentiert und begründet.

Entscheidung	Begründung	ADR
Microservices	Unabhängige Deployments, Team-Autonomie, horizontale Skalierung	ADR-001
Database per Service	Verhindert Kopplung auf DB-Ebene; unabhängige Schema-Migrationen	ADR-001, ADR-006
REST + JSON	Standardprotokoll, breite Toolunterstützung	ADR-005
Asynchrones Messaging	Entkopplung; Resilienz bei Teilausfällen	ADR-002
Transactional Outbox	Atomare Event-Persistenz; verhindert Dual-Write-Problem	ADR-004, ADR-009
Container / Kubernetes	Einheitliches Deployment, Auto-Scaling, Cloud-Portabilität	ADR-001
Externer OIDC-IdP	Security-Best-Practices, MFA, DSGVO-Compliance	ADR-003
Angular / Vue.js SPA	Reaktive UX; bekanntes Ökosystem im Team	ADR-005

4.3 Übergreifende Architekturprinzipien

Die folgenden Prinzipien sind die nicht-verhandelbaren Leitlinien der EduPlanner-Architektur. Sie gelten serviceübergreifend und dienen als Entscheidungsrahmen im Entwicklungsalltag: Wann immer eine Design-Frage aufkommt, geben diese Prinzipien die Richtung vor. Einzelne Architekturentscheidungen (Kap. 9) lassen sich jeweils auf eines oder mehrere dieser Prinzipien zurückführen.

- **Favour Async over Sync:** Zustandsändernde Operationen zwischen Bounded Contexts erfolgen ausschliesslich via Domain Events. Synchrone REST-Aufrufe nur für lesende Operationen.
- **No Shared Databases:** Kein direkter Cross-Service-Datenbankzugriff.
- **Fail Fast, Recover Gracefully:** Transiente Fehler durch Retry + Exponential Backoff. Nicht-kritische Ausfälle (z. B. `notification`) blockieren nie den Kernprozess.
- **Security by Default:** JSON-Web-Token-Validierung (JWT) am Gateway; zusätzliche rollenbasierte Zugriffskontrolle (Role-Based Access Control, RBAC) in jedem Service.
- **Privacy by Design:** Keine personenbezogenen Daten (Personally Identifiable Information, PII) in Logs, Events oder Metriken. Events referenzieren ausschliesslich `userId`, nie E-Mail-Adresse oder Name.
- **Explizite Architekturentscheidungen:** Jede signifikante Entscheidung wird als Architecture Decision Record (ADR) dokumentiert.

4.4 Qualitätsziel-Mapping

Jede Architekturentscheidung in EduPlanner dient mindestens einem übergeordneten Qualitätsziel. Die folgende Tabelle macht diesen Zusammenhang explizit: Sie ordnet den zentralen Qualitätszielen nach ISO 25010[8] die jeweils wichtigsten Architekturmassnahmen zu. So wird nachvollziehbar, *warum* bestimmte Muster und Technologien gewählt wurden – und welche Kapitel die jeweilige Massnahme im Detail beschreiben.

Qualitätsziel (ISO 25010)	Architekturmassnahme	Referenz
Functional Correctness	*Optimistic Locking* stellt sicher, dass parallele Änderungen an denselben Daten (z. B. zwei gleichzeitige Einschreibungen auf den letzten Platz) erkannt und konfliktfrei aufgelöst werden. Alle Schreiboperationen erfolgen transaktional. Event-Handler sind *idempotent* – d. h. ein mehrfach empfangenes Event führt nicht zu doppelten Einschreibungen.	Kap. 5.4, 8.4
Reliability – Availability	Kubernetes *Health-Probes* (Liveness / Readiness) erkennen ausgefallene Pods und starten sie automatisch neu. *Circuit Breaker* (via Resilience4j) verhindern, dass der Ausfall eines Services sich kaskadenartig auf andere ausbreitet. Die PostgreSQL-Datenbanken laufen redundant; der Message Broker (RabbitMQ) wird als Cluster betrieben.	Kap. 8.2, 7.4
Maintainability	Die strikten Bounded-Context-Grenzen sorgen dafür, dass Änderungen an einem Service keine Seiteneffekte in anderen Services verursachen. Kein Service greift direkt auf die Datenbank eines anderen zu. Alle architekturrelevanten Entscheidungen sind in ADRs (Kap. 9) festgehalten, sodass das Projektteam auch Monate später nachvollziehen kann, *warum* etwas so entschieden wurde.	Kap. 5, 9
Security	JSON Web Tokens (JWT) werden am API-Gateway validiert; jeder Service führt zusätzlich eine eigene rollenbasierte Zugriffskontrolle (RBAC) durch. Sämtliche Kommunikation läuft über HTTPS/TLS. In Logs, Events und Metriken werden keine personenbezogenen Daten (PII) gespeichert.	Kap. 8.1, 8.2
Usability	Die Single-Page-Application (Angular oder Vue.js) bietet eine reaktive Benutzeroberfläche ohne Seitenwechsel. Formulare sind auf minimale Eingaben optimiert. *Progressive Loading* lädt nur die aktuell benötigten Inhalte nach, um die wahrgenommene Ladezeit zu reduzieren. Der Mobile-First-Ansatz stellt sicher, dass das Interface auch auf Smartphones nutzbar ist.	ADR-005

[8] **ISO 25010** ist ein internationaler Standard, der ein Qualitätsmodell für Softwareprodukte definiert. Er gliedert Qualität in acht Hauptmerkmale – darunter Funktionalität, Zuverlässigkeit, Sicherheit, Wartbarkeit und Benutzbarkeit. Das Modell dient als gemeinsame Sprache, um Qualitätsanforderungen messbar und vergleichbar zu formulieren.

4.5 Skalierungsstrategie pro Bounded Context

Nicht alle Services von EduPlanner sind gleich stark belastet. Der `registration`-Service erlebt beispielsweise zu Einschreibungsfristen hohe Lastspitzen, während `catalog` überwiegend Lesezugriffe bedient. Deshalb wird jeder Service individuell skaliert.

Die Skalierung erfolgt automatisiert über den Kubernetes *Horizontal Pod Autoscaler* (HPA): Sobald ein definierter Schwellenwert (*Scale-Trigger*) überschritten wird, startet Kubernetes zusätzliche Pod-Instanzen des betroffenen Services – bis zur konfigurierten Obergrenze. Fällt die Last wieder, werden überschüssige Pods abgebaut. Die Spalten *Min Pods* und *Max Pods* definieren den Skalierungskorridor pro Service.

Service	Min Pods	Max Pods	Scale-Trigger
registration	2	10	CPU-Auslastung > 70 % oder Warteschlangentiefe > 100 Nachrichten
catalog	1	3	CPU-Auslastung > 80 % (primär Lesezugriffe)
scheduling	2	5	CPU-Auslastung > 70 %
notification	1	10	Warteschlangentiefe > 500 Nachrichten
identity	2	5	Antwortzeit (Request-Latency) > 200 ms

5 Bausteinsicht

Die Bausteinsicht zeigt die statische Zerlegung des Systems in seine Bausteine (Services, Komponenten, Klassen) sowie deren Abhängigkeiten. Sie beantwortet die Frage: *Aus welchen Teilen besteht EduPlanner und wie hängen diese zusammen?*

Die Darstellung folgt dem **Zoom-in-Prinzip** des arc42-Templates: Ebene 1 zeigt das Gesamtsystem als Whitebox mit seinen fünf Services und der Infrastruktur. Die Ebenen 2 und 3 zoomen schrittweise in einzelne Bounded Contexts hinein – bis hin zu kritischen Klassen und Algorithmen.

Für die Diagramme wird die **C4-Notation**[9] verwendet. Jede Ebene enthält zunächst ein Diagramm zur Orientierung, gefolgt von einer Tabelle bzw. Auflistung mit den Verantwortlichkeiten der dargestellten Bausteine.

5.1 Ebene 1 – Gesamtsystem (Whitebox)

Das folgende C4-Container-Diagramm (Abb. 3) zeigt alle deployierbaren Einheiten von EduPlanner sowie die Kommunikationswege zwischen ihnen. Die fünf fachlichen Services (blau) entsprechen den Bounded Contexts aus Kapitel 4. Infrastrukturbausteine (grau) – API Gateway, Message Broker und Datenbanken – sind unterstützende Komponenten ohne eigene Fachlogik. Externe Systeme (orange) liegen ausserhalb der Systemgrenze.

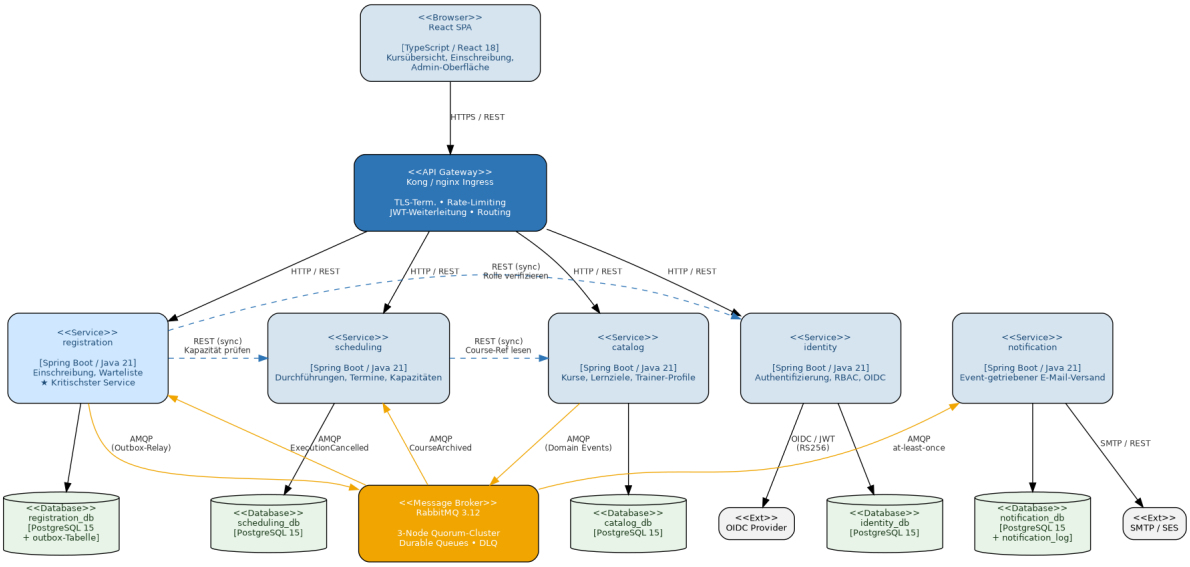


Abbildung 3: *Ebene 1 – C4-Container-Diagramm: Gesamtsystem EduPlanner mit Services, Infrastruktur und externen Systemen.*

Die nachfolgende Tabelle beschreibt die Verantwortlichkeit jedes Bausteins im Detail:

[9] **C4-Modell** (nach Simon Brown). Ein hierarchisches Diagrammmodell für Softwarearchitektur mit vier Abstraktionsebenen: *Context* (System im Umfeld), *Container* (deploybare Einheiten), *Component* (logische Bausteine innerhalb eines Containers) und *Code* (Klassen/Interfaces). Jede Ebene zoomt tiefer in das System hinein. Vgl. `https://c4model.com`.

Baustein	Verantwortlichkeit
`catalog`	Kursbeschreibungen, Lernziele, Trainer-Profile. Publiziert `CourseCreated`, `CoursePublished`, `CourseArchived`.
`scheduling`	Planung von Kursdurchführungen inkl. Termine, Kapazitäten, Trainer-Zuweisung.
`registration`	Einschreibungsverwaltung: Kapazitätsprüfung, Warteliste, Stornierungen. **Kritischster Service.**
`notification`	Event-getriebener E-Mail-Versand; kein Einfluss auf Kernprozesse.
`identity`	Authentifizierung via OIDC-IdP; Rollen- und Benutzerverwaltung.
API Gateway	TLS-Terminierung, Rate-Limiting (100 Req/s), JWT-Weiterleitung, Routing.
Message Broker	Entkoppelte asynchrone Kommunikation via Durable Queues und Dead-Letter-Exchange.
Reporting (BFF)	v1.0: Dashboard-Daten aus `catalog` und `registration` separat abgefragt und im Frontend aggregiert (BFF-Light via Angular / Vue.js). v2.0: Dedizierter Reporting-Service geplant (Kap. 14.3).

5.2 Ebene 2 – Bounded Context: `catalog`

Der `catalog`-Service ist fachlich der einfachste Bounded Context: Er verwaltet Kursbeschreibungen und Trainer-Profile und steuert über eine Zustandsmaschine, welche Kurse für Teilnehmer sichtbar sind. Abbildung 4 zeigt die internen Bausteine und deren Zusammenspiel – vom REST-Controller über die Fachlogik bis hin zur Datenbank und Event-Publikation.

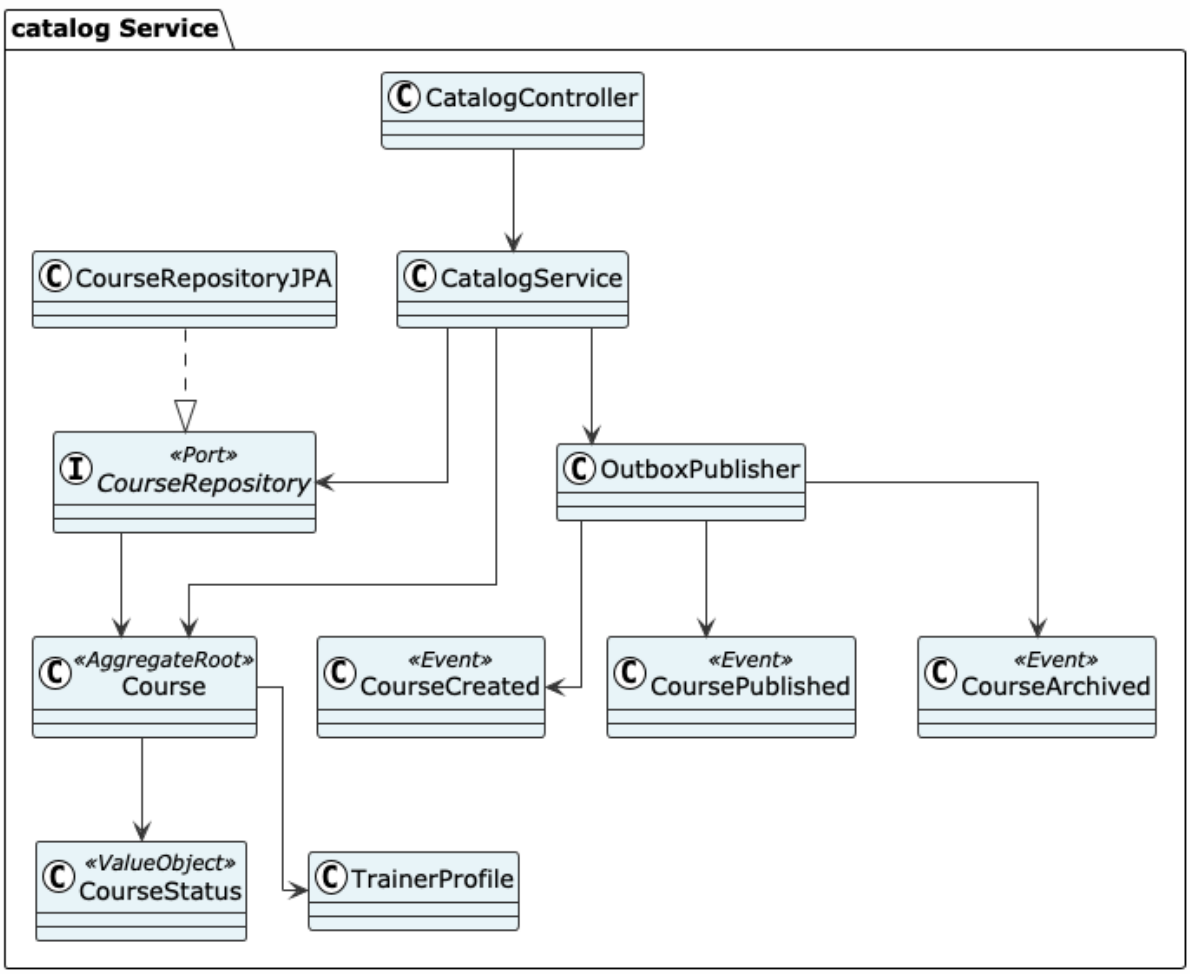


Abbildung 4: *Ebene 2 – UML-Klassendiagramm: Interne Bausteine des* `catalog`*-Service.*

Wichtige Klassen / Aggregate:

- **Course** (Aggregate Root): `CourseId`, Titel, Beschreibung, Lernziele, `CourseStatus` (Draft → Published → Archived), Trainer.
- **TrainerProfile**: Stammdaten, Fachgebiete, Verfügbarkeitsstatus.
- **CourseStatus**: Value Object; steuert erlaubte Zustandsübergänge (State Machine).
- **CourseRepository**: Port-Interface; Implementierung via JPA/Hibernate.

Domain Events:

- `CourseCreated` – Neuer Kurs wurde angelegt
- `CoursePublished` – Kurs wurde veröffentlicht
- `CourseArchived` – Kurs wurde archiviert

5.2.1 CourseStatus – Zustandsübergangsdiagramm

Das `CourseStatus`-Value-Object steuert den Lebenszyklus eines Kurses über eine einfache Zustandsmaschine mit drei Zuständen und erlaubten Übergängen (Abb. 5). Rückwärtsübergänge sind bewusst nicht vorgesehen (ADR-006): Ein archivierter Kurs kann nicht reaktiviert werden – stattdessen muss ein neuer Kurs angelegt werden.

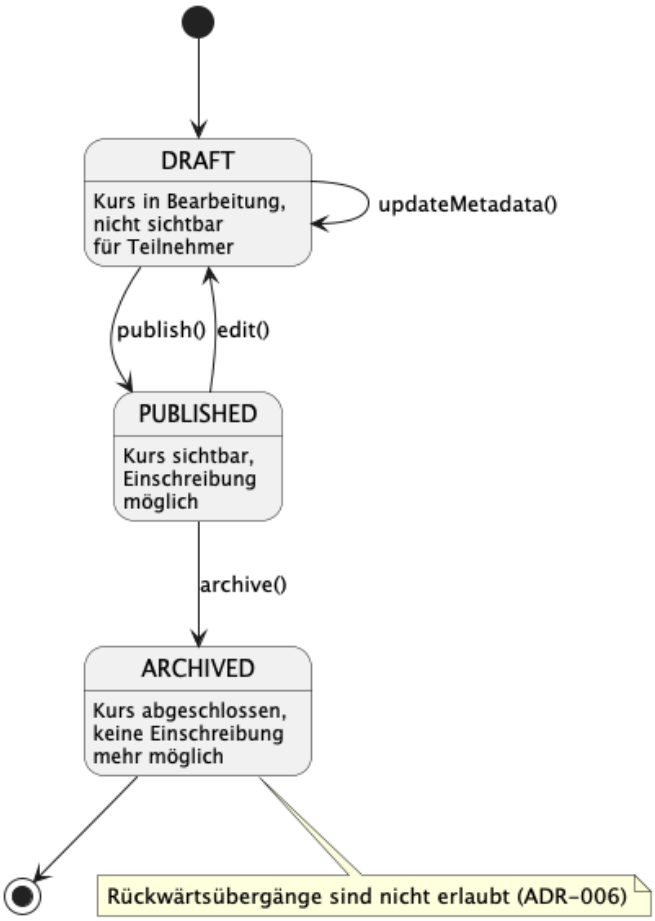


Abbildung 5: *Zustandsübergangsdiagramm:* `CourseStatus` *mit den drei Zuständen DRAFT, PUBLISHED und ARCHIVED. Rückwärtsübergänge sind nicht vorgesehen.*

5.3 Ebene 2 – Bounded Context: `scheduling`

Der `scheduling`-Service ist für die zeitliche und organisatorische Planung von Kursdurchführungen verantwortlich. Er referenziert Kurse aus `catalog` ausschliesslich über die `CourseId` (Anti-Corruption Layer) und enthält die kritische Logik zur Vermeidung von Trainer-Doppelbuchungen. Abbildung 6 zeigt seine internen Komponenten – man beachte insbesondere den ACL-Baustein rechts, der die Entkopplung vom `catalog`-Domänenmodell sicherstellt.

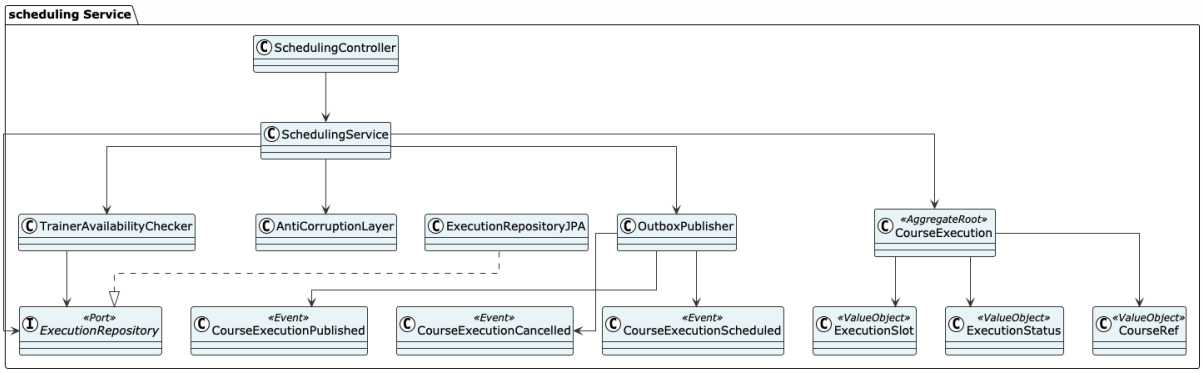


Abbildung 6: *Ebene 2 – UML-Klassendiagramm: Interne Bausteine des* `scheduling`*-Service inkl. Anti-Corruption Layer zum* `catalog`*-Context.*

Wichtige Klassen / Aggregate:

- **CourseExecution** (Aggregate Root): `CourseRef` (nur ID – Anti-Corruption Layer), Startdatum, Enddatum, Ort, Kapazität, `ExecutionStatus`, Trainer.
- **SchedulingService**: Prüft Trainer-Verfügbarkeit und verhindert Überschneidungen. Die Prüflogik verwendet transaktionale Locks (SELECT... FOR UPDATE) zur Race-Condition-Prävention (Details siehe Laufzeitsicht, Kap. 6).
- **ExecutionSlot**: Value Object für einen Zeitblock einer Durchführung.

Domain Events:

- `CourseExecutionScheduled` – Durchführung wurde geplant
- `CourseExecutionPublished` – Durchführung wurde veröffentlicht
- `CourseExecutionCancelled` – Durchführung wurde abgesagt

Fallback-Verhalten bei Ausfall des `catalog`-Service Der `scheduling`-Service ist abhängig vom `catalog`-Service, um Kursnamen zu laden. Sollte `catalog` ausfallen oder nicht erreichbar sein, darf dies die Kernfunktionalität von `scheduling` nicht blockieren. Daher implementiert `scheduling` ein intelligentes Fallback-Verhalten:

- **Normale Situation**: `scheduling` ruft `catalog` auf und erhält den Kursnamen (z. B. "Java Grundlagen").
- **Ausfall von `catalog`**: Der `scheduling`-Service erkennt den Fehler über einen **Circuit Breaker** (Resilience4j). Statt die Anfrage zu blockieren oder zu fehlschlagen, gibt `scheduling` die Durchführungsdaten mit einem Platzhalter zurück: `[Kursinformation nicht verfügbar]`. Die eigentlichen Daten (Termin, Ort, Trainer) bleiben korrekt.
- **Circuit Breaker Logik**: Nach 5 Fehlern innerhalb von 10 Sekunden geht der Circuit Breaker in den Zustand "OPEN". Weitere Anfragen an `catalog` werden sofort abgelehnt, ohne zu warten. Nach 30 Sekunden wechselt der Circuit Breaker in den Zustand "HALF-OPEN" und versucht einen Testaufruf. Ist dieser erfolgreich, geht der Circuit Breaker zurück in den Zustand "CLOSED".

Resultat: Benutzer sehen zwar, dass Kursinformationen temporär nicht verfügbar sind, können aber weiterhin auf Durchführungstermine zugreifen. Dies ist ein Beispiel für *Graceful Degradation*: Das System funktioniert mit reduzierter Funktionalität weiter, statt komplett auszufallen.

5.4 Ebene 2 – Bounded Context: `registration`

Der `registration`-Service ist der **kritischste Bounded Context** des Systems: Er verarbeitet Einschreibungen, verwaltet Wartelisten und muss unter allen Umständen verhindern, dass mehr Teilnehmer eingeschrieben werden als Plätze vorhanden sind – auch bei hoher gleichzeitiger Last. Aus diesem Grund ist er der einzige Service, für den eine Ebene-3-Sicht dokumentiert wird (siehe Abschnitt 5.4.1).

Abbildung 7 zeigt die internen Komponenten inkl. der beiden kritischen Bausteine `CapacityChecker` (Race-Condition-Schutz) und `OutboxPublisher` (garantierte Event-Publikation).

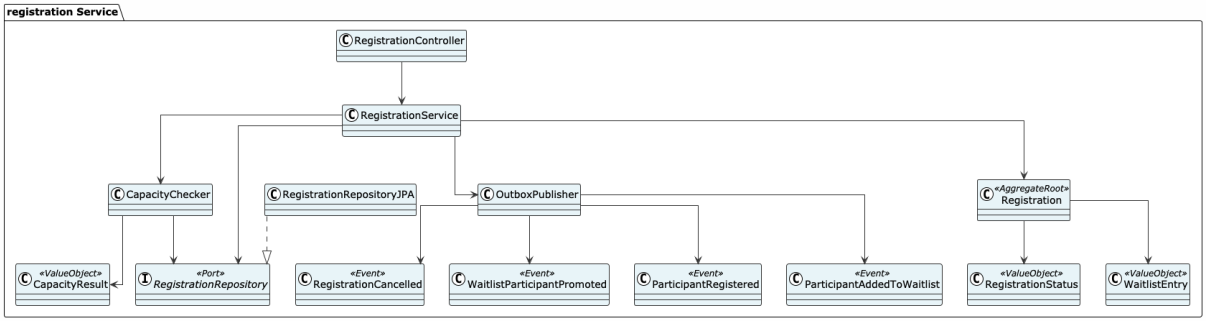


Abbildung 7: *Ebene 2 – UML-Klassendiagramm: Interne Bausteine des* `registration`*-Service inkl. CapacityChecker, OutboxPublisher und Anbindung an den Message Broker.*

Wichtige Klassen / Aggregate:

- **Registration** (Aggregate Root): `registrationId` (UUID), `executionId` (ExecutionRef), `participantId`, `status` (REGISTERED | CANCELLED), `registeredAt`, `version` (Long, Optimistic Locking).
- **CapacityChecker**: Prüft Kapazität via `SELECT ... FOR UPDATE` / Optimistic Locking. Die detaillierte Logik ist in Abschnitt 5.4.1 dokumentiert.
- **WaitlistEntry**: Value Object mit Position (Integer) und Zeitstempel.
- **OutboxPublisher**: Schreibt Domain Events in die `outbox`-Tabelle innerhalb derselben DB-Transaktion (Transactional Outbox Pattern, ADR-004).

Domain Events:

- `ParticipantRegistered` – Teilnehmer wurde eingeschrieben
- `ParticipantAddedToWaitlist` – Teilnehmer zur Warteliste hinzugefügt
- `RegistrationCancelled` – Einschreibung wurde storniert
- `WaitlistParticipantPromoted` – Wartelisten-Teilnehmer wurde befördert

5.4.1 Ebene 3 – CapacityChecker und OutboxPublisher (Whitebox)

Die folgenden beiden Bausteine sind die architekturkritischsten Komponenten des gesamten Systems. Der `CapacityChecker` stellt sicher, dass niemals mehr Einschreibungen akzeptiert werden als Plätze vorhanden sind – auch wenn Hunderte von Anfragen gleichzeitig eintreffen. Der `OutboxPublisher` garantiert, dass jedes Domain Event genau dann publiziert wird, wenn die zugehörige Geschäftstransaktion erfolgreich war (kein Dual-Write-Problem).

CapacityChecker – Interne Logik Der `CapacityChecker` schützt sich vor Race Conditions durch eine Kombination aus pessimistischen Locks und Optimistic Locking:

```
1. Lese Registration-Aggregate mit version-Feld:
   SELECT * FROM registrations
   WHERE execution_id = ? FOR UPDATE
   ODER: Lese via JPA @Version (Optimistic Locking)

2. Pruefe: currentCapacity > 0
   -> Einschreibung erlaubt
   -> Einschreibung abgelehnt (auf Warteliste)

3. Dekrementiere Kapazitaet; persistiere
```

```
   Registration-Entity

4. Bei OptimisticLockingFailure: Retry (max. 3x)
   mit exponentieller Backoff
```

Szenario: Zwei Anfragen treffen gleichzeitig ein und versuchen, die letzte verfügbare Kapazität zu buchen. Die erste Anfrage erwirbt den Lock via `FOR UPDATE`, dekrementiert die Kapazität und committed. Die zweite Anfrage wartet auf den Lock, liest dann die aktualisierte (nun leere) Kapazität und lehnt die Einschreibung ab – Teilnehmer wird auf Warteliste gesetzt.

OutboxPublisher – Interne Logik Der `OutboxPublisher` implementiert das Transactional Outbox Pattern (ADR-004) zur garantierten Event-Publikation:

```
1. Domain Event serialisieren (JSON)

2. INSERT INTO outbox
   (event_type, payload, status=PENDING, created_at)
   INNERHALB derselben JDBC-Transaktion wie
   die Business-Logik

3. Commit der Transaktion:
   - Entweder: Beide (Registration + Event) werden
     persistent
   - Oder: Beide werden rollback (Atomarität)

4. Separater Background-Thread (Polling):
   - Liest PENDING-Events aus der outbox-Tabelle
   - Sendet an RabbitMQ
   - Update Status -> status=SENT (oder status=FAILED)

5. Dead-Letter-Queue (DLQ):
   - Sollte ein Event nach 5 Retry-Versuchen
     noch nicht gesendet sein, landet es in der DLQ
   - Manuelle Intervention erforderlich
```

Vorteil gegenüber Dual-Write: Früher hätte man zuerst die Registration in die DB schreiben und dann das Event an RabbitMQ senden müssen. Bei Fehler zwischen diesen beiden Operationen entsteht Inkonsistenz. Mit dem Outbox Pattern ist beides eine atomare Operation.

5.5 Ebene 2 – Bounded Context: `notification`

Der `notification`-Service ist der einfachste Service und hat bewusst keine fachliche Logik: Er konsumiert Domain Events von anderen Services und versendet E-Mails. Diese Entkopplung stellt sicher, dass E-Mail-Ausfälle den Kernprozess (Einschreibung, Planung) nicht blockieren. Abbildung 8 zeigt die internen Bausteine und deren Zusammenspiel.

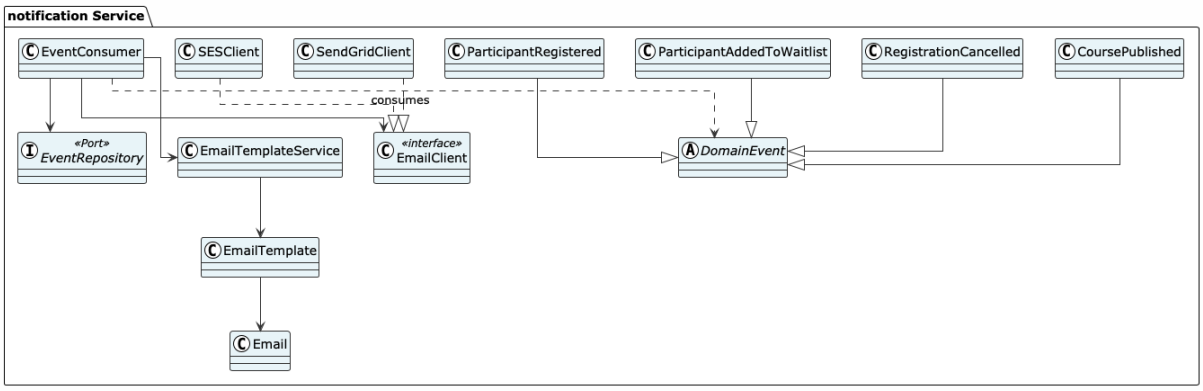


Abbildung 8: *Ebene 2 – UML-Klassendiagramm: Interne Bausteine des* `notification`*-Service.*

Wichtige Klassen:

- **EventConsumer**: Liest Domain Events aus dem Message Broker (RabbitMQ) und delegiert an `EmailTemplateService`.
- **EmailTemplateService**: Wählt das passende Template basierend auf Event-Typ und rendert die E-Mail mit Event-Daten.
- **TemplateRegistry**: Zentrales Verzeichnis aller E-Mail-Templates (z. B. Bestätigung, Warteliste, Stornierung).
- **EmailClient** (Interface): Abstrahiert den konkreten E-Mail-Anbieter. Implementierungen: `SESClient` (Amazon SES) oder `SendGridClient` (SendGrid).
- **Email**: Value Object mit Empfänger, Betreff, HTML-Body.

Domain Events (konsumiert):

- `ParticipantRegistered` – Bestätigungsmail versenden
- `ParticipantAddedToWaitlist` – Wartelistenbestätigung versenden
- `RegistrationCancelled` – Stornierungsbestätigung versenden
- `CoursePublished` – Ankündigung an Interessenten versenden

Event-getriebener Versand Der `notification`-Service ist vollständig *event-getrieben*: Er hat keine eigene Geschäftslogik, sondern reagiert nur auf Events, die von anderen Services publiziert werden. Dies hat mehrere Vorteile:

- **Entkopplung**: Andere Services müssen nicht wissen, dass E-Mails versendet werden.
- **Fehlertoleranz**: Sollte der `notification`-Service ausfallen, blockiert dies nicht die Einschreibung oder Planung.
- **Skalierbarkeit**: E-Mail-Versand ist zeitaufwändig. Durch asynchrone Events kann der Service unabhängig skaliert werden.
- **Wartbarkeit**: Template-Änderungen erfordern keine Änderungen an anderen Services.

5.6 Ebene 2 – Bounded Context: `identity`

Der `identity`-Service ist ein dünner Adapter zwischen EduPlanner und dem externen Identity Provider (Keycloak via OIDC). Er verwaltet keine Passwörter oder Credentials, sondern delegiert

alle Authentifizierung an den IdP und cached nur Rollen und Berechtigungen. Abbildung 9 zeigt die internen Bausteine und deren Zusammenspiel.

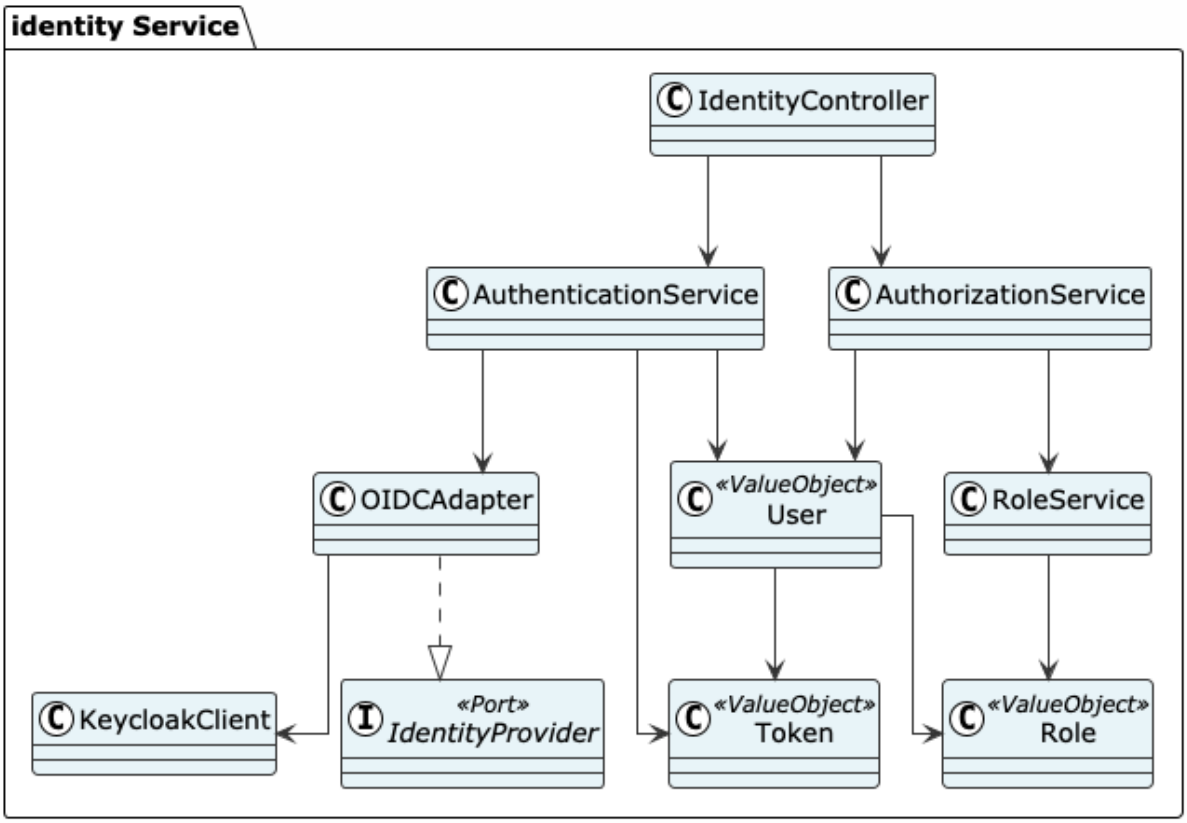


Abbildung 9: *Ebene 2 – UML-Klassendiagramm: Interne Bausteine des `identity`-Service.*

Wichtige Klassen:

- **AuthenticationService**: Orchestriert den Authentifizierungsprozess. Delegiert an `OIDCAdapter` und `TokenValidator`.
- **OIDCAdapter**: Kommuniziert mit dem externen Identity Provider (Keycloak). Implementiert das OpenID Connect Protokoll.
- **TokenValidator**: Validiert eingehende JWT-Tokens auf Signatur, Ablaufdatum und Gültigkeit.
- **AuthorizationService**: Prüft Berechtigungen basierend auf Rollen und Permissions.
- **RoleService**: Managed den Cache von Benutzerrollen. Wird bei jedem Login aktualisiert.
- **AuthorizationInterceptor**: Wird von anderen Services aufgerufen, um Rollen zu prüfen.

Authentifizierung vs. Autorisierung Der `identity`-Service trennt zwei kritische Konzepte:

- **Authentifizierung** ("Wer bist du?"): Erfolgt via OIDC beim externen IdP (Keycloak). Der Service speichert Passwörter *nicht*. Stattdessen erhält der Benutzer einen JWT-Token, der in allen nachfolgenden Anfragen mitgesendet wird.
- **Autorisierung** ("Darfst du das?"): Erfolgt lokal im `identity`-Service. Der Service cached die Rollen des Benutzers (z. B. "Trainer", "Staff", "Admin") und prüft, ob diese die angeforderte

Aktion erlauben (z. B. "Darf dieser Benutzer einen Kurs löschen?").

Sicherheitsaspekte

- **Keine Passwörter im System**: Alle Authentifizierung erfolgt via Keycloak. EduPlanner sieht niemals Passwörter.
- **JWT-Tokens mit Ablaufdatum**: Tokens sind zeitlich begrenzt gültig (z. B. 1 Stunde). Nach Ablauf muss der Benutzer sich neu authentifizieren.
- **Token-Signatur**: Tokens sind mit einem geheimen Schlüssel signiert. Manipulierte Tokens werden erkannt.
- **Role-based Access Control (RBAC)**: Berechtigungen werden granular auf Rollenebene gesteuert. Neue Rollen können im IdP hinzugefügt werden, ohne EduPlanner zu ändern.

Domain Events:

- `UserCreated` – Neuer Benutzer wurde angelegt (Admin oder IdP-Sync)
- `UserRoleAssigned` – Rolle wurde einem Benutzer zugewiesen

Hinweis: Domain Events im `identity`-Context

Diese Events dienen ausschliesslich internen Administrationszwecken (z. B. Audit-Log, Kap. 8.2). Kein anderer Bounded Context konsumiert sie im Kernprozess. Sie erscheinen in der vollständigen Event-Übersicht (Kap. 12.2 und Anhang A) der Vollständigkeit halber.

6 Laufzeitsicht

6.1 Szenario 1: Teilnehmer schreibt sich ein (Happy Path)

Das folgende Szenario beschreibt den Normalablauf einer erfolgreichen Kursanmeldung. Der Ablauf umfasst die Authentifizierung am API Gateway, die Rollenprüfung über den `identity`-Service sowie die transaktionale Persistierung der Registrierung inklusive Outbox-Eintrag. Die anschliessende Event-Publikation und der E-Mail-Versand erfolgen asynchron und sind damit vom eigentlichen Anmeldeprozess entkoppelt.

Vorbedingungen: Gültiger JWT; CourseExecution publiziert und mit freier Kapazität.

1. Teilnehmer sendet `POST /api/v1/registrations` mit `{executionId, userId}` + JWT.
2. API Gateway prüft Token-Signatur; bei Fehler: 401 Unauthorized.
3. Weiterleitung an `registration`-Service.
4. Rollenverifikation via `identity`-Service (PARTICIPANT / STAFF / ADMIN erlaubt).
5. CapacityChecker liest Kapazität (Optimistic Locking).
6. Kapazität > 0: Registration-Aggregate persistiert; Kapazitätszähler dekrementiert.
7. OutboxPublisher schreibt `ParticipantRegistered` in `outbox`-Tabelle (gleiche Transaktion).
8. Antwort: **201 Created** mit Registrierungs-ID.
9. Outbox-Relay publiziert Event an Message Broker (asynchron).
10. `notification`-Service sendet Bestätigungsmail.

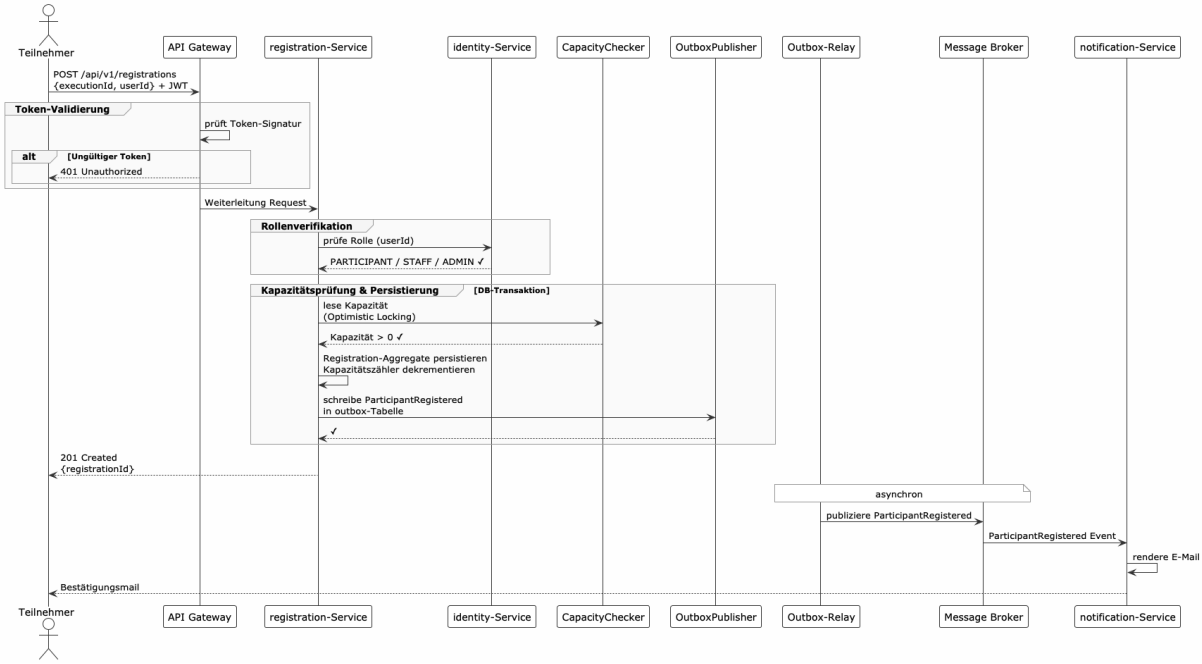


Abbildung 10: *Sequenz Diagramm für Teilnehmer schreibt sich ein (Happy Path)*

6.2 Szenario 2: Kurs ausgebucht – Warteliste

Dieses Szenario behandelt den Fall, dass beim Einschreiben keine freie Kapazität mehr vorhanden ist. Der Ablauf ist identisch mit Szenario 1 bis zur Kapazitätsprüfung; anstelle einer Registrierung

wird ein `WaitlistEntry` mit Zeitstempel und Wartelisten-Position persistiert. Der Teilnehmer erhält eine **202 Accepted**-Antwort sowie eine asynchrone Benachrichtigungsmail mit seiner Position.

Identisch mit Szenario 1 bis Schritt 5, dann:

6. Kapazität = 0: WaitlistEntry mit Zeitstempel und nächster freier Position.
7. Outbox-Event: `ParticipantAddedToWaitlist` (mit Position).
8. Antwort: **202 Accepted** mit Wartelisten-Position.

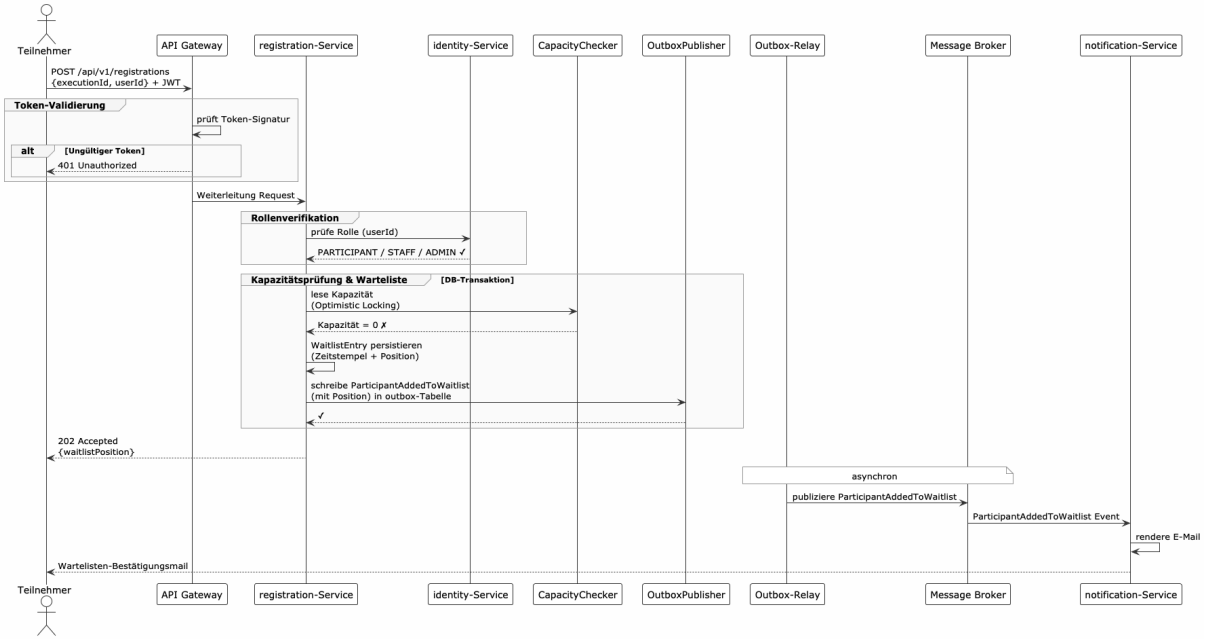


Abbildung 11: *Sequenz Diagramm für Teilnehmer schreibt sich ein, aber keine freien Plätze mehr*

6.3 Szenario 3: Einschreibung stornieren

Dieses Szenario beschreibt die Stornierung einer bestehenden Kursanmeldung. Nach der Ownership-Prüfung – nur eigene Einschreibungen dürfen storniert werden – wird der Status transaktional auf `CANCELLED` gesetzt und die Kapazität wieder freigegeben. Ist die Warteliste nicht leer, rückt der älteste Eintrag automatisch nach; alle Benachrichtigungen erfolgen asynchron über den `notification`-Service.

1. `DELETE /api/v1/registrations/{id}` + JWT.
2. Ownership-Prüfung: Nur eigene Einschreibungen dürfen storniert werden (403 Forbidden sonst).
3. Status → CANCELLED; Kapazität inkrementiert.
4. Outbox-Event: `RegistrationCancelled`.
5. Falls Warteliste nicht leer: ältester Eintrag rückt nach → `WaitlistParticipantPromoted` + `ParticipantRegistered`.
6. Antwort: **204 No Content**.

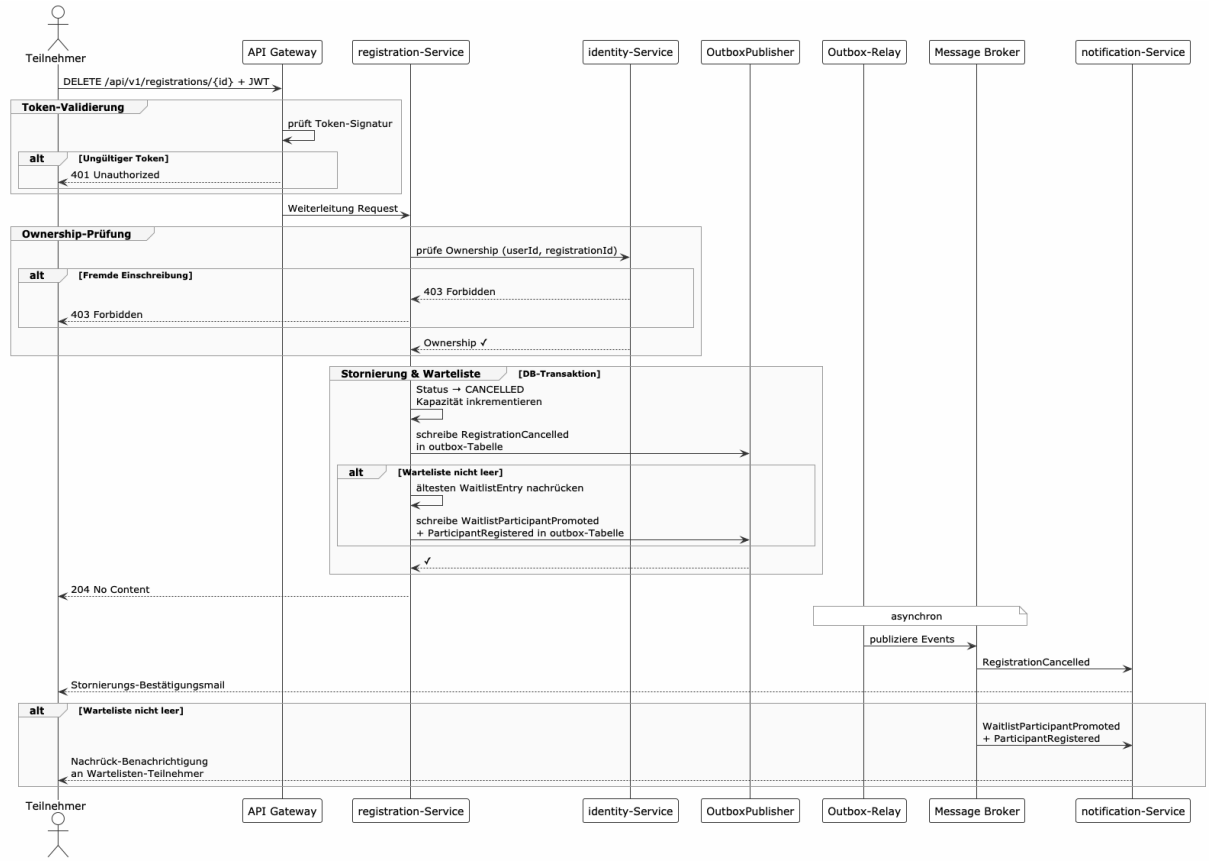


Abbildung 12: *Sequenz Diagramm für die Stornierung einer Kursanmeldung*

6.4 Szenario 4: Admin veröffentlicht einen Kurs

Dieses Szenario beschreibt die Veröffentlichung eines Kurses durch einen Administrator. Der Ablauf ist bewusst schlank gehalten: Nach der Rollenprüfung – ausschliesslich ADMIN-Rolle ist berechtigt – wird der `CourseStatus` transaktional von `DRAFT` auf `PUBLISHED` gesetzt und das `CoursePublished`-Event über die Outbox asynchron publiziert. Eine direkte Teilnehmer-Benachrichtigung ist in diesem Szenario nicht vorgesehen.

1. `PUT /api/v1/courses/{id}/publish` + JWT.
2. ADMIN-Rolle erforderlich (403 Forbidden für alle anderen).
3. `CourseStatus`: DRAFT → PUBLISHED.
4. Outbox-Event: `CoursePublished`.
5. Antwort: **200 OK**.

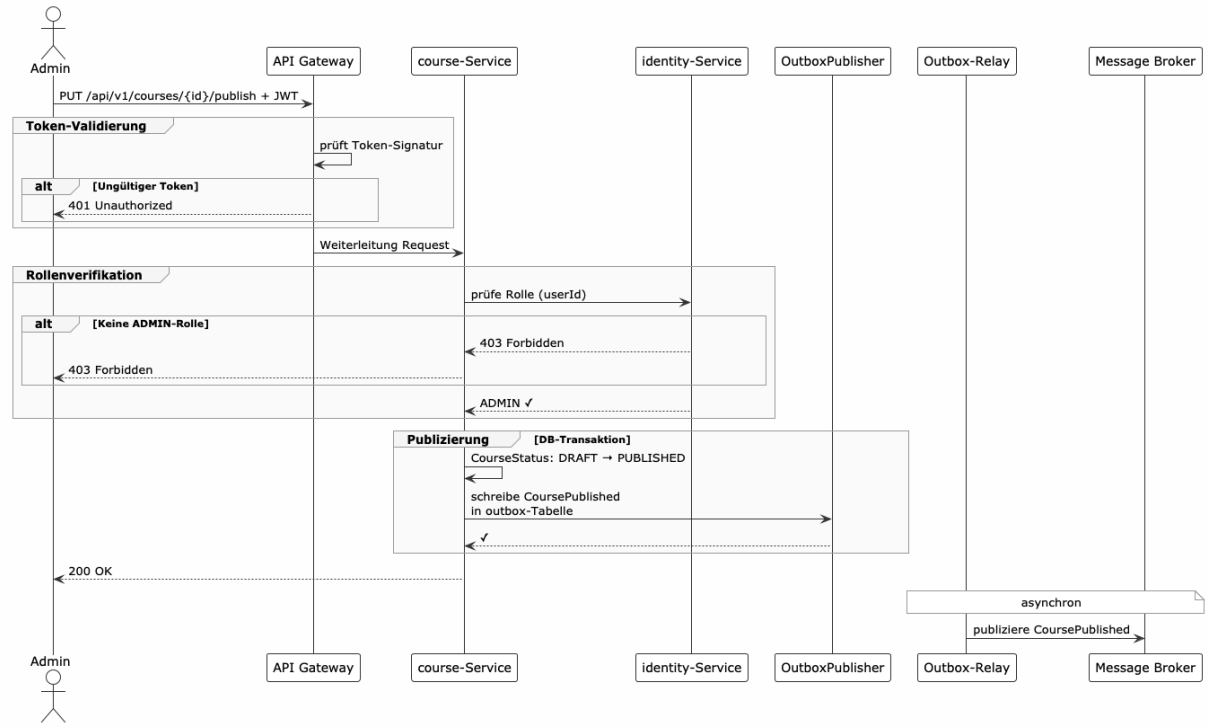


Abbildung 13: *Sequenz Diagramm für Admin veröffentlicht einen Kurs*

6.5 Szenario 5: Cross-Context-Transaktion (Kurs archivieren)

Dieses Szenario illustriert eine kontextübergreifende Transaktion, bei der das Archivieren eines Kurses eine Kaskade von Zustandsänderungen in mehreren Services auslöst. Da keine verteilte Transaktion verwendet wird, koordinieren sich `catalog`, `scheduling`, `registration` und `notification` ausschliesslich über Events – die vollständige Konsistenz wird erst nach ∼2–5 Sekunden erreicht (Eventual Consistency).

1. `PUT /api/v1/courses/{id}/archive` (Admin).
2. `catalog`: Status → ARCHIVED; Event `CourseArchived`.
3. `scheduling` konsumiert → alle Executions → CANCELLED; Events `CourseExecutionCancelled`.
4. `registration` konsumiert → alle Registrations → CANCELLED; Events `RegistrationCancelled`.
5. `notification` versendet Massenstornierungsmails.

Eventual Consistency: Alle Services nach ∼2–5 Sek. konsistent.

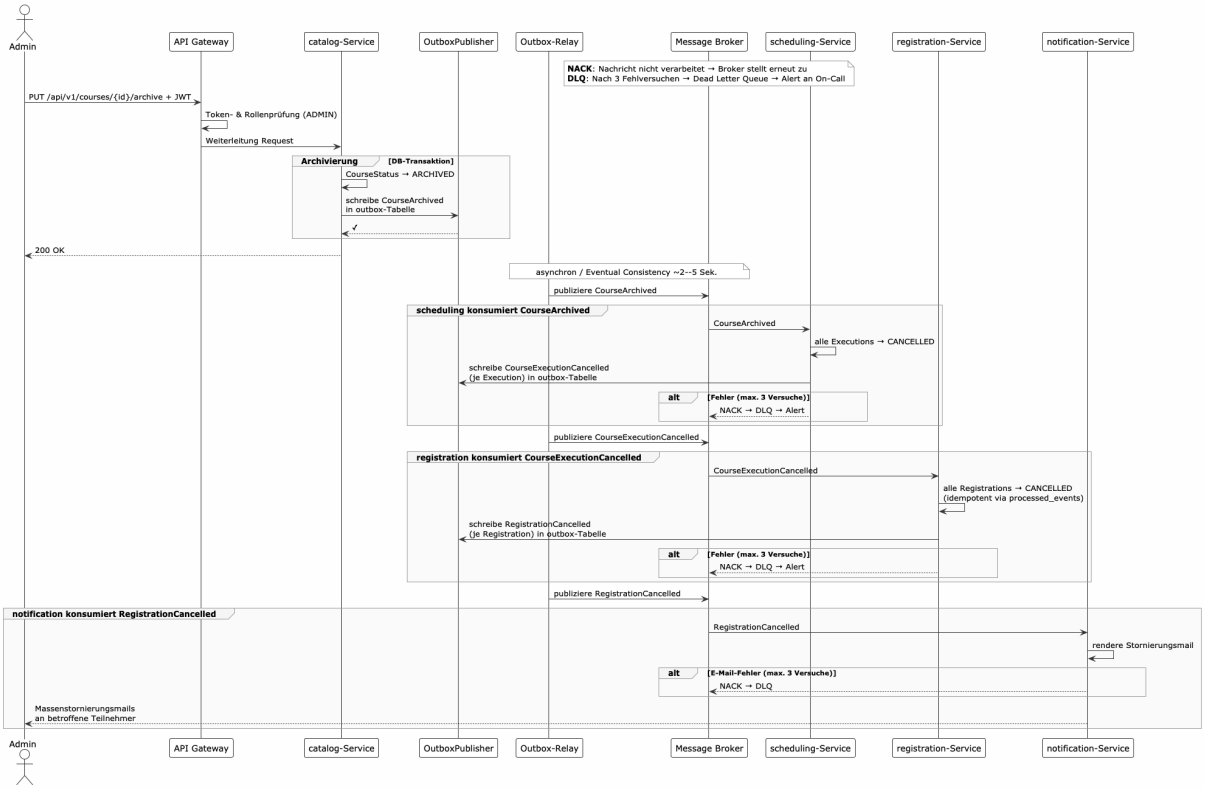


Abbildung 14: *Sequenz Diagramm für Kurs archivieren*

Fehlerbehandlung – symmetrisch für alle Consumer

Bei `registration` *(konsumiert `CourseExecutionCancelled`): Exception im Event-Handler → RabbitMQ stellt erneut zu (at-least-once) → nach 3 Fehlversuchen DLQ → Alert → On-Call-Entwickler. Idempotenter Handler verhindert Doppelstornierungen.*

Bei `scheduling` *(konsumiert `CourseArchived`): Identischer DLQ-Mechanismus – alle Executions werden auf CANCELLED gesetzt, sobald der Event erfolgreich verarbeitet ist. Idempotenz: `executionId` wird in `processed_events`-Tabelle geprüft.*

Bei `notification` *(konsumiert `RegistrationCancelled`): Exception im E-Mail-Provider → Retry (3x) → DLQ. Der Prozess (Einschreibung/Stornierung) ist bereits erfolgreich abgeschlossen.*

7 Verteilungssicht

7.1 Infrastruktur-Übersicht

Die Produktionsumgebung läuft in der Public Cloud von Microsoft Azure mit Kubernetes als Orchestrierungsschicht. Alle Microservices werden als containerisierte Workloads in einem Azure Kubernetes Service (AKS) Cluster betrieben. Die Datenhaltung erfolgt über verwaltete Azure-Dienste (Azure Database for PostgreSQL), womit Betrieb, Patching und Backup-Management an den Cloud-Provider delegiert werden.

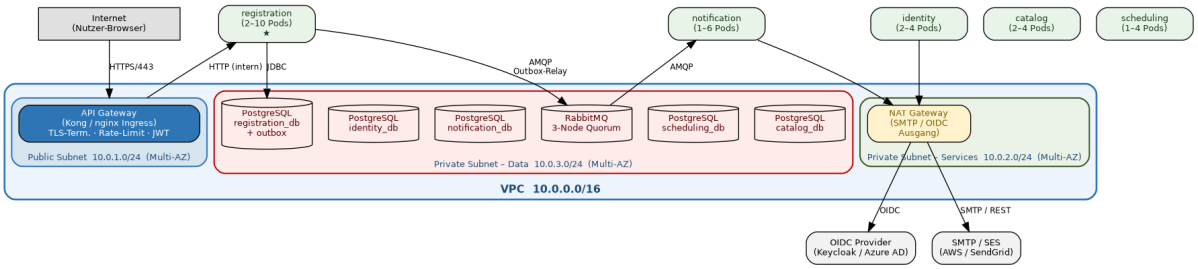


Abbildung 15: *Deployment-Diagramm: Netzwerk-Topologie (Kubernetes / Azure)*

7.1.1 Cloud-Provider-Entscheid (ADR-010)

Der Entscheid fiel zugunsten von **Microsoft Azure** (ADR-010). Ausschlaggebend waren:

- **DSGVO-Compliance**: Azure-Regionen *Switzerland North* (Zürich) und *Switzerland West* (Genf) gewährleisten Datenresidenz in der Schweiz und erfüllen die Anforderungen gemäss DSG / DSGVO.
- **Managed Services**: Azure Database for PostgreSQL Flexible Server bietet automatisches Failover, PITR und integriertes Monitoring ohne Eigenaufwand.
- **Vendor Lock-in**: Minimiert durch den Einsatz cloud-agnostischer Technologien (Kubernetes, PostgreSQL, RabbitMQ) – ein Anbieterwechsel bleibt möglich (vgl. R7).
- **Team-Kenntnisse**: Bestehendes Know-how im Betriebsteam mit Azure-Diensten reduziert den Einarbeitungsaufwand und das operative Risiko.

7.2 Deployment-Umgebungen

Das System durchläuft vier klar voneinander getrennte Umgebungen – von der lokalen Entwicklung bis zum produktiven Betrieb. Jede Umgebung ist auf ihren Zweck zugeschnitten und wird über dieselbe CI/CD-Pipeline versorgt.

Umgebung	Plattform	Beschreibung
Development	Docker Compose	Lokal; alle Services; vereinfachte Konfiguration (kein TLS, lokale DB). Hot-Reload.
Testing	K8s Namespace	Automatisch via CI/CD provisioniert; ephemeral – nach Tests abgebaut.
Staging	AKS Cluster	Produktionsähnlich; Abnahmetests; anonymisierte Prod-Daten.
Production	AKS Cluster (Multi-AZ)	HPA; automatische Failover-Mechanismen; Blue-Green-Deployment.

7.3 Infrastrukturkomponenten

Die folgende Tabelle zeigt alle eingesetzten Infrastrukturkomponenten mit ihrer konkreten Technologiewahl und Konfiguration. Die Kombination aus verwalteten Azure-Diensten und cloud-agnostischen Open-Source-Technologien minimiert den Betriebsaufwand bei gleichzeitig geringem Vendor Lock-in.

Komponente	Technologie	Konfiguration
API Gateway	NGINX Ingress Controller	TLS-Terminierung, Rate-Limiting, JWT-Weiterleitung an Services; alternativ Spring Cloud Gateway für reaktive Routing-Logik
Orchestrierung	Azure Kubernetes Service 1.28+	HPA, Readiness-/Liveness-Probes, Pod Disruption Budgets, Azure Monitor Integration
Datenbanken	Azure Database for PostgreSQL 15 Flexible Server	Je Service eine Instanz (Database-per-Service); PITR 7 Tage; automatische Backups; Failover $<$ 60 Sek.
Message Broker	RabbitMQ 3.12	3-Node-Quorum-Cluster; persistente Queues; DLQ; Management-UI über internen Ingress
Monitoring	Prometheus + Grafana	RED-Pattern (Rate, Errors, Duration); Alerting bei Error-Rate $>$ 1 % und Latenz $>$ 500 ms (p99)
Logging	ELK-Stack	Strukturierte JSON-Logs; Correlation-ID über alle Services; 30 Tage Retention; Index-Rotation täglich
Tracing	OpenTelemetry + Jaeger	Distributed Tracing End-to-End; Sampling-Rate 10 % in Production, 100 % in Staging
CI/CD	GitHub Actions	Pipeline: Build $\rightarrow$ Unit-Tests $\rightarrow$ Integration-Tests $\rightarrow$ Docker-Image $\rightarrow$ AKS-Deploy; automatisches Rollback bei fehlgeschlagenen Probes
Secret Management	Azure Key Vault	Zentrales Secret-Management; Secrets werden zur Laufzeit per CSI-Treiber in Pods gemountet

7.4 Disaster Recovery Plan

Der DR-Plan definiert verbindliche Zielwerte für Wiederherstellungszeit (RTO) und maximal tolerierbaren Datenverlust (RPO)[10] je Infrastrukturkomponente. Die Massnahmen sind auf automatisches Failover ausgerichtet – manuelle Eingriffe bleiben auf Ausnahmefälle beschränkt.

Komponente	RTO	RPO	Strategie
PostgreSQL	15 min	5 min	Auto-Failover zu Read Replica; PITR 7 Tage; wöchentliche Restore-Tests (So, 03:00 UTC).
RabbitMQ	5 min	0 min	3-Node Quorum-Cluster; kein Datenverlust.
AKS Cluster	30 min	N/A	Multi-AZ; Pod Disruption Budgets; Azure Availability Zones.
Application	2 min	N/A	Blue-Green-Deployment; Liveness-Probes; automatisches Rollback.

DR-Testverantwortung

Der DR-Plan auf Systemebene wird halbjährlich als vollständiger Failover-Test in der Staging-Umgebung durchgeführt. Verantwortung: IT-Leiter (Patrick Hari). Protokoll geht an den CISO (Andrea Senn) für die Compliance-Dokumentation.

7.5 Netzwerk-Topologie

Das Netzwerk wird in drei Zonen aufgeteilt, die dem Prinzip der minimalen Exposition folgen: Eingehender Verkehr läuft ausschliesslich über das öffentliche Subnet mit NGINX Ingress; alle Microservices und Datenbankinstanzen sind in privaten Subnetzen ohne direkten Internetzugang betrieben. Ausgehende Verbindungen der Services – etwa für E-Mail-Versand oder OIDC – werden über ein zentrales NAT Gateway[11] geführt.

Netzwerkzone	CIDR	Beschreibung
VNet	10.0.0.0/16	Gesamtes Azure Virtual Network
Public Subnet	10.0.1.0/24 (Multi-AZ)	NGINX Ingress / API Gateway – einziger öffentlicher Zugangspunkt; Azure DDoS Protection Standard
Private (Services)	10.0.2.0/24 (Multi-AZ)	Alle Microservices – kein direkter Internet-Zugang; Network Policies zwischen Services aktiv
Private (Data)	10.0.3.0/24 (Multi-AZ)	PostgreSQL-Instanzen und RabbitMQ – nur aus Service-Subnet erreichbar; Private Endpoints
NAT Gateway	Azure NAT Gateway	Ausgehende Verbindungen der Services (SMTP via SendGrid, OIDC-Endpunkte)

[10] RTO (Recovery Time Objective): maximale Zeitspanne bis zur Wiederherstellung. RPO (Recovery Point Objective): maximaler tolerierter Datenverlust in Zeit; RPO = 0 min bedeutet kein Datenverlust.

[11] NAT Gateway (Network Address Translation): übersetzt private IP-Adressen der Services auf eine öffentliche IP-Adresse für ausgehende Verbindungen – ohne eingehenden Zugriff aus dem Internet zu erlauben.

8 Querschnittliche Konzepte

Dieser Abschnitt beschreibt Konzepte und Entscheidungen, die nicht einem einzelnen Service zugeordnet sind, sondern das gesamte System betreffen. Sie bilden die gemeinsame technische Basis für Sicherheit, Betrieb, Datenhaltung und Qualitätssicherung.

8.1 Sicherheitskonzept

Das Sicherheitskonzept basiert auf dem Prinzip der minimalen Rechte: Authentifizierung und Autorisierung sind konsequent voneinander getrennt, zentral über den `identity`-Service koordiniert und in jedem Service durchgesetzt.

8.1.1 Authentifizierung

Die Authentifizierung ist vollständig an einen externen OIDC-Provider delegiert. Das System selbst speichert keine Passwörter – es vertraut ausschliesslich signierten JWT-Tokens, deren Gültigkeit am API Gateway geprüft wird.

- OIDC / OAuth 2.0: Authentifizierung vollständig extern (z. B. Keycloak, Azure AD).
- JWT-Access-Token (RS256-signiert, 15 Min.), Refresh-Token (8 h).
- API Gateway validiert JWT-Signatur via JWKS-Endpoint; ungültig → 401.
- Silent Refresh: Frontend erneuert Token automatisch.

8.1.2 Autorisierung (RBAC)

Die Zugriffskontrolle erfolgt rollenbasiert (RBAC). Rollen werden als Claims im JWT übertragen und von jedem Service eigenständig ausgewertet – ein zentraler Autorisierungs-Roundtrip entfällt damit pro Request. Der folgende Payload-Ausschnitt zeigt die Struktur eines dekodiertes JWT:

```
{
  "sub": "user-uuid-123",
  "roles": ["PARTICIPANT"],
  "locale": "de",
  "tenant_id": "skillbridge",
  "exp": 1700000900
}
```

E-Mail ist im OIDC-Token vorhanden, wird aber nie geloggt (PII-Schutz, Kap. 8.6).

Die folgende Tabelle zeigt, welche Aktionen den einzelnen Rollen erlaubt sind:

Aktion	Admin	Staff	Trainer	Participant
Kurs erstellen	✓	✓	✗	✗
Kurs veröffentlichen	✓	✗	✗	✗
Kurs archivieren	✓	✗	✗	✗
Durchführung planen	✓	✓	✗	✗
Sich einschreiben	✓	✓	✓	✓
Eigene Stornierung	✓	✓	✓	✓
Alle Einschreibungen sehen	✓	✓	✗	✗
Nutzer verwalten	✓	✗	✗	✗

RBAC – Einschreiberecht für TRAINER

Die Berechtigung „Sich einschreiben" für die Rolle TRAINER ist bewusst gewählt: Trainer können Weiterbildungsangebote ihrer Kollegen als Teilnehmer besuchen (Beispiel: ein Java-Trainer schreibt sich in einen Python-Kurs ein). Diese Rollenakkumulation ist im `identity`-Service durch die Multi-Rollen-Unterstützung abgebildet (`"roles: ["TRAINER", "PARTICIPANT"]` im JWT möglich). Falls ein Kursanbieter dies nicht wünscht, kann die Berechtigung per Konfiguration deaktiviert werden.

Die folgende Tabelle beschreibt die eingesetzten Resilience-Muster, die das System gegenüber transienten Fehlern und Kaskadenausfällen absichern:

Muster	Beschreibung
Circuit Breaker	Resilience4j: 5 Fehler in 10 s → Open; Half-Open nach 30 s.
Retry + Exp. Backoff	Bis zu 3×; Backoff: 100 ms / 500 ms / 2 s.
Dead-Letter-Queue	Nach 3 Fehlversuchen; Alerting bei DLQ-Tiefe > 0.
Graceful Degradation	Ausfall von `notification` blockiert nie die Einschreibung.
Idempotente Handler	EventId-Prüfung in `notification_log`; Duplikate werden ignoriert.
Timeout	Connect-Timeout 2 s; Read-Timeout 5 s.
Bulkhead	Separate Thread-Pools für `identity`- und `catalog`-Aufrufe.

Akzeptiertes Restrisiko: Kein mTLS in v1.0 (TS1)

In v1.0 läuft die Service-zu-Service-Kommunikation innerhalb des Kubernetes-Clusters über **unverschlüsseltes HTTP**. Ein Angreifer mit Zugang zum internen Netz (z. B. durch einen kompromittierten Pod) könnte den Datenverkehr zwischen Services mitlesen.

Warum ist das für v1.0 tolerierbar?

- Das interne Kubernetes-Netzwerk ist durch **Network Policies** segmentiert: Jeder Service kommuniziert nur mit explizit erlaubten Peers.
- Der Perimeter ist durch TLS am API Gateway gesichert; kein externer Traffic erreicht die Services direkt.
- Das Betriebsteam hat in v1.0 kein ausreichendes Know-how für Sidecar-Proxies und Zertifikatsrotation (Budget- und Zeitrahmen MVP).

Unter welchen Bedingungen muss nachgebessert werden?

- Vor Go-Live eines zweiten Tenants (Multi-Tenancy v2.0, ADR-008): Datentrennung auf Netzwerkebene wird dann zwingend.
- Bei einem Security-Audit-Befund, der dieses Muster als Critical/High einstuft (Kap. 1.1, Go-Live-Kriterium).
- Spätestens mit Einführung von v2.0: Linkerd ersetzt das fehlende mTLS vollständig (ADR-007, TS1).

Dieses Risiko ist als technische Schuld **TS1** erfasst und im Tilgungsplan verankert (Kap. 11).

8.2 Logging und Observability

Observability ist als Querschnittsanforderung in allen Services einheitlich umgesetzt. Strukturierte Logs, verteiltes Tracing und Prometheus-Metriken bilden zusammen die Grundlage für Fehleranalyse, Alerting und Kapazitätsplanung im Betrieb.

Aspekt	Beschreibung
Strukturiertes Logging	JSON-Format (Logback + SLF4J): `timestamp, level, service, correlationId, message, exception`.
Correlation-ID	UUID (`X-Correlation-ID`) wird als HTTP-Header propagiert und in alle Logs geschrieben.
PII-Schutz	E-Mail und Name werden vor dem Logging maskiert (Log-Scrubbing-Middleware).
Metriken	Prometheus (RED-Muster); exportiert via `/actuator/prometheus`.
Distributed Tracing	OpenTelemetry; Traces in Jaeger; `traceparent`-Header-Propagation.
Alerting	Error-Rate > 1 %, P99-Latenz > 1 s, Queue-Tiefe > 500, DLQ > 0, DB-Pool > 90 %, SSL-Expiry < 30 Tage.

8.3 Datenpersistenz

Jeder Service verwaltet seine Daten vollständig autonom in einer eigenen PostgreSQL-Instanz. Kein Service greift direkt auf die Datenbank eines anderen zu – Datenaustausch erfolgt ausschliesslich über Events. Dieses Muster garantiert lose Kopplung und ermöglicht unabhängige Schema-Migrationen.

Konzept	Beschreibung
Database per Service	Eigene PostgreSQL-DB je BC; kein Cross-Service-Zugriff.
Optimistic Locking	`version`-Feld im Aggregate; Konflikt → 409 Conflict + Client-Retry.
Transactional Outbox	Events atomar mit Domänenänderung; Relay-Prozess publiziert asynchron (vgl. ADR-009).
Schema-Migration	Flyway; versionierte SQL-Skripte; idempotent und rückwärtskompatibel.
Backups	Täglich; PITR 7 Tage; wöchentliche Restore-Tests.

8.4 API-Design-Richtlinien

Alle Services folgen denselben API-Design-Richtlinien, um eine konsistente Schnittstelle für Clients und Service-zu-Service-Kommunikation sicherzustellen. Grundlage ist ein **Contract-First-Ansatz** (ADR-011): Die OpenAPI-Spezifikation (`openapi/eduplanner-api.yaml`) ist die *einzige Quelle der Wahrheit* – Interfaces und Server-Stubs werden daraus generiert, nicht umgekehrt. Die vollständige Spezifikation liegt in Anhang D.

8.4.1 Contract-First-Workflow

Im Contract-First-Ansatz wird die Reihenfolge gegenüber Code-First umgekehrt: Zuerst entsteht der Vertrag, dann der Code.

1. **Spec schreiben:** Die Datei `openapi/eduplanner-api.yaml` wird manuell gepflegt und in Git versioniert. Sie ist der einzige Ort, an dem API-Änderungen initiiert werden.

2. **Stubs generieren:** Der `openapi-generator-maven-plugin` erzeugt beim Build automatisch Spring-Boot-Interfaces (Server-Stubs) aus der Spec:

```
<!-- pom.xml (Auszug) -->
<plugin>
  <groupId>org.openapitools</groupId>
  <artifactId>openapi-generator-maven-plugin</artifactId>
  <configuration>
    <inputSpec>
      ${project.basedir}/../../openapi/eduplanner-api.yaml
    </inputSpec>
    <generatorName>spring</generatorName>
    <configOptions>
      <interfaceOnly>true</interfaceOnly>
      <useSpringBoot3>true</useSpringBoot3>
      <useTags>true</useTags>
    </configOptions>
  </configuration>
</plugin>
```

3. **Implementieren:** Das Dev-Team implementiert die generierten Interfaces (`CoursesApi`, `RegistrationsApi` usw.). Der Compiler erzwingt Konformität – eine Abweichung vom Vertrag führt zu einem Build-Fehler.
4. **Swagger UI bereitstellen:** Jeder Service stellt die interaktive Swagger UI unter `/swagger-ui/index.html` bereit (via `springdoc-openapi-starter-webmvc-ui`). Die zugrundeliegende YAML-Datei ist unter `/v3/api-docs.yaml` abrufbar.
5. **CI-Validierung:** Die Pipeline validiert die Spec vor jedem Merge mit `vacuum` (Linting) und `openapi-generator` (Generierbarkeit). Ein ungültiger Vertrag blockiert den Build.

Contract-First – Vorteile für EduPlanner

Frühe Einigung: *Frontend- und Backend-Team können parallel entwickeln, sobald der Vertrag feststeht – kein Warten auf funktionierende Implementierungen.*

Konsistenz: *Generierte Stubs und DTOs sind immer synchron mit der Spec. Manuelle Abweichungen sind nicht möglich.*

Contract Testing (TS2): *Pact-Tests validieren, dass Consumer und Provider denselben Vertrag einhalten (Tilgung Sprint 8, Kap. 11.2).*

8.4.2 API-Design-Regeln

- **RESTful:** Ressourcenorientierte URLs, korrekte HTTP-Verben (GET / POST / PUT / DELETE).
- **Versionierung:** `/api/v1/...` – Breaking Changes führen zu einer neuen Major-Version (`/api/v2/...`). Additive Änderungen sind rückwärtskompatibel.
- **Swagger UI:** Erreichbar unter `/swagger-ui/index.html` pro Service. Zugriffsschutz in Produktion via API-Gateway-Policy (nur internes Netz).
- **Fehlerantworten:** RFC 7807 Problem Details (`application/problem+json`) mit `type`, `title`, `status`, `detail`, `instance` und optionalem `errors`-Array für Validierungsfehler.
- **Pagination:** Cursor-basiert (`?cursor=...&limit=20`, max. 100). Kein Offset-Paging (inkonsistent bei hoher Schreiblast).
- **Idempotency-Key:** Optionaler `Idempotency-Key`-Header (UUID) bei allen POST-Endpunkten – verhindert Doppeleinschreibungen bei Netzwerkfehlern (vgl. Kap. 5.4).
- **Authentifizierung:** RS256-signierter JWT Bearer Token; Validierung am API Gateway; Services vertrauen dem weitergeleiteten Header (Defense in Depth via lokales RBAC).

Beispiel: RFC 7807 Fehlerantwort

```
{
  "type":     "https://eduplanner.skillbridge.ch/problems/validation-error
      ",
  "title":    "Validation Error",
  "status":   400,
  "detail":   "Das Feld 'title' darf nicht leer sein.",
  "instance": "/api/v1/courses",
  "errors":   [{ "field": "title", "message": "Darf nicht leer sein." }]
}
```

8.5 Teststrategie (Ebene 1–3)

Die Teststrategie folgt der Testpyramide: Schnelle, isolierte Unit-Tests bilden die Basis; Integration- und Contract-Tests sichern das Zusammenspiel der Komponenten; E2E- und Performance-Tests validieren das Systemverhalten unter realistischen Bedingungen. Security- und Chaos-Tests ergänzen die Strategie um Robustheit und Angriffssicherheit.

Testtyp	Umfang	Ziel	Tool / Tooling
Unit Tests	Einzelne Klassen / Aggregate	Functional Correctness; Business-Logik-Abdeckung > 80 %	JUnit 5, Mockito.
Integration Tests	Slice (Controller → Repo)	Prüfung DB-Interaktion, JPA-Mappings, JSON-Serialisierung	Testcontainers (PostgreSQL).
Contract Tests	Service-Schnittstellen (CDC)	API-Kompatibilität zwischen catalog/-sched/reg (Tilgung TS2)	Pact (geplant ab Sprint 8).
E2E Tests	Krit. Journeys	Staging-Deploy bricht bei Fehler	Cypress; Einschreibung, Stornierung, Veröffentlichung.
Performance	P95 < 500 ms	Staging-Alert bei Überschreitung	k6, 1 000 concurrent users, wöchentlich.
Chaos Engineering	Wöchentlich	Ergebnisse im Wiki	Pod-Kills, Network-Delays (Chaos Mesh).
Security Tests	Vor Release	Release blockiert bei Critical/High	OWASP ZAP, Trivy, Snyk.

9 Architekturentscheidungen (ADRs)

Architecture Decision Records (ADRs)[12] dokumentieren wichtige Architekturentscheidungen mit Kontext, Alternativen und Konsequenzen. Jeder ADR ist unveränderlich – er wird nicht überschrieben, sondern durch einen neuen ADR abgelöst oder ergänzt. Die nachfolgenden ADRs decken alle in diesem Dokument referenzierten Entscheidungen vollständig ab.

9.1 ADR-001: Microservices entlang der Bounded Contexts

Feld	Inhalt
Status	Akzeptiert – 2026-01-20
Qualitätsziele	Maintainability (primär), Reliability-Availability, Performance Efficiency
Entscheidungstreiber	Unabhängige Deploybarkeit der Teams; unterschiedliche Skalierungsbedarfe je BC; langfristige Wartbarkeit durch klare fachliche Grenzen (DDD).
Kontext	5 klar abgrenzbare fachliche Domänen mit unterschiedlichen Änderungsraten, Skalierungsbedarfen und Team-Verantwortlichkeiten.
Entscheidung	Jeder Bounded Context als eigenständiger Microservice mit eigenem Repository und eigener Datenbank.
Alternativen	(1) *Modularer Monolith* – ernsthaft evaluiert: geringere Betriebskomplexität, einfachere lokale Entwicklung, ACID-Transaktionen über Modulgrenzen. Für ein 5–8-köpfiges Team und einen 6-Monats-MVP ist dies die objektiv einfachere Wahl. Abgelehnt wegen (a) unterschiedlicher Skalierungsbedarfe: `registration` muss zur Spitzenzeit unabhängig skalieren; (b) Erfahrung zeigt, dass Modulgrenzen im Monolithen mit der Zeit erodieren ("Big Ball of Mud"); (c) Lernziel: Microservices realitätsnah dokumentieren. Fallback explizit geplant: falls Betriebsaufwand das Team überfordert, Rückbau auf modularen Monolithen (Risiko R5). (2) Zwei Services – abgelehnt: fachliche Grenzen verwischen.
Konsequenzen	(+) Unabhängige Deployments. (+) Team-Autonomie. (+) Separate Skalierbarkeit. (-) Erhöhte Betriebskomplexität. (-) Eventual Consistency.

[12] *Architecture Decision Record (ADR)*: Ein ADR ist ein kurzes, strukturiertes Dokument, das eine einzelne, architekturrelevante Entscheidung samt Kontext, betrachteten Alternativen und Begründung festhält. Das Format geht auf Michael Nygard zurück (https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions).

9.2 ADR-002: Asynchrone Kommunikation via Message Broker

Feld	Inhalt
Status	Akzeptiert – 2026-01-22
Qualitätsziele	Reliability-Availability (primär), Maintainability, Functional Correctness
Entscheidungstreiber	Resilienz bei Teilausfällen; Entkopplung von Sender und Empfänger; kein Kaskadenausfall durch synchrone Abhängigkeiten.
Entscheidung	Domain Events über RabbitMQ (Pub/Sub).
Alternativen	(1) Direkter REST-Aufruf – abgelehnt: starke Kopplung, Kaskadenfehler. (2) Polling der DB – abgelehnt: ineffizient. (3) Kafka – zurückgestellt: Overkill für aktuelles Volumen.
Konsequenzen	(+) Lose Kopplung. (+) Resilienz. (-) Eventual Consistency. (-) Zusätzliche Infrastruktur.

9.3 ADR-003: Externer Identity Provider

Feld	Inhalt
Status	Akzeptiert – 2026-01-25
Qualitätsziele	Security (primär), Maintainability
Entscheidungstreiber	Kein eigenes Passwort-Management (Sicherheitsrisiko); MFA und SSO ohne Eigenentwicklung; DSGVO-konforme Datenhaltung beim IdP.
Entscheidung	Externer OIDC-konformer IdP (z. B. Keycloak). EduPlanner verwaltet keine Passwörter.
Alternativen	(1) Eigener Auth-Service – abgelehnt: hohes Sicherheitsrisiko. (2) Auth0/Okta – in Evaluierung für v2.0.
Konsequenzen	(+) Security-Best-Practices, MFA, SSO, DSGVO. (-) Externe Abhängigkeit (R3).

9.4 ADR-004: Transactional Outbox für zuverlässige Events

Feld	Inhalt
Status	Akzeptiert – 2026-02-01
Qualitätsziele	Functional Correctness (primär), Reliability-Availability
Entscheidungstreiber	Atomizität zwischen DB-Schreibvorgang und Event-Publikation; Vermeidung von Dual-Write-Problem; Garantie: kein verlorenes Event.
Entscheidung	Events atomar mit DB-Änderung in `outbox`-Tabelle; Relay-Prozess publiziert asynchron (Technologie: ADR-009).
Alternativen	(1) Dual Write – abgelehnt: kein Two-Phase-Commit. (2) Event-Sourcing – zu komplex.
Konsequenzen	(+) Keine verlorenen Events. (+) Atomare Konsistenz. (-) Idempotente Handler erforderlich.

9.5 ADR-005: Angular SPA als Frontend-Technologie

Feld	Inhalt
Status	Akzeptiert – 2026-02-05
Qualitätsziele	Usability (primär), Maintainability
Entscheidungstreiber	Team-Kenntnisse in Vue.js und Angular vorhanden; SPA ermöglicht reaktive UX ohne Server-Round-Trip; TypeScript erhöht Wartbarkeit.
Entscheidung	Angular, TypeScript.
Alternativen	(1) Next.js – zurückgestellt (v2.0). (2) Vue.js – als gleichwertige Alternative evaluiert; Angular gewählt aufgrund stärkerer Typisierung und integriertem Framework-Ökosystem.
Konsequenzen	(+) Bekanntes Ökosystem. (+) Starke Typisierung via TypeScript. (-) Bundle-Grösse (Code Splitting erforderlich). (-) SEO-Aufwand (kein SSR in v1.0).

9.6 ADR-006: PostgreSQL als primäre Datenbank

Feld	Inhalt
Status	Akzeptiert – 2026-02-08
Qualitätsziele	Functional Correctness (primär), Reliability-Availability
Entscheidungstreiber	ACID-Garantien für Transaktionen; JSONB-Support[13] für flexible Payloads; Flyway-Kompatibilität; breite Java-Ökosystem-Unterstützung.
Entscheidung	PostgreSQL 15 (Managed) für alle Services.
Alternativen	(1) MySQL – abgelehnt (JSONB-Support, Lizenz). (2) MongoDB – abgelehnt (ACID-Transaktionen).
Konsequenzen	(+) Starke Konsistenz. (+) JSONB-Flexibilität. (-) Skalierbarkeit nur vertikal (Read-Replicas helfen).

9.7 ADR-007: Kein Service Mesh in v1.0

Dieses ADR begründet den bewussten Verzicht auf ein Service Mesh (z. B. Linkerd oder Istio) in der ersten Produktionsversion. Es korrespondiert direkt mit der technischen Schuld TS1 und wird in v2.0 durch die Einführung von Linkerd abgelöst.

[13] JSONB (JSON Binary): speichert JSON-Daten binär dekomprimiert – im Gegensatz zu JSON indizierbar (GIN-Index) und deutlich schneller abfragbar.

Feld	Inhalt
Status	Akzeptiert – 2026-02-10
Qualitätsziele	Maintainability (primär); Security (nachrangig)
Entscheidungstreiber	Geringes Team-Know-how mit Service Mesh; erhöhter Betriebsaufwand für Sidecar-Infrastruktur übersteigt den Nutzen in v1.0; Budget- und Zeitrahmen des MVP.
Kontext	Ein Service Mesh ergänzt Kubernetes um mTLS[14] zwischen Services, L7-Observability und automatisches Retry. Ohne Service Mesh kommunizieren Services intern über unverschlüsseltes HTTP innerhalb des privaten Kubernetes-Netzwerks.
Entscheidung	Kein Service Mesh in v1.0. Netzwerk-Sicherheit wird durch Kubernetes Network Policies und TLS am API Gateway kompensiert. Tilgung: Linkerd in v2.0 (TS1).
Alternativen	(1) Linkerd – zurückgestellt auf v2.0: geringerer Overhead als Istio, bevorzugte Option. (2) Istio – abgelehnt: zu hohe operative Komplexität für Team-Grösse.
Konsequenzen	(+) Reduzierte Betriebskomplexität in v1.0. (+) Schnellerer MVP-Launch. (-) Kein mTLS intern (Sicherheitsrisiko TS1). (-) Eingeschränkte L7-Observability.

9.8 ADR-008: Single-Tenant-Architektur in v1.0

Dieses ADR begründet den bewussten Verzicht auf Multi-Tenancy in der ersten Version. Es korrespondiert mit der technischen Schuld TS4 und wird in v2.0 durch ein Schema-per-Tenant-Modell abgelöst (vgl. Kap. 14.3).

[14] *mTLS (Mutual TLS):* Gegenseitige TLS-Authentifizierung, bei der sich beide Seiten (Client *und* Server) mit einem Zertifikat ausweisen. Im Gegensatz zu einfachem TLS, das nur den Server authentifiziert, stellt mTLS sicher, dass auch der aufrufende Service vertrauenswürdig ist – ein wichtiges Konzept zur Absicherung der Service-zu-Service-Kommunikation in Microservice-Architekturen.

Feld	Inhalt
Status	Akzeptiert – 2026-02-12
Qualitätsziele	Maintainability (primär, kurzfristig); Functional Correctness
Entscheidungstreiber	Zeitrahmen MVP (6 Monate); Multi-Tenancy erfordert durchgängige `tenant_id`-Spalten, separates IdP-Realm und Tenant-Routing – zu aufwändig für v1.0. Erste Zielgruppe: ein einzelner Kursanbieter (SkillBridge AG).
Kontext	Multi-Tenancy bezeichnet die Fähigkeit einer SaaS-Plattform, mehrere Mandanten (Kursanbieter) auf derselben Infrastruktur mit strikter Datentrennung zu betreiben. In v1.0 wird EduPlanner für einen einzigen Kursanbieter betrieben.
Entscheidung	Single-Tenant-Betrieb in v1.0. Alle Services teilen eine gemeinsame Deployment-Instanz ohne mandantenbezogene Datentrennung.
Alternativen	(1) Schema-per-Tenant von Anfang an – abgelehnt: zu komplex für MVP-Zeitrahmen. (2) Database-per-Tenant – abgelehnt: zu hohe Infrastrukturkosten in v1.0.
Konsequenzen	(+) Deutlich vereinfachte Datenbankschicht in v1.0. (+) Schnellerer MVP-Launch. (-) Technische Schulden für v2.0 (TS4). (-) Migration auf Multi-Tenancy erfordert Schema-Änderungen an allen Tabellen (R11).

9.9 ADR-009: Outbox Relay – Technologiewahl

Dieses ADR spezifiziert die konkrete Implementierung des Relay-Prozesses, der die `outbox`-Tabelle aus ADR-004 ausliest und Events an RabbitMQ publiziert.

Feld	Inhalt
Status	Akzeptiert – 2026-02-15
Qualitätsziele	Functional Correctness (primär), Reliability-Availability
Entscheidungstreiber	Einfache, wartbare Lösung ohne zusätzliche externe Abhängigkeiten; muss *at-least-once*-Delivery garantieren; muss idempotent konsumierbar sein.
Kontext	Der Relay-Prozess ist ein Background-Thread innerhalb jedes Services, der periodisch die `outbox`-Tabelle nach `PENDING`-Einträgen abfragt, diese an RabbitMQ sendet und den Status auf `SENT` setzt. Bei Fehlern nach 5 Versuchen landet das Event in der Dead-Letter-Queue (DLQ).
Entscheidung	Polling-basierter Relay-Thread (Spring `@Scheduled`, Intervall 500 ms) innerhalb jedes Services. Kein separater Prozess, keine externe Abhängigkeit.
Alternativen	(1) Debezium[15] (CDC) – zurückgestellt auf v2.0: leistungsfähiger, aber höhere operative Komplexität. (2) Separater Relay-Service – abgelehnt: unnötige Infrastruktur für aktuelles Volumen.
Konsequenzen	(+) Keine zusätzliche Infrastruktur. (+) Einfach zu debuggen. (-) Polling-Latenz ($\leq$ 500 ms). (-) Erhöhte DB-Last bei hohem Event-Volumen.

9.10 ADR-010: Cloud-Provider – Microsoft Azure

Dieses ADR dokumentiert die Wahl von Microsoft Azure als Cloud-Provider für alle Produktions- und Staging-Umgebungen von EduPlanner (vgl. Kap. 7.1.1).

[15] *Debezium*: Open-Source-Plattform für Change Data Capture (CDC), die Datenbankänderungen aus dem Transaction Log ausliest und als Events publiziert. Leistungsfähiger als Polling, aber deutlich komplexer im Betrieb.

Feld	Inhalt
Status	Akzeptiert – 2026-02-20
Qualitätsziele	Reliability-Availability (primär), Security, Maintainability
Entscheidungstreiber	DSGVO/DSG-Compliance mit Datenresidenz Schweiz; Managed Services für PostgreSQL und Kubernetes; bestehendes Team-Know-how mit Azure; Budget $\leq$ CHF 800/Monat.
Kontext	EduPlanner muss personenbezogene Daten ausschliesslich in der EU bzw. der Schweiz speichern (Kap. 2.3). Nicht alle Cloud-Provider bieten Schweizer Regionen mit vollständigem Managed-Service-Angebot an.
Entscheidung	Microsoft Azure mit primärer Region *Switzerland North* (Zürich) und Failover-Region *Switzerland West* (Genf). Eingesetzte Managed Services: Azure Kubernetes Service (AKS 1.28+), Azure Database for PostgreSQL 15 Flexible Server, Azure Key Vault, Azure Monitor.
Alternativen	(1) AWS – abgelehnt: Schweizer Region (*eu-central-2*) verfügbar, aber geringeres Team-Know-how; höhere Migrationskosten. (2) Google Cloud – abgelehnt: kein Schweizer Rechenzentrum zum Entscheidungszeitpunkt. (3) On-Premise – ausgeschlossen durch Randbedingung (Kap. 2.1).
Konsequenzen	(+) Datenresidenz Schweiz erfüllt DSGVO/DSG. (+) Managed PostgreSQL: automatisches Failover, PITR, integriertes Monitoring. (+) Team-Know-how reduziert Einarbeitungsaufwand. (-) Gewisser Vendor Lock-in (mitigiert durch cloud-agnostische Tools: K8s, PostgreSQL, RabbitMQ – vgl. R7).

9.11 ADR-011: Contract-First mit OpenAPI 3.1 und Swagger UI

Dieses ADR begründet den Contract-First-Ansatz mit OpenAPI 3.1.0 als verbindliche Entwicklungsstrategie für alle REST-Schnittstellen von EduPlanner sowie die Bereitstellung der interaktiven Swagger UI pro Service. Die konsolidierte Spezifikation liegt in Anhang D.

Feld	Inhalt
Status	Akzeptiert – 2026-02-22
Qualitätsziele	Maintainability (primär), Functional Correctness, Usability
Entscheidungstreiber	API-First als verbindliche Randbedingung (Kap. 2.1); vollständige OpenAPI-Dokumentation als Go-Live-Kriterium v1.0 (Kap. 1.1); Contract Testing (TS2) setzt maschinenlesbare Spec voraus; parallele Frontend-/Backend-Entwicklung ohne Warten auf Implementierung.
Kontext	Im *Code-First*-Ansatz (z. B. Springdoc-Annotationen) entsteht die Spec nachgelagert aus dem Code – sie beschreibt, was implementiert wurde, nicht was vereinbart wurde. Dies führt zu Drift zwischen Spec und Implementierung sowie zu blockierter Frontend-Entwicklung. Im *Contract-First*-Ansatz ist die YAML-Datei die einzige Quelle der Wahrheit: Sie entsteht zuerst, wird in Git versioniert, und Server-Stubs werden daraus generiert. Der Compiler erzwingt Konformität.
Entscheidung	**Contract-First** mit OpenAPI 3.1.0 (YAML). Die Datei `openapi/eduplanner-api.yaml` wird manuell gepflegt. Server-Stubs (Spring-Boot-Interfaces) werden via `openapi-generator-maven-plugin` generiert (`interfaceOnly=true`, `useSpringBoot3=true`). Das Dev-Team implementiert ausschliesslich die generierten Interfaces. Jeder Service stellt die interaktive Swagger UI unter `/swagger-ui/index.html` bereit (`springdoc-openapi-starter-webmvc-ui`); in Produktion nur aus dem internen Netz erreichbar. Die Spec wird in der CI-Pipeline mit `vacuum` (Linting) und `openapi-generator` (Generierbarkeit) validiert – ein ungültiger Vertrag blockiert den Build.
Alternativen	(1) *Code-First / Springdoc:* abgelehnt – Spec entsteht nachgelagert, Frontend-Entwicklung blockiert, Drift möglich. (2) *GraphQL:* zurückgestellt auf v2.0 als optionale Schicht neben REST (Kap. 14.3). (3) *AsyncAPI:* evaluiert für Domain Events (RabbitMQ); zurückgestellt auf v1.1 – ergänzt OpenAPI, ersetzt es nicht. (4) *Keine formale Spec:* abgelehnt – Contract Testing (TS2) und externes Onboarding nicht möglich.
Konsequenzen	(+) Spec und Implementierung sind strukturell nie aus dem Sync (Compiler-Enforcement). (+) Frontend kann mit generierten TypeScript-Clients parallel entwickeln. (+) Swagger UI (`/swagger-ui/index.html`) bietet interaktives API-Testing ohne externes Tool. (+) Basis für Pact Contract Tests (TS2-Tilgung Sprint 8). (-) Initiale Spec-Erstellung erfordert Disziplin; kein "quick and dirty" Endpoint möglich. (-) OpenAPI beschreibt nur synchrone REST – für asynchrone Domain Events ist AsyncAPI ergänzend nötig (v1.1).

10 Qualitätsanforderungen

Dieses Kapitel beschreibt die nichtfunktionalen Anforderungen an EduPlanner anhand von ISO/IEC 25010 Qualitätskategorien. Die Anforderungen werden zunächst als hierarchischer Qualitätsbaum strukturiert und anschließend in messbaren Szenarien konkretisiert, die als Grundlage für Architekturentscheidungen und Akzeptanztests dienen.

10.1 Qualitätsbaum

Der Qualitätsbaum (arc42 v9: Pflicht) gibt eine hierarchische Übersicht aller relevanten Qualitätseigenschaften von EduPlanner, gegliedert nach ISO/IEC 25010 Hauptkategorien. Er dient als Navigation in die konkreten Szenarien in Kap. 10.2 und ermöglicht die schnelle Priorisierung im ATAM-light-Prozess.

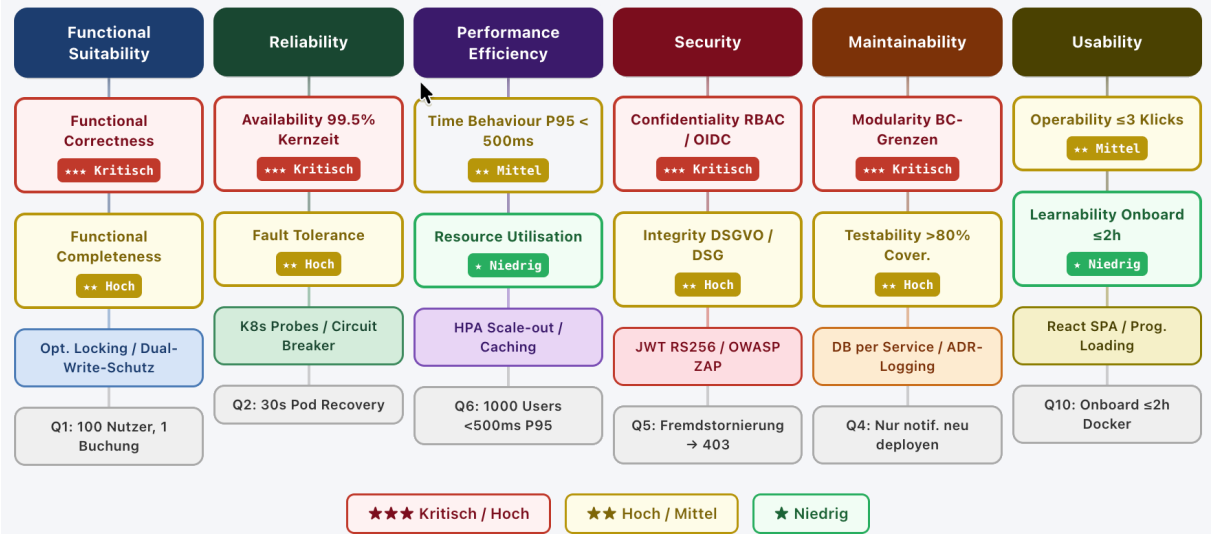


Abbildung 16: *Qualitätsbaum: EduPlanner Qualitätsanforderungen*

ATAM-light – Wesentliche Trade-offs in EduPlanner:[16]

Microservices (ADR-001): ↑ Maintainability, ↑ Scalability, ↓ Betriebskomplexität
Jeder Bounded Context wird als eigenständiger Service deployt. *Gewinn:* Ein Team kann den `registration`-Service ändern und deployen, ohne dass andere Services neu gestartet werden müssen – das erhöht die Wartbarkeit erheblich. Ausserdem kann der `registration`-Service zur Einschreibungsfrist auf 10 Pods hochskaliert werden, während `catalog` bei 1 Pod bleibt. *Preis:* Das System besteht nun aus fünf laufenden Prozessen, fünf Datenbanken, einem Message Broker und einem API Gateway – all das muss überwacht, gepatcht und betrieben werden. Ein Entwickler, der lokal arbeitet, startet nicht mehr eine Applikation, sondern eine ganze Landschaft via Docker Compose. Dieser Overhead ist das bewusst eingegangene Risiko für die gewonnene Unabhängigkeit (vgl. Risiko R5).

Asynchrones Messaging (ADR-002): ↑ Availability, ↑ Maintainability, ↓ Eventual Consistency
Zustandsändernde Ereignisse zwischen Services (z. B. "Teilnehmer eingeschrieben") werden nicht per direktem REST-Aufruf übermittelt, sondern als Domain Event über RabbitMQ publiziert.

[16] *Architecture Tradeoff Analysis Method (ATAM):* Strukturierte Methode zur Architekturbewertung, die konkurrierende Qualitätsziele systematisch analysiert und Trade-offs explizit macht. „ATAM-light"bezeichnet hier eine vereinfachte, projektangepasste Variante ohne vollständigen ATAM-Workshop.

Gewinn: Fällt der `notification`-Service aus, scheitert deshalb *nicht* die Einschreibung – das Event wartet in der Queue, bis der Service wieder läuft. Die Services sind entkoppelt und können unabhängig voneinander geändert werden. *Preis:* Das System ist nicht sofort konsistent. Nach dem Archivieren eines Kurses sind die abhängigen Durchführungen und Einschreibungen erst nach ca. 2–5 Sekunden ebenfalls auf "storniert" gesetzt. Ein Admin, der unmittelbar nach dem Archivieren nachschaut, könnte kurzzeitig noch offene Einschreibungen sehen. Dieses Verhalten (Eventual Consistency) muss im Frontend und in der Dokumentation explizit kommuniziert werden.

Single-Tenant v1.0 (ADR-008): ↑ Maintainability kurzfristig, ↓ Technische Schulden v2.0
In v1.0 wird EduPlanner für *einen einzigen* Kursanbieter betrieben. Jede Datenbanktabelle enthält keine `tenant_id`-Spalte; das Routing auf verschiedene Mandanten entfällt. *Gewinn:* Die Datenbankschicht ist erheblich einfacher – keine mandantenbezogenen Abfragen, kein Tenant-Routing, kein separates IdP-Realm pro Kursanbieter. Das beschleunigt den MVP deutlich. *Preis:* Sobald in v2.0 mehrere Kursanbieter auf derselben Infrastruktur betrieben werden sollen (echtes SaaS), muss *jede* Tabelle um eine `tenant_id`-Spalte ergänzt werden. Das ist eine aufwändige, riskante Datenmigration an allen Services gleichzeitig (Risiko R11, Tilgung TS4).

Kein Service Mesh in v1.0 (ADR-007): ↑ Betriebseinfachheit, ↓ Security (kein mTLS intern)
Ein Service Mesh (z. B. Linkerd) würde automatisch verschlüsselte Verbindungen zwischen allen Services herstellen (mTLS: gegenseitige TLS-Authentifizierung) sowie L7-Observability und automatische Retries liefern. *Gewinn des Verzichts:* Das Team muss sich in v1.0 nicht mit Sidecar-Proxies, Zertifikatsrotation und Mesh-Konfiguration befassen. Der Betriebsaufwand bleibt im Rahmen des Budgets und der Teamgrösse. *Preis:* Die Service-zu-Service-Kommunikation innerhalb des Kubernetes-Clusters läuft über unverschlüsseltes HTTP. Ein Angreifer, der Zugang zum internen Netz erlangt (z. B. durch einen kompromittierten Pod), könnte den Datenverkehr mitlesen. Dieses Risiko wird durch Kubernetes Network Policies mitigiert, ist aber als technische Schuld TS1 explizit erfasst und wird in v2.0 durch Linkerd geschlossen.

10.2 Qualitätsszenarien

Die nachfolgende Tabelle konkretisiert die im Qualitätsbaum identifizierten Eigenschaften als messbare Szenarien nach dem arc42-Schema.

Methodischer Aufbau der Szenarien

Jedes Szenario folgt drei klar getrennten Schritten:

1. ***Stimulus:*** *Was passiert – die auslösende Situation oder der Auslöser (Nutzeraktion, Systemereignis, Fehler).*
2. ***Erwartete Reaktion:*** *Was das System daraufhin tun muss – das architektonisch verankerte Verhalten.*
3. ***Nachweis (Spalte "Messbar"):*** *Wie die Erfüllung objektiv überprüft wird – Testwerkzeug, Logdatei, Audit-Report o. ä. Erst durch den Nachweis wird ein Szenario im Architekturreview belastbar.*

Die Szenarien bilden die Akzeptanzkriterien für Architekturreviews und sind die Basis für Lasttests (Kap. 8.5) und den Security-Audit (Go-Live-Kriterium v1.0, Kap. 1.1).

#	Attribut	Stimulus	Erwartete Reaktion	Messbar
Q1	Functional Correctness	100 Nutzer buchen letzten Platz gleichzeitig	Genau 1 Einschreibung; 99 Wartelisteneinträge; keine Überbuchung	k6[17] Lasttest mit Assertions (ADR-004)
Q2	Reliability – Avail.	Pod-Neustart von `registration`	System reagiert in 30 s; 2. Instanz übernimmt	K8s-Probe-Logs (ADR-001)
Q3	Reliability – Avail.	`notification` fällt aus	Einschreibungen möglich; E-Mails nach Recovery ($\leq$ 10 min)	Chaos Engineering (ADR-002)
Q4	Maintainability	Neues E-Mail-Template hinzufügen	Nur `notification`-Deploy; kein Re-Deploy anderer Services	Deploy-Log (ADR-001)
Q5	Security	Fremdstornierung durch Teilnehmer	403 Forbidden; Audit-Log; keine Datenänderung	Penetrationstest (Kap. 8.1)
Q6	Performance Eff.	1 000 concurrent Nutzer, Kurskatalog	P95 $<$ 500 ms; Error-Rate $<$ 0,1 %	k6 Lasttest
Q7	Security	Abgelaufenes JWT	401 in $<$ 100 ms; kein Datenzugriff	Automat. Test (Kap. 8.1.1)
Q8	Privacy / Security	Nutzer beantragt Löschung	PII[18] in 30 Tagen gelöscht/anonymisiert	Manueller Akzeptanztest (Kap. 8.6)
Q9	Functional Corr.	Stornierung + gleichzeitiges Nachrücken	Genau 1 Nachrücker; keine Doppelbuchung	Concurrency-Test (Kap. 6.6)
Q10	Maintainability	Neuer Entwickler ongeboardet	Docker Compose lauffähig in $\leq$ 2 h	Onboarding-Checkliste
Q11	Performance Eff.	10 000 Einschreibungen/Stunde[19]	P95 $<$ 1 s; HPA skaliert auf max. 10 Pods	k6 in Staging (Kap. 4.6)
Q12	Compliance	DSGVO-Audit	Controls dokumentiert; Löschkonzept nachgewiesen	Audit-Report (Kap. 8.6)
Q13	Maintainability	6. Service "payment" hinzufügen	Kein Refactoring bestehender Services; $\leq$ 2 Sprints	Architecture Review (ADR-001)
Q14	Maintainability	`registration` isoliert testen	Unit Tests ohne Netzwerk/DB; Integration Tests $<$ 5 min	CI-Laufzeit (Kap. 8.5)

[17] *k6* ist ein Open-Source-Lasttest-Tool (Grafana Labs), mit dem HTTP-Szenarien in JavaScript definiert und automatisiert gegen APIs ausgeführt werden. „Assertions"prüfen dabei automatisch die Korrektheit der Antworten.

[18] *Personally Identifiable Information (PII):* Alle Daten, die eine natürliche Person direkt oder indirekt identifizierbar machen, z. B. Name, E-Mail-Adresse oder IP-Adresse (vgl. Art. 4 Nr. 1 DSGVO).

[19] Dieser Wert modelliert ein Spitzenlastszenario, wie es z. B. zum Semesterstart auftreten kann, wenn viele Studierende gleichzeitig Kurse buchen. Er entspricht ca. 2,8 Einschreibungen pro Sekunde und ist damit als realistisches Worst-Case-Szenario für EduPlanner zu verstehen.

#	Attribut	Stimulus	Erwartete Reaktion	Messbar
Q15	Maintainability	Neuer externer API-Konsument onboarded	Swagger UI unter `/swagger-ui/index.html` erreichbar; Server-Stub aus Spec generierbar; Client-Stub in $\leq$ 30 min	Contract-First CI-Validierung; `openapi-generator` (ADR-011, Anhang D)

11 Risiken und technische Schulden

Dieses Kapitel dokumentiert die identifizierten Architektur- und Projektrisiken sowie die bewusst eingegangenen technischen Schulden von EduPlanner. Ziel ist es, Risiken frühzeitig sichtbar zu machen, ihre Eintrittswahrscheinlichkeit und ihren potenziellen Schaden zu bewerten und konkrete Gegenmassnahmen festzuhalten. Technische Schulden werden priorisiert und mit einem Tilgungsplan versehen, um einen kontrollierten Abbau im Projektverlauf sicherzustellen.

11.1 Identifizierte Risiken

Die nachfolgende Tabelle erfasst alle bekannten Risiken und bewertet sie anhand zweier Dimensionen: Wahrscheinlichkeit (W) und Impact (I), jeweils auf einer vierstufigen Skala (Niedrig / Mittel / Hoch / Kritisch). Zu jedem Risiko ist eine konkrete Gegenmassnahme definiert. Die Risikobewertungsmatrix in Abb. 17 visualisiert die Verteilung aller Risiken und macht kritische Cluster auf einen Blick erkennbar.

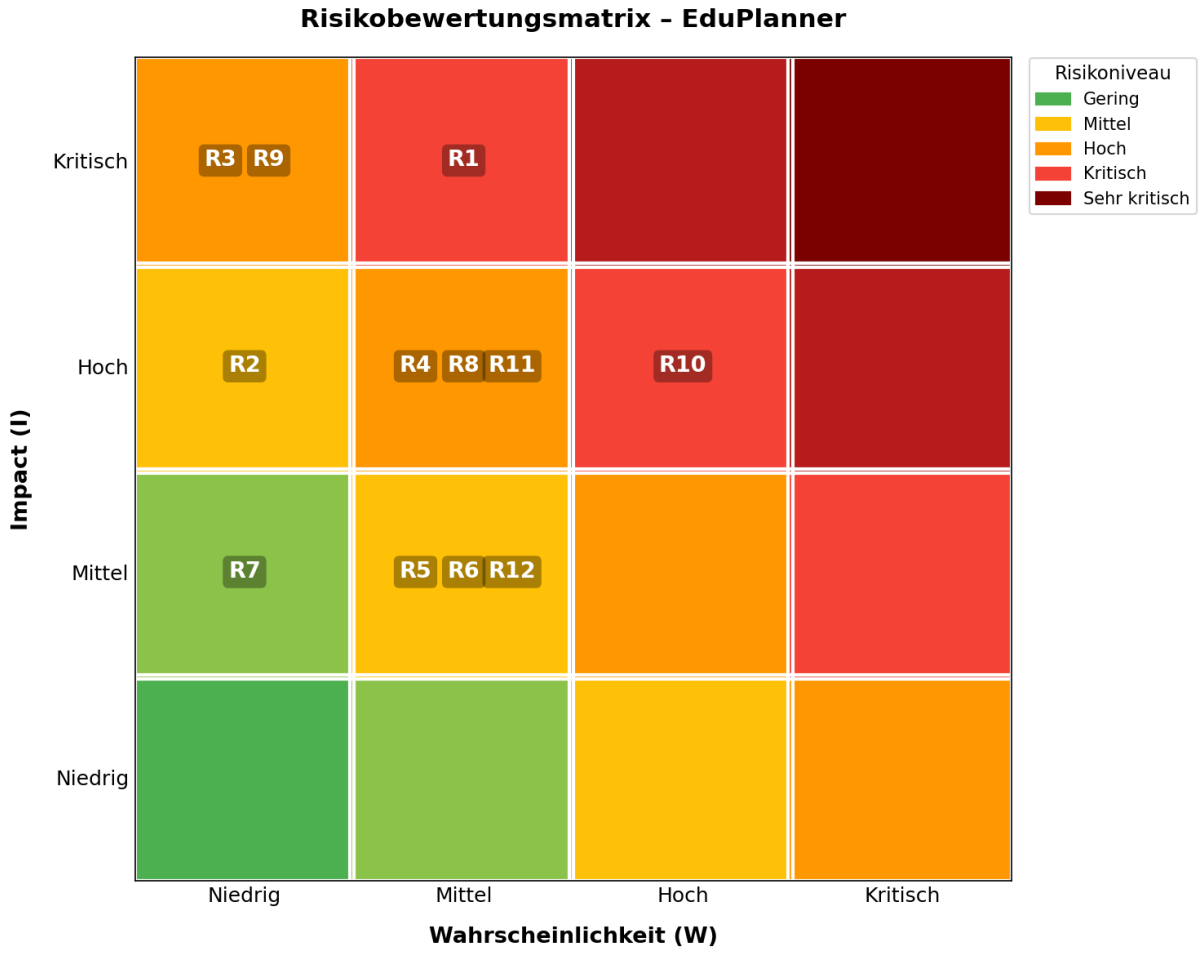


Abbildung 17: *Risikobewertungsmatrix: Wahrscheinlichkeit vs. Impact (R1–R12)*

Bewertungsmatrix: W = Wahrscheinlichkeit, I = Impact (jeweils: Niedrig / Mittel / Hoch / Kritisch)

#	Risiko	W	I	Massnahme
R1	Race Condition bei Kapazitätsprüfung	Mittel	Kritisch	Optimistic Locking + DB-Unique-Constraint; Lasttests vor Go-Live. Validiert durch Q1.
R2	RabbitMQ als Single Point of Failure	Niedrig	Hoch	3-Node Quorum-Cluster; Monitoring; auto. Failover getestet.
R3	Ausfall des Identity Providers	Niedrig	Kritisch	SLA $\geq$ 99,9 %; Read-Only-Fallback; JWT-Cache (24 h).
R4	PII gelangt in Logs	Mittel	Hoch	Log-Scrubbing; PII-Scanner im CI (detect-secrets).
R5	Microservice-Overhead überfordert Team	Mittel	Mittel	Modularer Monolith als Fallback definiert; Entscheidung nach MVP.
R6	Event-Schema-Änderungen brechen Consumer	Mittel	Mittel	Schema Registry evaluieren; nur additive Änderungen.
R7	Vendor Lock-in	Niedrig	Mittel	Cloud-agnostische Tools (K8s, PostgreSQL, RabbitMQ).
R8	Key-Person-Abhängigkeit	Mittel	Hoch	Pair-Programming; ADRs als Gedächtnis; Architektur-Workshops.
R9	DB-Korruption / fehlgeschlagene Migration	Niedrig	Kritisch	Wöchentliche Restore-Tests; Flyway rückwärtskompatibel.
R10	Team-Burnout *[PM-Risiko]*	Hoch	Hoch	Hiring-Plan Q3; On-Call-Rotation max. 1 Wo/Monat. *Anm.: primär Projektmanagement-Risiko, zur Diskussion als Lehrbeispiel.*
R11	Datenmigration bei Schema-Änderungen	Mittel	Hoch	Expand-Contract-Pattern; Zero-Downtime-Migrations via mehrphasiges Deployment; Staging-Test zwingend.
R12	Budgetüberschreitung bei Lastspitzen	Mittel	Mittel	Cloud-Kosten variabel bei ungeplantem Traffic. Gegenmassnahme: Reserved Instances für Basisload (CHF 80 Einsparung = 10 % Puffer); HPA-Limits verhindern unkontrolliertes Scale-out; monatliches Capacity Review.

11.2 Technische Schulden

Technische Schulden bezeichnen bewusste architektonische Vereinfachungen, die kurzfristig Entwicklungsgeschwindigkeit ermöglichen, langfristig aber Wartungsaufwand oder Sicherheitslücken erzeugen können. Die nachfolgende Tabelle listet alle bekannten Schulden von EduPlanner, bewertet ihre Priorität und definiert einen konkreten Tilgungsplan. Schulden hoher Priorität sind für die nächsten Sprints eingeplant; Schulden niedriger Priorität werden spätestens vor dem v2.0-Release adressiert.

#	Schuld	Prio	Erläuterung und Tilgungsplan
TS1	Kein Service Mesh	Mittel	mTLS intern fehlt. Tilgung: Linkerd in v2.0 (ADR-007).
TS2	Kein Contract Testing (Pact)	Hoch	API-Kompatibilität manuell koordiniert. Tilgung: Sprint 8.
TS3	E-Mail-Templates im Container	Niedrig	Änderungen erfordern Re-Deploy. Tilgung: Template-UI in v1.1.
TS4	Fehlende Multi-Tenancy	Mittel	Single-Tenant (ADR-008). Tilgung: Schema-per-Tenant in v2.0.
TS5	Kein API-Versionsmanagement > v1	Niedrig	Tilgung: API-Versionierungsstrategie vor erstem externem API-Zugang – geplant Sprint 12 (vor v1.0-Release).
TS6	Keine Accessibility-Tests	Niedrig	WCAG 2.1 AA nicht geprüft. Tilgung: axe-core in v1.1.
TS7	Rate-Limiting nicht nach Rolle	Niedrig	Alle Rollen gleiche Limits. Tilgung: Kong-Konfig. Sprint 10.

12 Glossar

Das Glossar definiert die Ubiquitous Language von EduPlanner.

12.1 Fachliche Begriffe

Begriff	Kontext	Definition
Course	[catalog]	Weiterbildungsangebot mit Titel, Beschreibung und Lernzielen. Beschreibt *was* gelehrt wird. Bsp.: "Python für Einsteiger".
CourseStatus	[catalog]	Zustand eines Kurses: DRAFT → PUBLISHED → ARCHIVED. Nur diese Übergänge erlaubt (State Machine).
TrainerProfile	[catalog]	Stammdaten eines Trainers inkl. Fachgebiete.
CourseExecution	[scheduling]	Konkrete, zeitlich geplante Instanz eines Kurses. Bsp.: "Python Apr 2025, Bern, max. 12 TN".
ExecutionStatus	[scheduling]	PLANNED → PUBLISHED → CANCELLED. Nur PUBLISHED offen für Einschreibungen.
Capacity	[scheduling, reg.]	Maximale Teilnehmeranzahl. Kapazität = 0 → Warteliste.
Registration	[registration]	Verbindliche Anmeldung eines Teilnehmers zu einer Execution. Bsp.: Max Muster meldet sich an → `status=REGISTERED`.
WaitlistEntry	[registration]	Wartelisteneintrag mit Position (1 = erster) und Zeitstempel.
Participant	[identity]	Natürliche Person, die sich einschreiben kann. Systemrolle: PARTICIPANT.
Trainer	[identity]	Kursleiter. Systemrolle: TRAINER.
Staff	[identity]	Mitarbeitende des Kursanbieters. Systemrolle: STAFF.
Admin	[identity]	Nutzer mit erweiterten Systemrechten. Systemrolle: ADMIN.
Domain Event	–	Unveränderliche Zustandsänderung als Fakt (Vergangenheitsform). Bsp.: `ParticipantRegistered` (nicht `RegisterParticipant`).
Bounded Context	–	Klar abgegrenzter Bereich mit eigener Fachsprache. Bsp.: "Trainer" in [catalog] hat andere Bedeutung als in [scheduling].
Aggregate	–	Gruppe von Domain-Objekten als konsistente Einheit. Aggregate Root ist einziger Einstiegspunkt. Bsp.: Registration-Aggregate verhindert Überbuchung.
Ubiquitous Language	–	Gemeinsame fachliche Sprache innerhalb eines BC. Code-Bezeichner verwenden exakt dieselben Begriffe wie die Fachsprache.

12.2 Domain Events – Vollständige Übersicht

Domain Event	Kontext	Auslöser
CourseCreated	catalog	Staff legt Kurs an
CourseTitleUpdated	catalog	Staff aktualisiert Kurstitel
TrainerAssignedToCourse	catalog	Staff ordnet Trainer zu
CoursePublished	catalog	Admin veröffentlicht
CourseArchived	catalog	Admin archiviert
CourseExecutionScheduled	scheduling	Staff plant Durchführung
TrainerAssignedToExecution	scheduling	Staff weist Trainer zu
CourseExecutionPublished	scheduling	Admin gibt frei
CourseExecutionCancelled	scheduling	Admin / System storniert
ParticipantRegistered	registration	Teilnehmer schreibt sich ein
ParticipantAddedToWaitlist	registration	Kurs voll
RegistrationCancelled	registration	Teilnehmer / Staff storniert
WaitlistParticipantPromoted	registration	Nachrücker von Warteliste
EmailNotificationSent	notification	System hat E-Mail versendet
UserCreated	identity	Admin / IdP legt Benutzer an
UserRoleAssigned	identity	Admin weist Rolle zu

13 Betrieb und Deployment – Runbook

Erweiterung des arc42-Templates

Dieses Kapitel ist keine *Bestandteil des offiziellen arc42-Templates v9, das mit Kapitel 12 (Glossar) endet. Es ergänzt die Architekturdokumentation um operative Aspekte, die für den produktiven Betrieb von EduPlanner notwendig sind. In einem realen Projekt würde dieses Runbook typischerweise als eigenständiges Dokument (z. B. im Team-Wiki) gepflegt und laufend aktualisiert.*

Dieses Kapitel dient als operatives Nachschlagewerk für den laufenden Betrieb von EduPlanner. Es beschreibt verbindliche Checklisten vor jedem Deployment, definiert Reaktionszeiten und Eskalationsstufen für Incidents sowie die regelmässig anfallenden Wartungsarbeiten. Das Runbook richtet sich an das Entwicklungs- und Betriebsteam und ist bei jeder Änderung am Deploymentprozess aktuell zu halten.

13.1 Pre-Deployment Checklist

Vor jedem produktiven Deployment ist die nachfolgende Checkliste vollständig abzuarbeiten. Kein Deployment darf ohne grüne CI-Pipeline und dokumentierten Rollback-Plan erfolgen. Bei Breaking Changes gegenüber bestehenden API-Konsumenten ist zusätzlich die frühzeitige Stakeholder-Kommunikation obligatorisch.

#	Prüfpunkt
1	Code Review abgeschlossen (min. 1 Approval)
2	Alle Unit- und Integrationstests grün (CI Pipeline)
3	Security Scan erfolgreich (Trivy: keine Critical/High)
4	Schema-Migration in Staging getestet (Flyway dryrun)
5	OpenAPI-Spezifikation aktualisiert (falls API-Änderung)
6	Pact Contract Tests grün (ab Sprint 8)
7	Docker Image mit Immutable Tag (Git SHA)
8	Release Notes erstellt
9	Rollback-Plan dokumentiert
10	Stakeholder informiert (bei Breaking Changes)

13.2 Incident Response

Tritt ein Störfall auf, wird dieser anhand der nachfolgenden Severity-Stufen klassifiziert. Die Einstufung bestimmt die maximale Reaktionszeit sowie den Eskalationspfad. SEV-1- und SEV-2-Incidents sind umgehend im internen Kommunikationskanal zu melden; für SEV-1 gilt zusätzlich die sofortige Benachrichtigung der Projektleitung. Nach jedem SEV-1- oder SEV-2-Incident ist ein Post-Mortem-Bericht innerhalb von 48 Stunden zu erstellen.

Severity	Beschreibung	Reaktionszei	Beispiele
SEV-1	System nicht verfügbar	15 min	Komplettausfall; Datenverlust; Sicherheitsvorfall
SEV-2	Kernfunktion eingeschränkt	1 h	Einschreibungen nicht möglich
SEV-3	Nicht-kritische Funktion	4 h	E-Mail-Versand verzögert
SEV-4	Kosmetisch / Minor	Nächster Sprint	UI-Darstellungsfehler

13.3 Routine-Wartungsarbeiten

Neben dem reaktiven Incident-Management sind folgende Wartungsarbeiten proaktiv und nach festgelegtem Rhythmus durchzuführen. Sie dienen der langfristigen Systemstabilität, der Datensicherheit und der Kostenkontrolle. Die Verantwortung für die termingerechte Durchführung liegt beim jeweils zuständigen Teammitglied; die Ergebnisse sind im internen Betriebslog zu dokumentieren.

Aufgabe	Frequenz	Beschreibung
DB-Backup-Validierung	Wöchentlich	Auto-Restore in Testing; Datenintegrität validieren
Dependency Updates	2-wöchentlich	Dependabot PRs; Security Patches priorisieren
Certificate Renewal	Monatlich prüfen	Let's Encrypt autorenew; 30 Tage vor Ablauf prüfen
Capacity Review	Monatlich	Cloud-Kosten; Reserved Instances evaluieren
Security Scan	Wöchentlich	Trivy, Snyk
Chaos Engineering	Wöchentlich	Pod-Kills, Network-Delays in Staging
DR-Systemtest	Halbjährlich	Vollständiger Failover-Test; Verantwortung: Patrick Hari

14 Roadmap und Evolution

Erweiterung des arc42-Templates

Auch dieses Kapitel ist kein *Bestandteil des offiziellen arc42-Templates v9. Zukunftspläne und technische Weiterentwicklung sind im arc42-Standard unter Kapitel 11 (Risiken und technische Schulden) angesiedelt. Eine dedizierte Roadmap geht darüber hinaus und ist hier als Lehrbeispiel für die Planung von Architekturevolution dokumentiert. In der Praxis lebt ein solches Dokument typischerweise als Product-Roadmap im Projektmanagement-Tool (z. B. Jira, Linear).*

Dieses Kapitel beschreibt die geplante Entwicklung von EduPlanner über die kommenden Versionen. Die Roadmap orientiert sich an den identifizierten technischen Schulden (Kap. 11), den Qualitätszielen (Kap. 10) sowie den geschäftlichen Anforderungen. Die Umsetzung beginnt im April 2026; alle Zeitangaben sind als Planungsziele zu verstehen und werden im Rahmen des regulären Sprint-Reviews angepasst.

14.1 Versionsübersicht

Die folgende Tabelle gibt einen kompakten Überblick über alle geplanten Versionen und ihre zentralen Meilensteine. Die Versionen bauen aufeinander auf: Das MVP legt die technische Basis, v1.0 konsolidiert und stabilisiert, v1.1 tilgt priorisierte technische Schulden, und v2.0 transformiert EduPlanner zu einer mandantenfähigen SaaS-Plattform.

Version	Zeitraum	Meilenstein
MVP (v0.1–v0.9)	Q2 2026	Kernfunktionen: Katalog, Einschreibung, E-Mail, RBAC
v1.0	Q3 2026	Vollständiger Release: API-Doku, Performance-Tests, Security-Audit
v1.1	Q4 2026	Polish: Contract Tests, Template-UI, Accessibility, Rate-Limiting
v2.0	Q1–Q2 2027	SaaS / Multi-Tenancy: Schema-per-Tenant, Service Mesh, Payment

14.2 v1.1 – Polish und Stabilität (Q4 2026)

Version 1.1 fokussiert sich auf die Tilgung der höchstpriorisierten technischen Schulden aus Kap. 11 sowie auf eine verbesserte Nutzererfahrung. Die in dieser Phase umgesetzten Massnahmen sind Voraussetzung für den stabilen Betrieb und bilden die Basis für die umfangreichen Erweiterungen in v2.0. Alle Punkte sind bereits in der Backlog-Priorisierung berücksichtigt.

#	Feature / Tilgung	Details
TS2	Contract Testing	Pact-Tests für alle Service-Schnittstellen
TS3	Template-Management-UI	E-Mail-Templates über Admin-UI konfigurierbar
TS6	Accessibility	axe-core in Cypress; WCAG 2.1 AA
TS7	Differenziertes Rate-Limiting	Unterschiedliche Limits pro Rolle
–	iCal-Export	Durchführungstermine als iCal-Feed

14.3 v2.0 – Multi-Tenancy und SaaS (Q1–Q2 2027)

Version 2.0 stellt den grössten Evolutionsschritt von EduPlanner dar: Die Plattform wird von einer Single-Tenant-Applikation zu einer vollwertigen SaaS-Lösung mit Mandantenfähigkeit weiterentwickelt. Diese Phase erfordert tiefgreifende Änderungen an Datenbankschicht, Authentifizierung und Infrastruktur. R11 (Datenmigration, Kap. 11.1) ist das grösste Risiko dieser Phase und wird durch das Expand-Contract-Pattern sowie zwingende Staging-Tests mitigiert.

Architekturhinweis v2.0

v2.0 erfordert signifikante Änderungen an der Datenbankschicht (ADR-008 Tilgung) und der Authentifizierung. R11 (Datenmigration) ist das grösste Risiko dieser Phase.

Feature	Beschreibung
Multi-Tenancy	Schema-per-Tenant; `tenant_id` in allen Tabellen; Tenant-Routing; separates IdP-Realm. (Tilgung TS4, ADR-008)
Service Mesh	Linkerd für mTLS, L7-Observability, auto. Retry. (Tilgung TS1, ADR-007)
Payment Gateway	Stripe-Integration; Rechnungsstellung; Gutschein-System.
Mobile App	React Native für iOS/Android; Push-Notifications.
Advanced Analytics	Data Warehouse; BI-Dashboard; ML-Kursempfehlungen.
LMS-Integration	API-Schnittstellen zu Moodle, Canvas.
GraphQL API	Optionale GraphQL-Schicht neben REST.

A Anhang A: Event Storming – Ergebnisse

Dieser Anhang dokumentiert die Ergebnisse des Event-Storming-Workshops, der zu Beginn der Architekturarbeit durchgeführt wurde. Event Storming[20] ist eine kollaborative Workshop-Technik aus dem Domain-Driven Design (DDD), bei der Fachexperten und Entwickler gemeinsam die relevanten Ereignisse (Events) eines Geschäftsbereichs auf farbcodierten Haftnotizen erarbeiten. Ziel ist es, ein gemeinsames Verständnis der Domäne zu schaffen, Bounded Contexts zu identifizieren und die Grundlage für eine ereignisorientierte Architektur zu legen – ohne technische Implementierungsdetails vorwegzunehmen.

A.1 Methodik

Das Event Storming wurde nach der Methode von Alberto Brandolini durchgeführt. Der Workshop gliedert sich in sechs aufeinander aufbauende Phasen, die vom bewusst unstrukturierten Sammeln von Ereignissen bis hin zur Identifikation fachlicher Grenzen (Bounded Contexts) führen.

1. **Chaotische Exploration** (∼30 min): Alle Teilnehmer schreiben Domain Events auf orangefarbene Haftnotizen.
2. **Timeline**: Events werden in zeitlicher Reihenfolge angeordnet.
3. **Hotspots**: Rote Karten für unklare Stellen.
4. **Kommandos**: Blaue Karten für Nutzeraktionen vor dem Event.
5. **Akteure und Systeme**: Gelbe Karten für Auslöser.
6. **Bounded Contexts**: Natürliche Gruppen werden eingerahmt.

A.2 Vollständige Domain-Event-Liste

Die nachfolgende Tabelle listet alle im Workshop erarbeiteten Domain Events von EduPlanner. Jedes Event ist einem auslösenden Akteur (Staff, Admin, Participant oder System) sowie einem Bounded Context zugeordnet. Die Bounded Contexts entsprechen direkt den Microservices der Zielarchitektur (vgl. Kap. 5).

[20] *Event Storming* wurde von Alberto Brandolini entwickelt und erstmals in seinem Buch *Introducing EventStorming* (Leanpub, 2021, https://leanpub.com/introducing_eventstorming) beschrieben. Weitere Informationen unter https://www.eventstorming.com.

Domain Event	Auslöser (Akteur)	Bounded Context
CourseCreated	Staff	catalog
CourseTitleUpdated	Staff	catalog
TrainerAssignedToCourse	Staff	catalog
CoursePublished	Admin	catalog
CourseArchived	Admin	catalog
CourseExecutionScheduled	Staff	scheduling
TrainerAssignedToExecution	Staff	scheduling
CourseExecutionPublished	Admin	scheduling
CourseExecutionCancelled	Admin / System	scheduling
ParticipantRegistered	Participant	registration
ParticipantAddedToWaitlist	System	registration
RegistrationCancelled	Participant / Staff	registration
WaitlistParticipantPromoted	System	registration
EmailNotificationSent	System	notification
UserCreated	Admin / IdP	identity
UserRoleAssigned	Admin	identity

B Anhang B: Diagramme in A4 Quer – Grossansicht

Hinweis zur Lesbarkeit

Die folgenden Diagramme sind identisch mit den im Fliesstext eingebetteten Abbildungen, werden hier jedoch auf A4-Querformat dargestellt, um die Detaillesbarkeit zu verbessern.

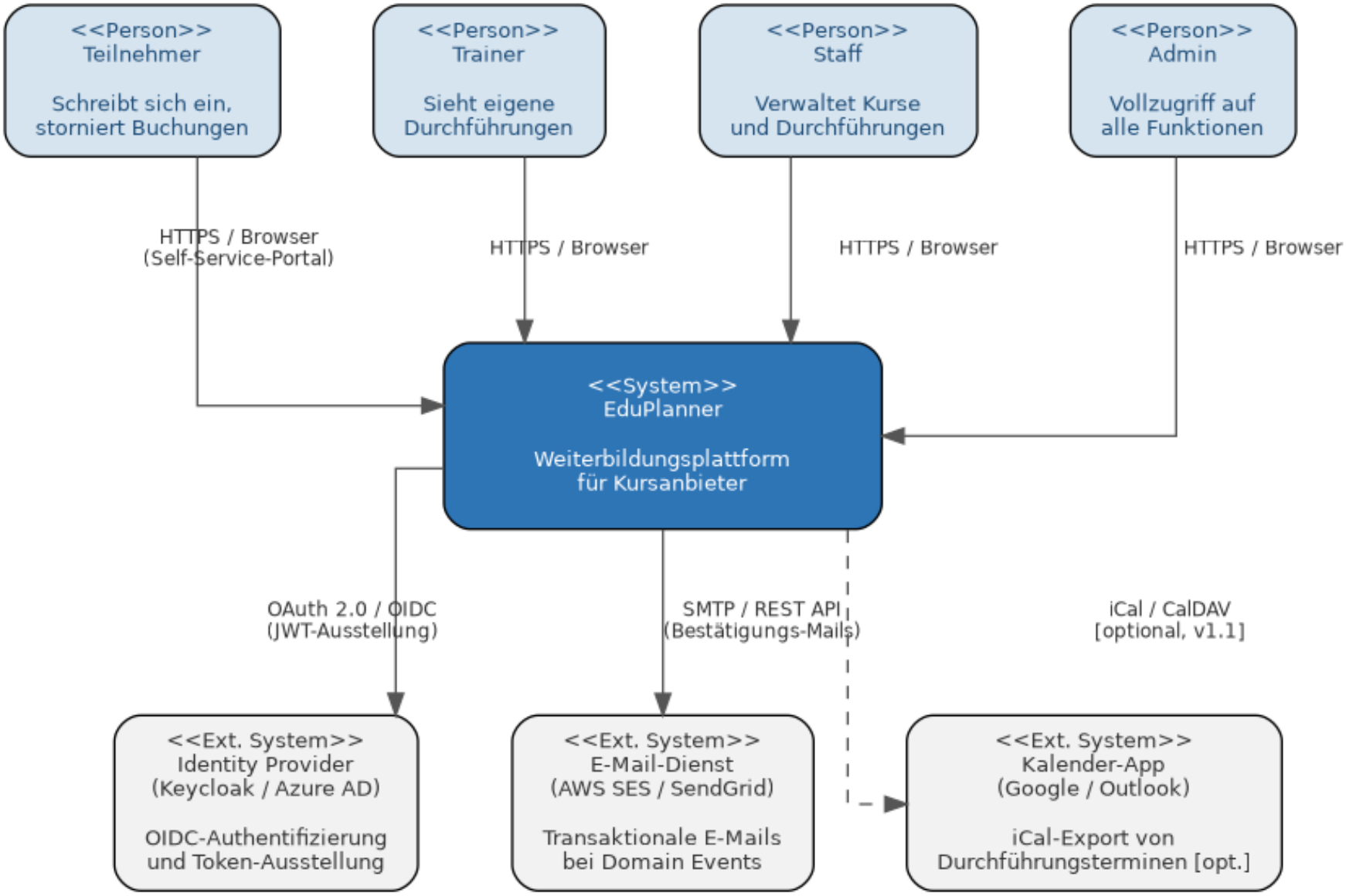


Abbildung 18: Grossansicht – *C4-Kontextdiagramm EduPlanner* (vgl. Kap. 3.1, Abb. 1)

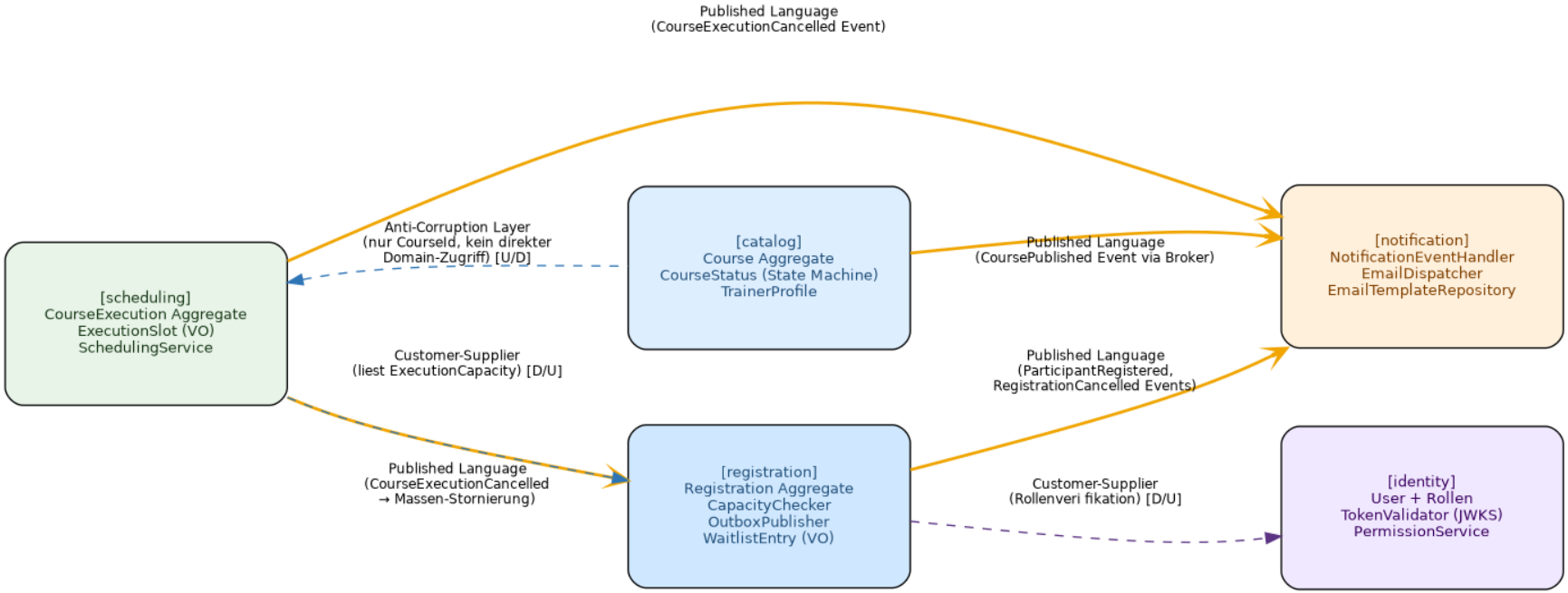


Abbildung 19: Grossansicht – *DDD Context Map: Bounded Contexts und Beziehungen* (vgl. Kap. 4.2.2, Abb. 2)

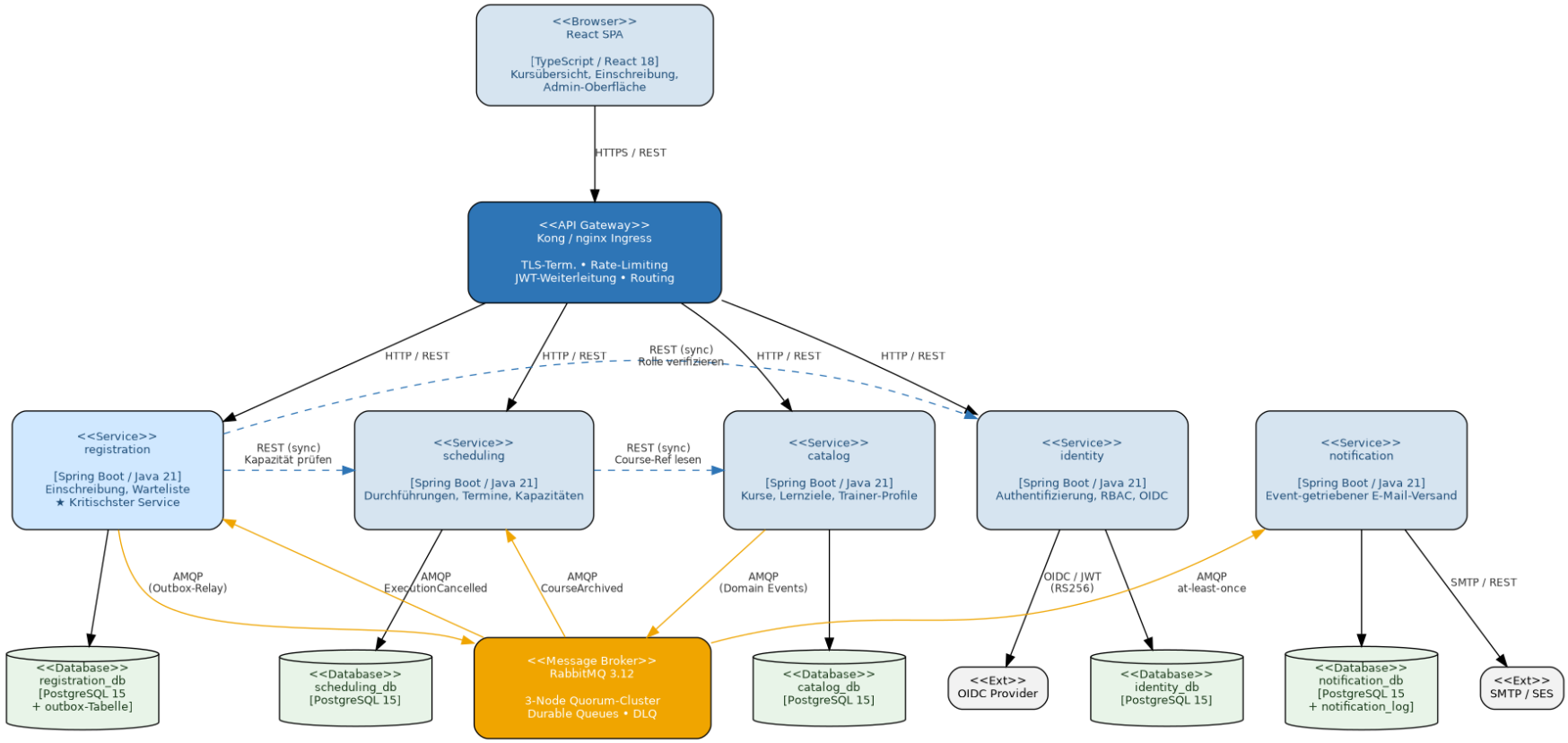


Abbildung 20: Grossansicht – *C4-Container-Diagramm: Gesamtsystem EduPlanner* (vgl. Kap. 5.1, Abb. 3)

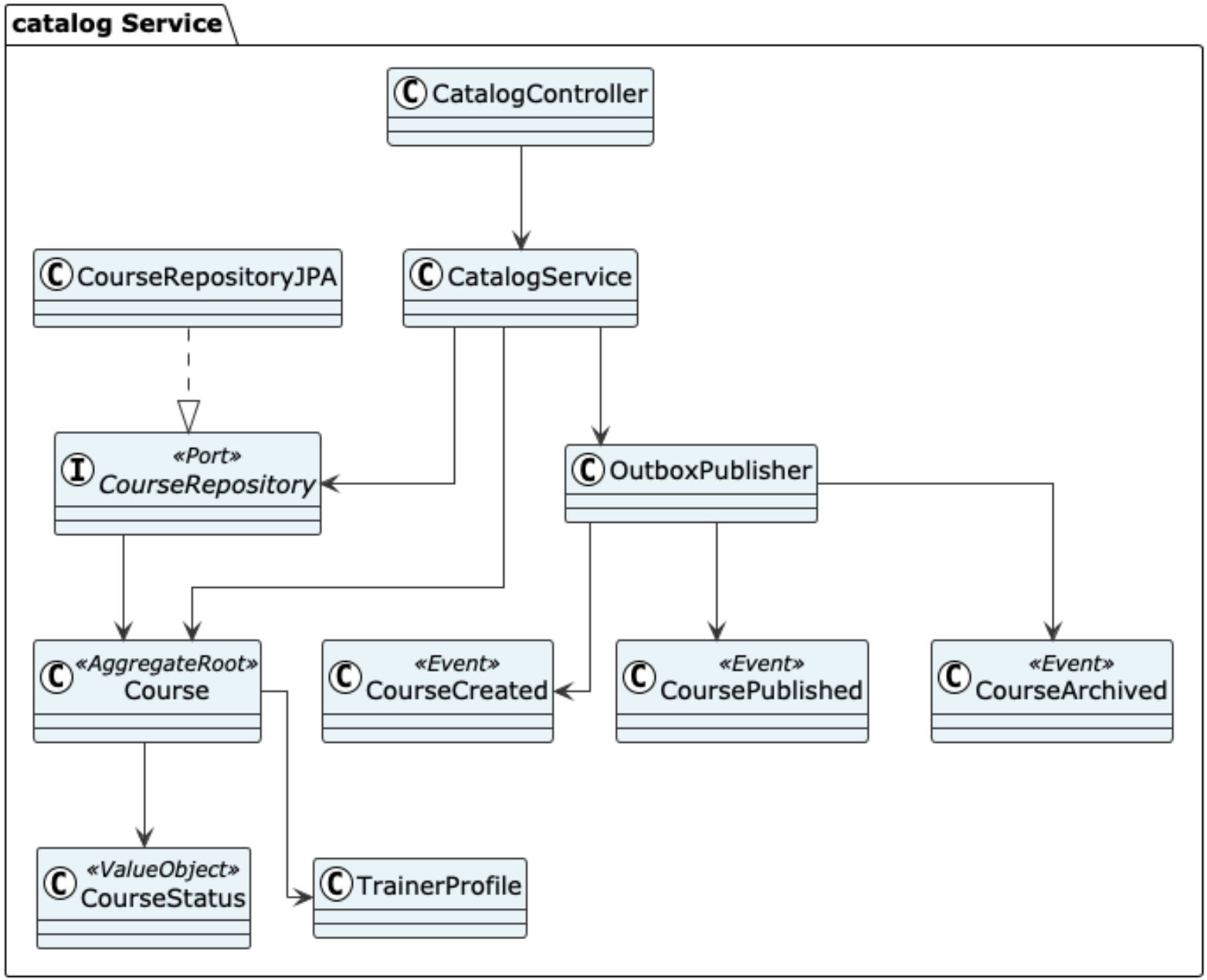
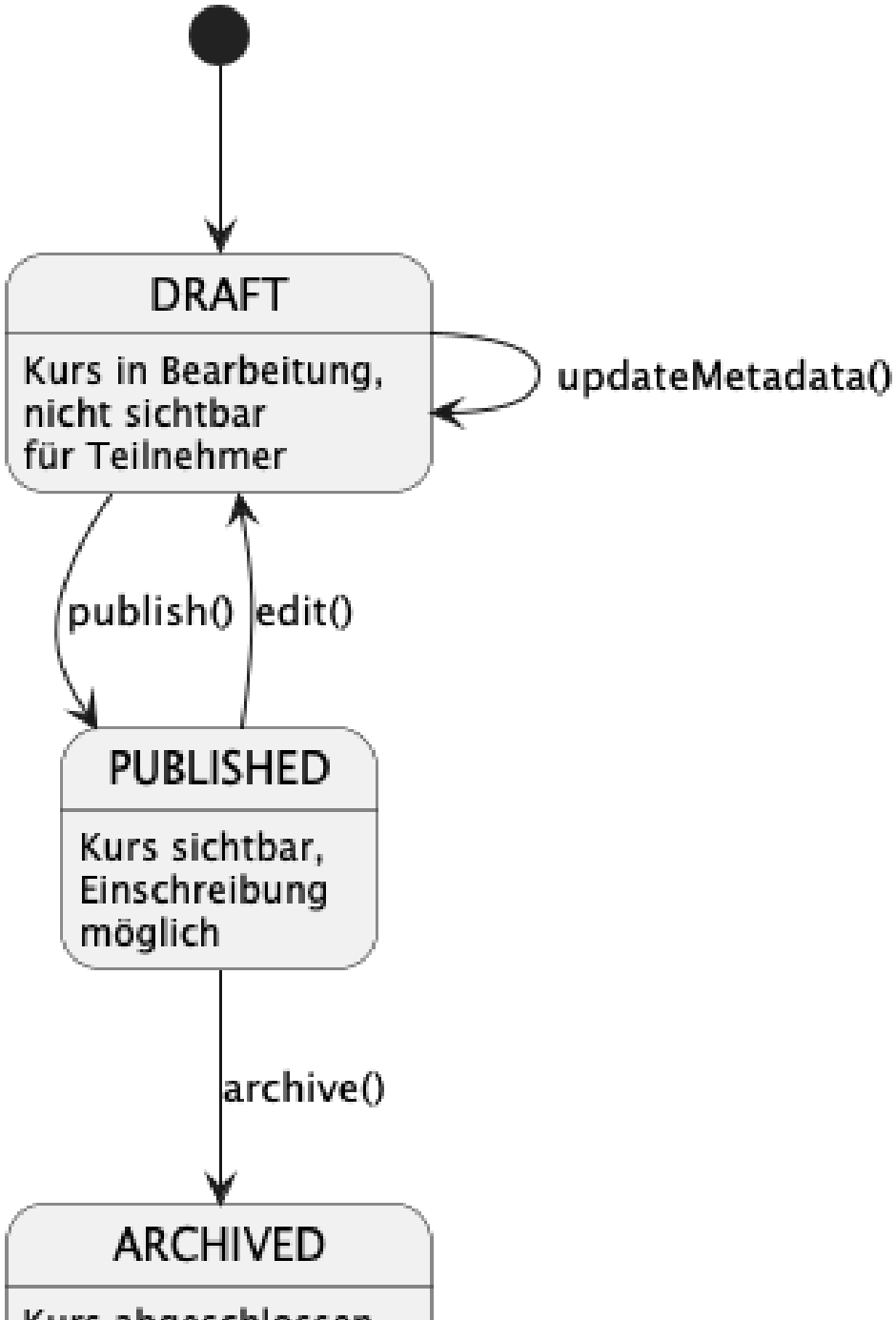


Abbildung 21: Grossansicht – *UML-Klassendiagramm:* `catalog`*-Service* (vgl. Kap. 5.2, Abb. 4)


DRAFT
Kurs in Bearbeitung, nicht sichtbar für Teilnehmer
updateMetadata()
publish()
edit()
PUBLISHED
Kurs sichtbar, Einschreibung möglich
archive()
ARCHIVED

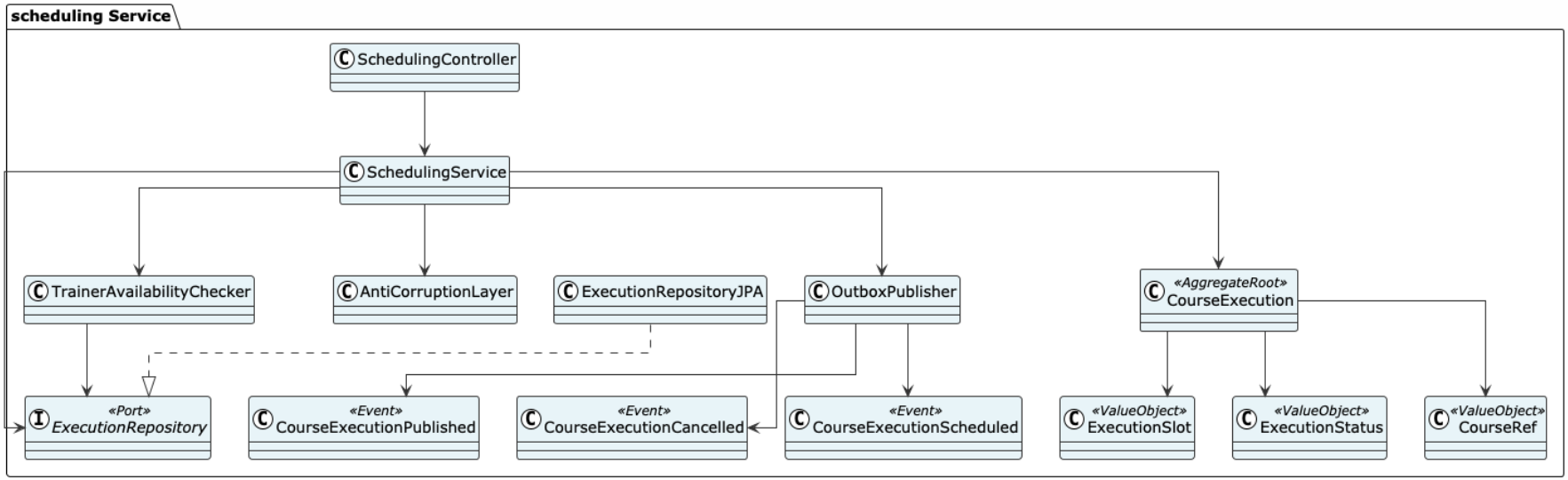


Abbildung 23: Grossansicht – *UML-Klassendiagramm:* `scheduling`*-Service inkl. Anti-Corruption Layer* (vgl. Kap. 5.3, Abb. 6)

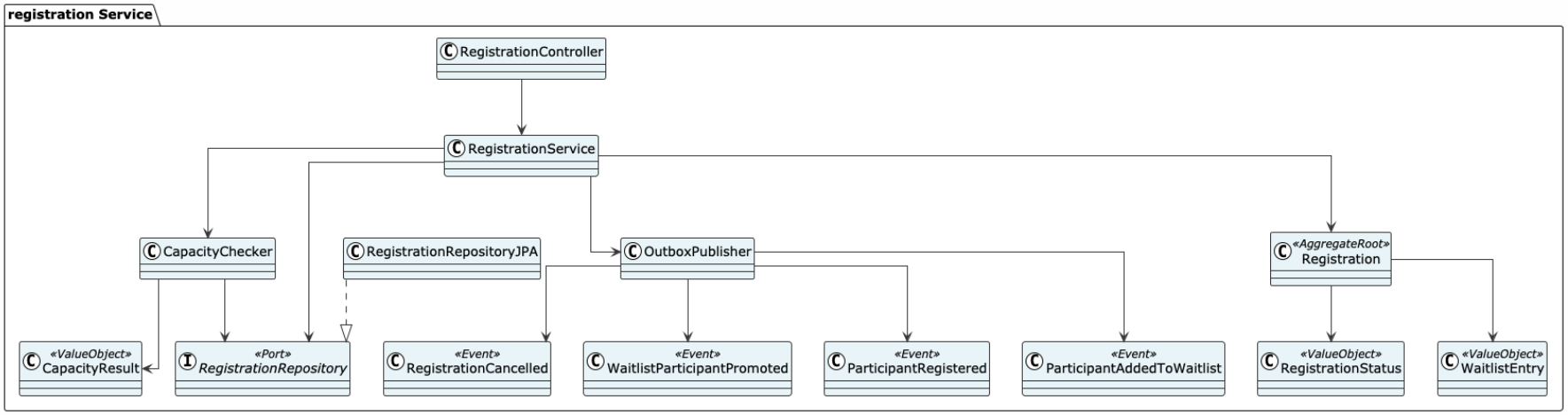


Abbildung 24: Grossansicht – *UML-Klassendiagramm:* ***registration****-Service* (vgl. Kap. 5.4, Abb. 7)

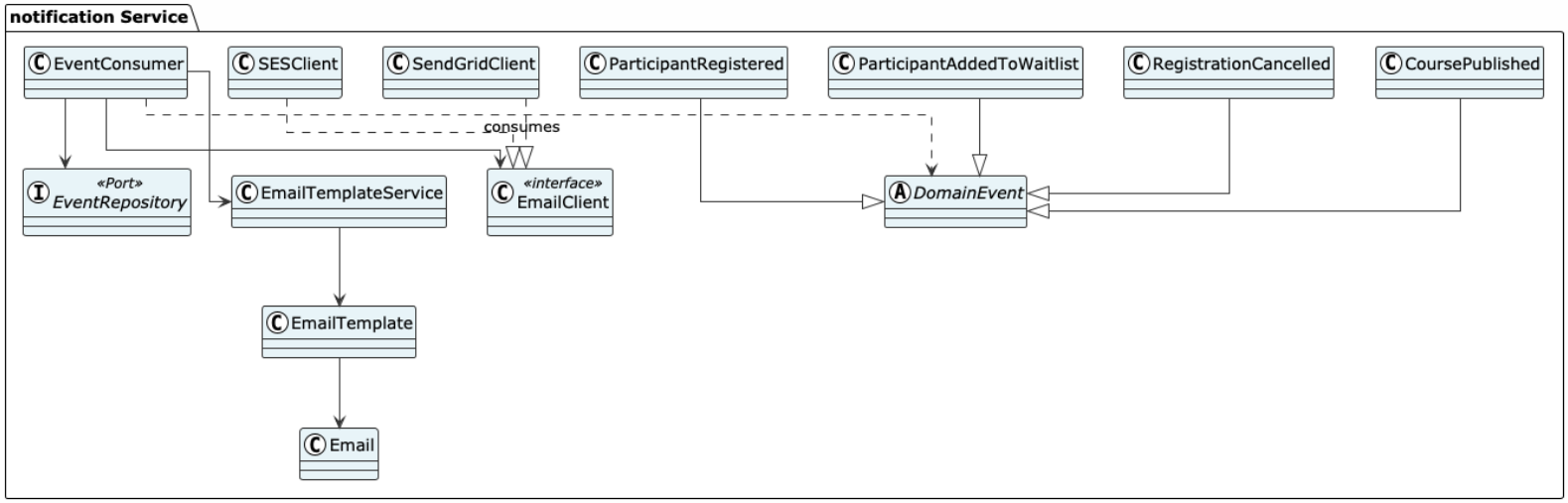


Abbildung 25: Grossansicht – *UML-Klassendiagramm:* `notification`*-Service* (vgl. Kap. 5.5, Abb. 8)

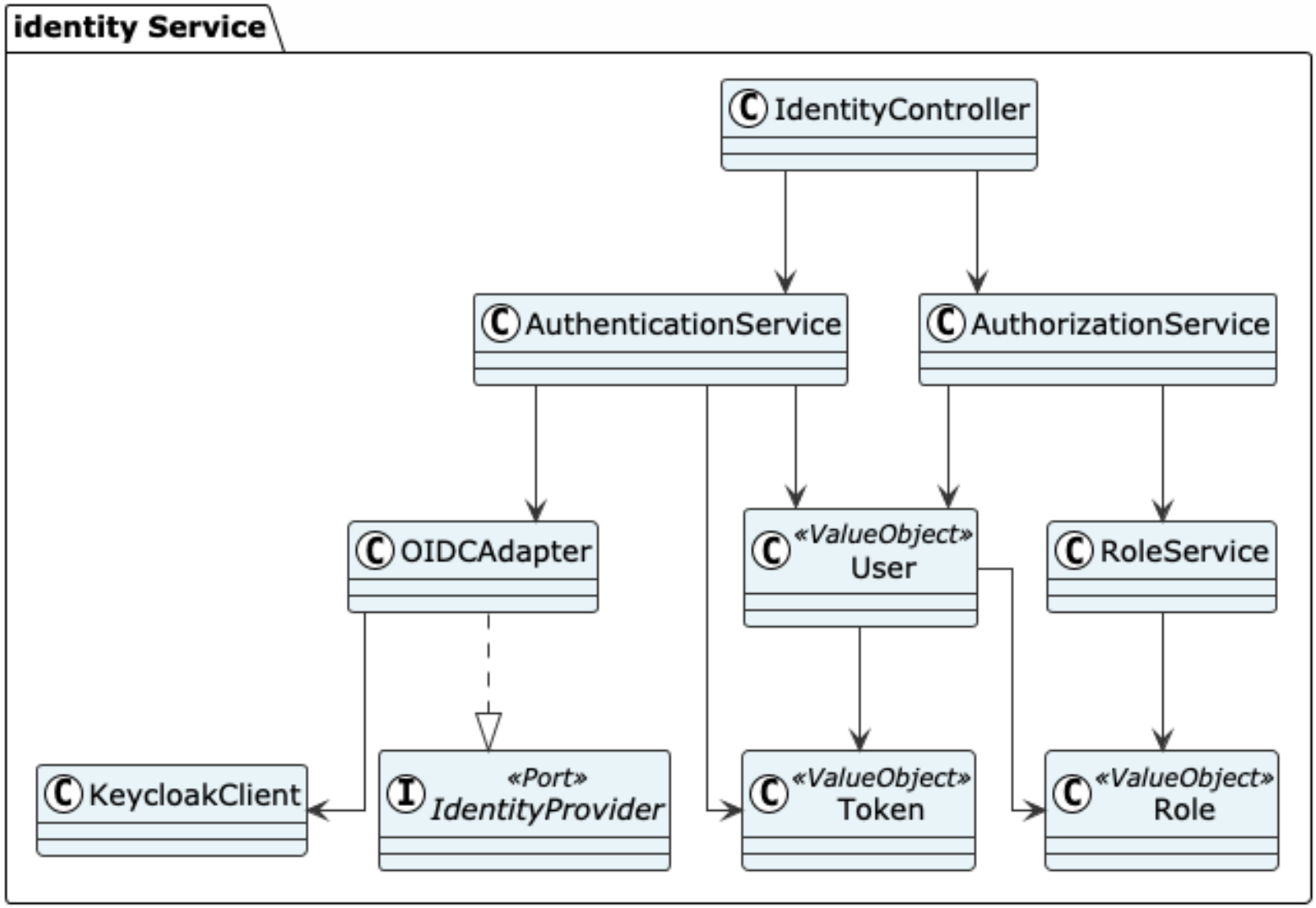


Abbildung 26: Grossansicht – *UML-Klassendiagramm:* `identity`*-Service* (vgl. Kap. 5.6, Abb. 9)

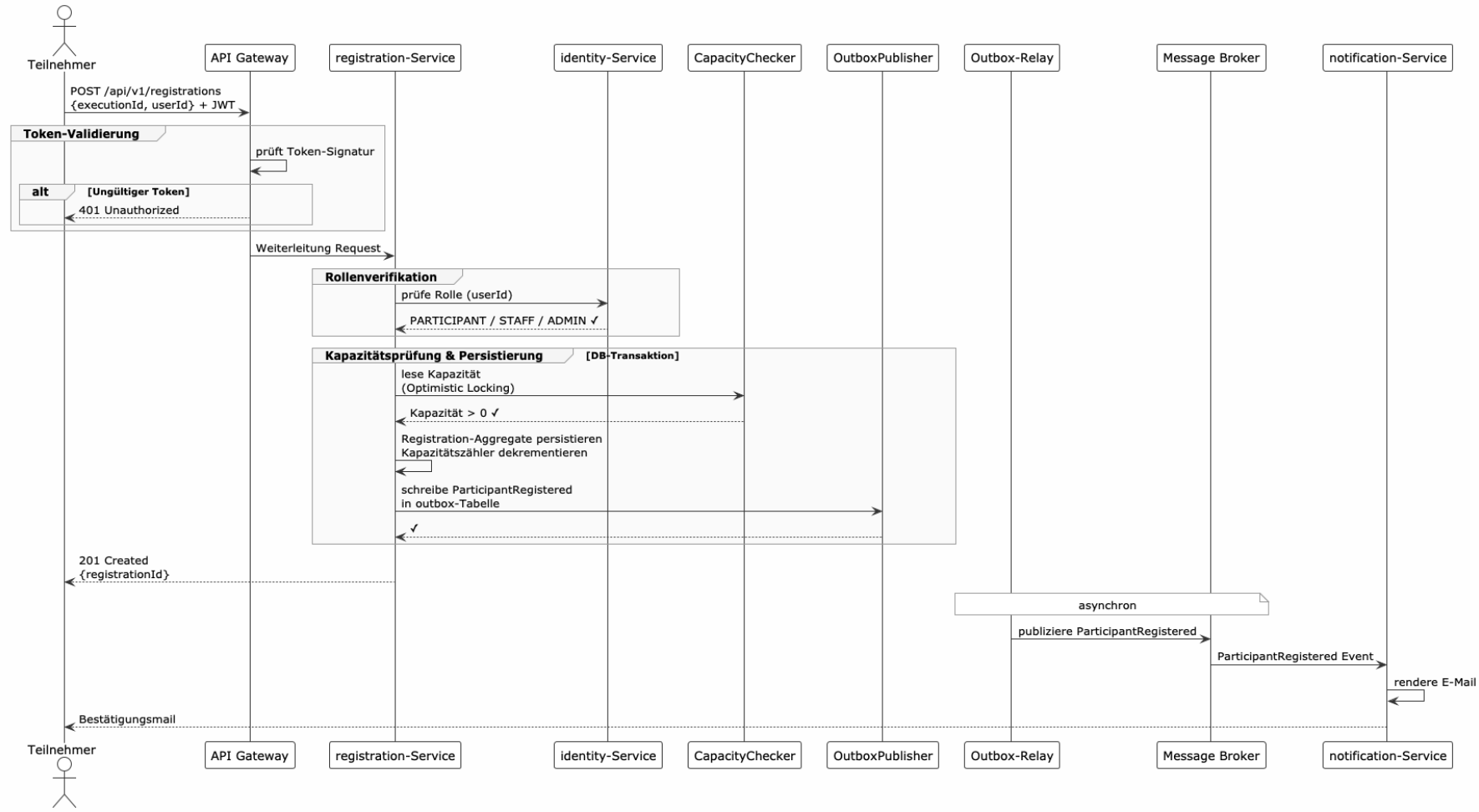


Abbildung 27: Grossansicht – *Szenario 1: Teilnehmer schreibt sich ein (Happy Path)* (vgl. Kap. 6.1, Abb. 10)

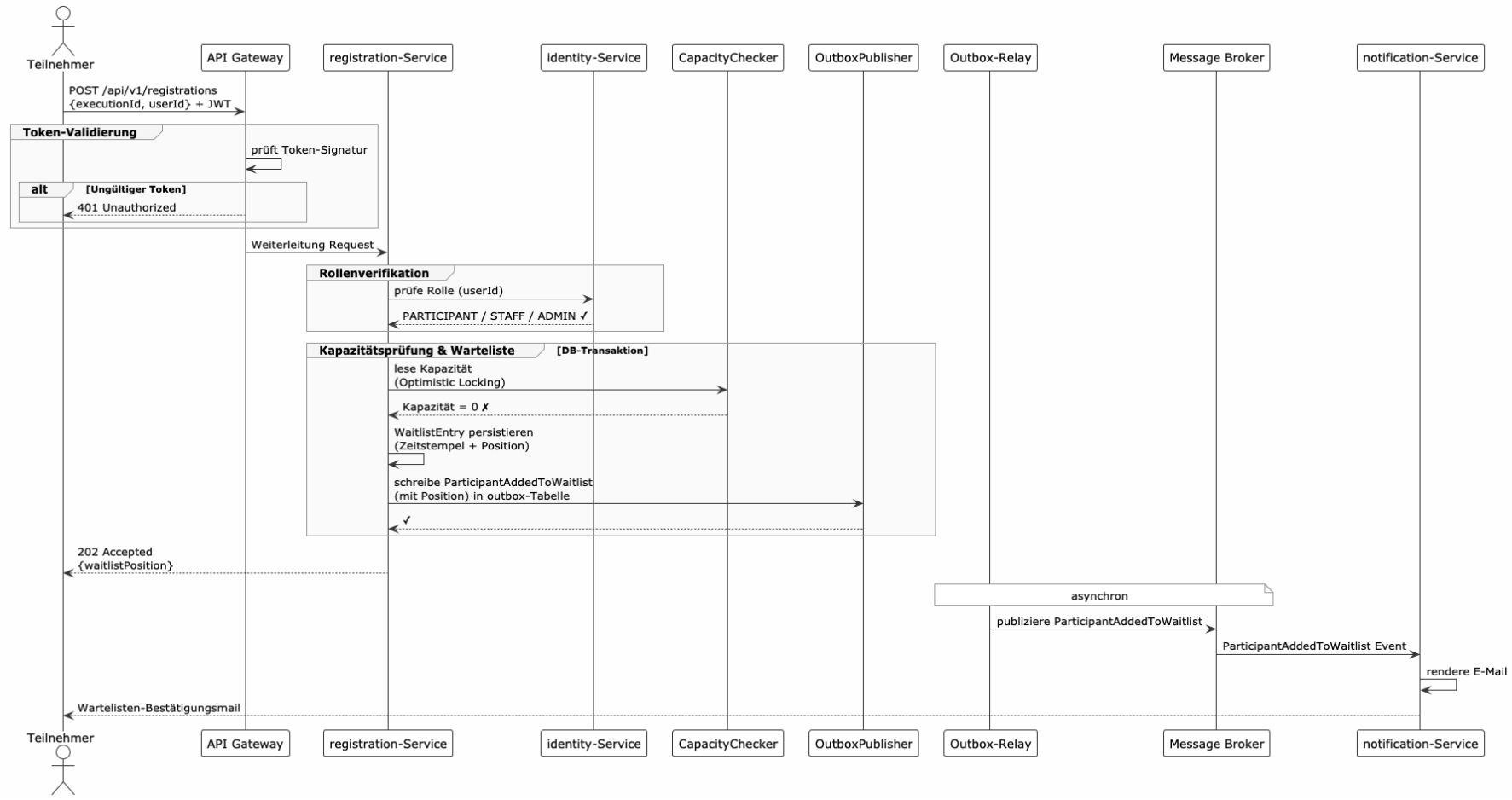


Abbildung 28: Grossansicht – *Szenario 2: Kurs ausgebucht – Warteliste* (vgl. Kap. 6.2, Abb. 11)

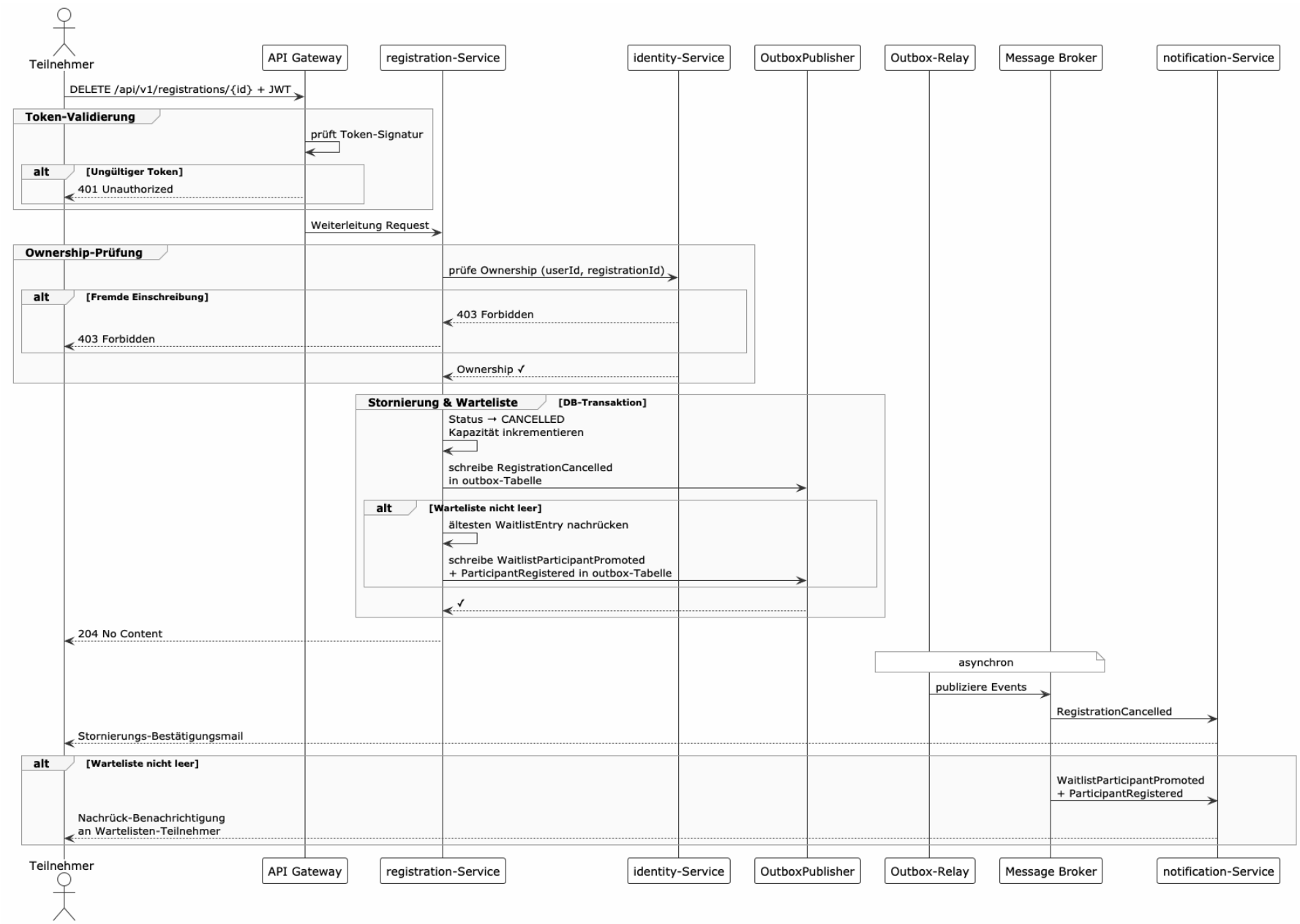


Abbildung 29: Grossansicht – *Szenario 3: Einschreibung stornieren* (vgl. Kap. 6.3, Abb. 12)

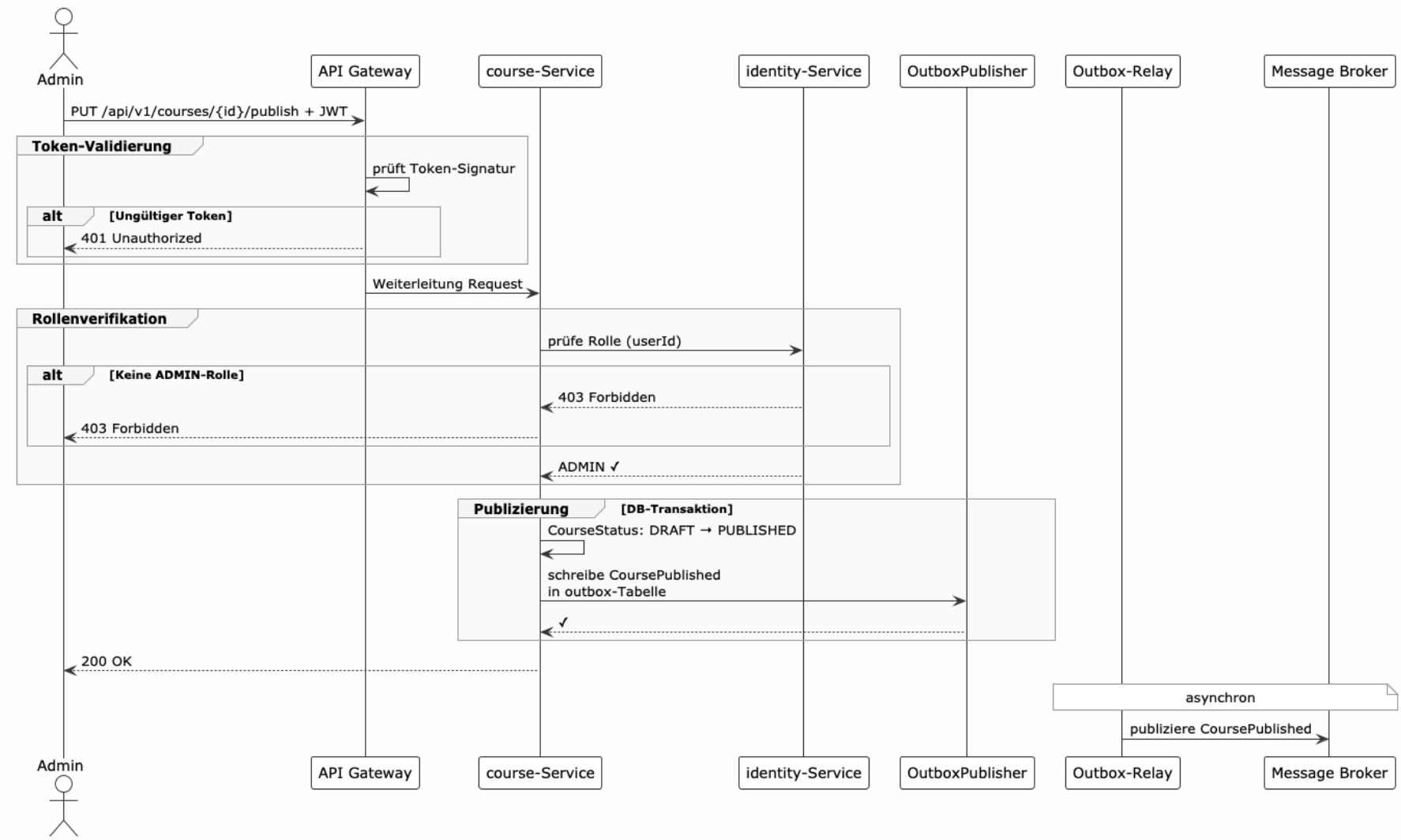


Abbildung 30: Grossansicht – *Szenario 4: Admin veröffentlicht einen Kurs* (vgl. Kap. 6.4, Abb. 13)

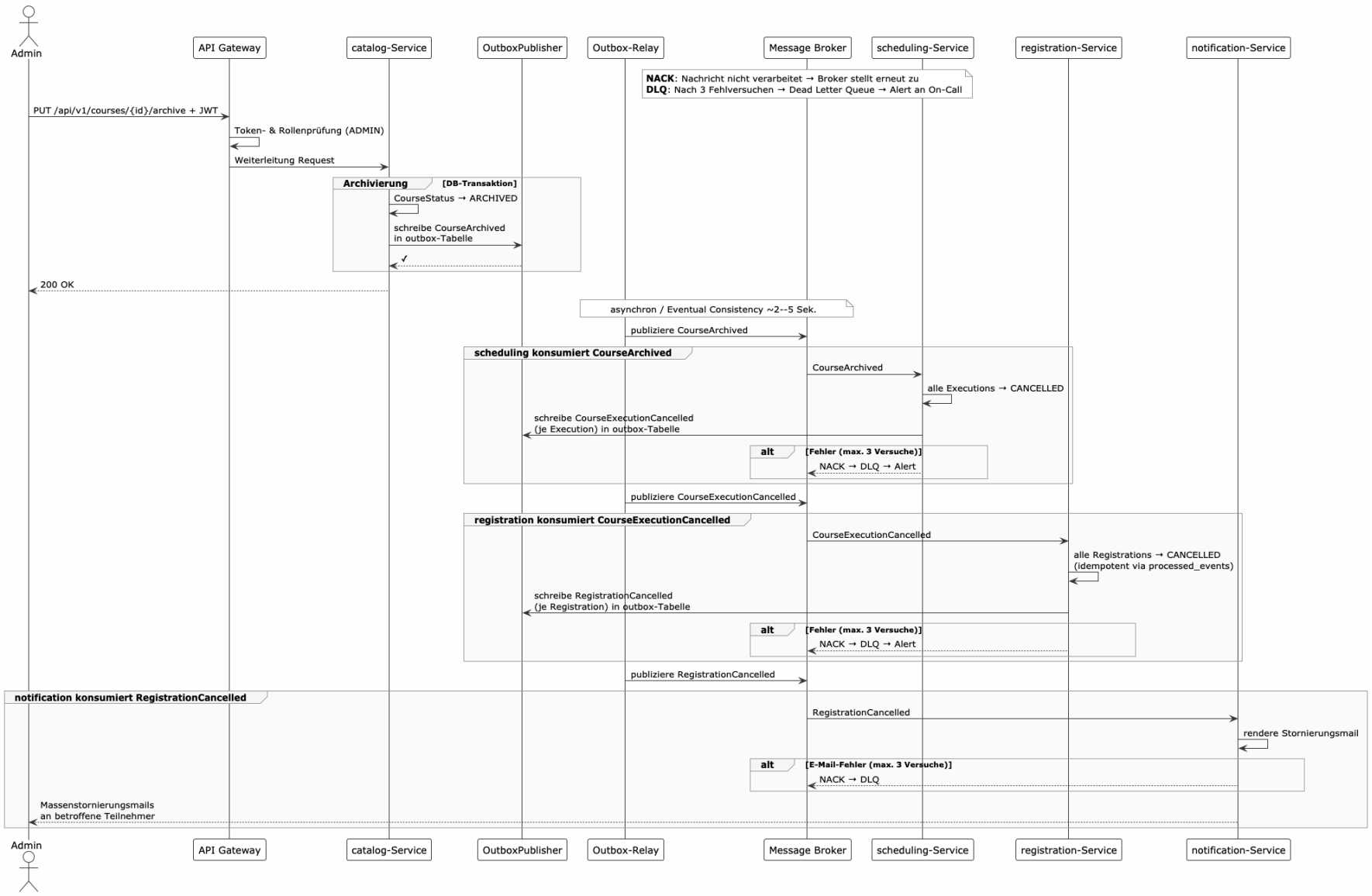


Abbildung 31: Grossansicht – *Szenario 5: Cross-Context-Transaktion (Kurs archivieren)* (vgl. Kap. 6.5, Abb. 14)

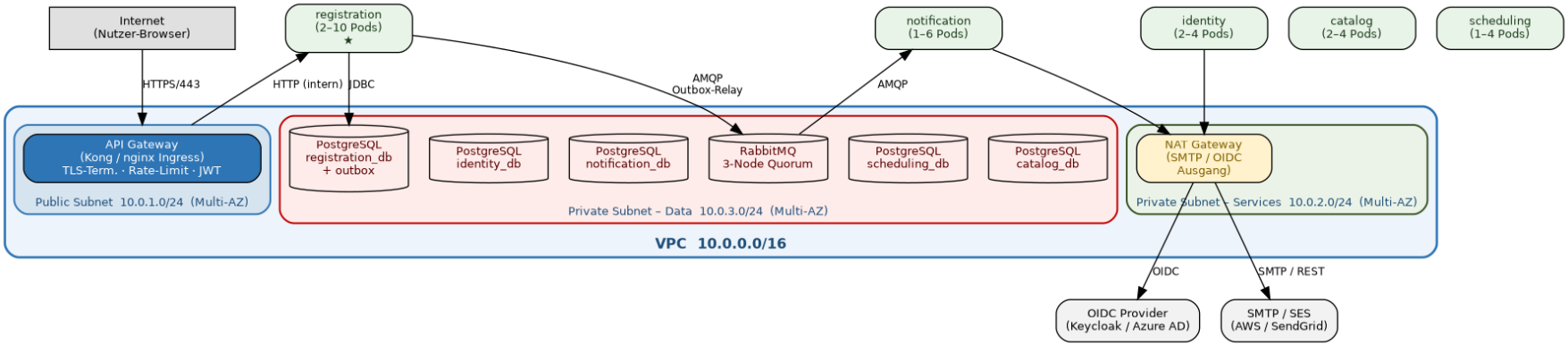


Abbildung 32: Grossansicht – *Deployment-Diagramm: Netzwerk-Topologie Kubernetes / Azure* (vgl. Kap. 7.1, Abb. 15)

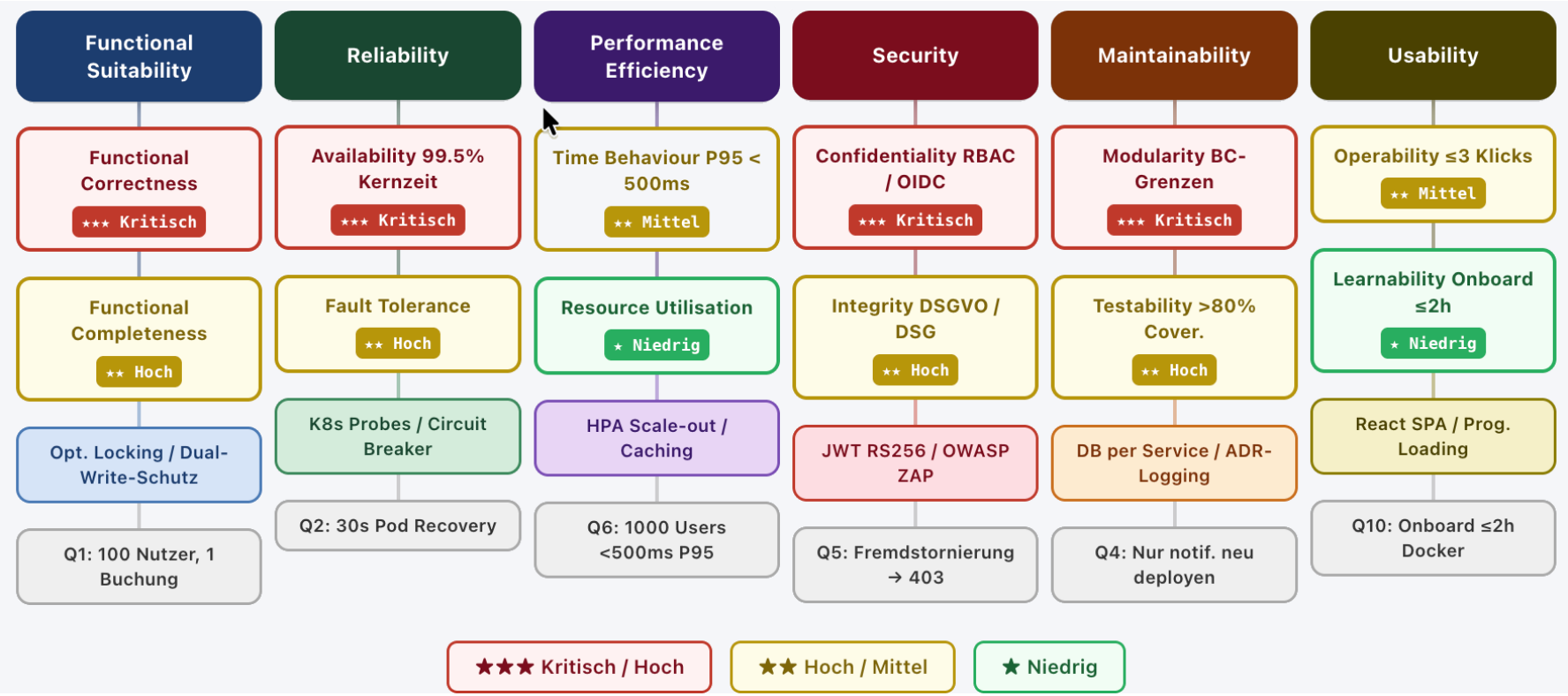


Abbildung 33: Grossansicht – *Qualitätsbaum: EduPlanner Qualitätsanforderungen* (vgl. Kap. 10.1, Abb. 16)

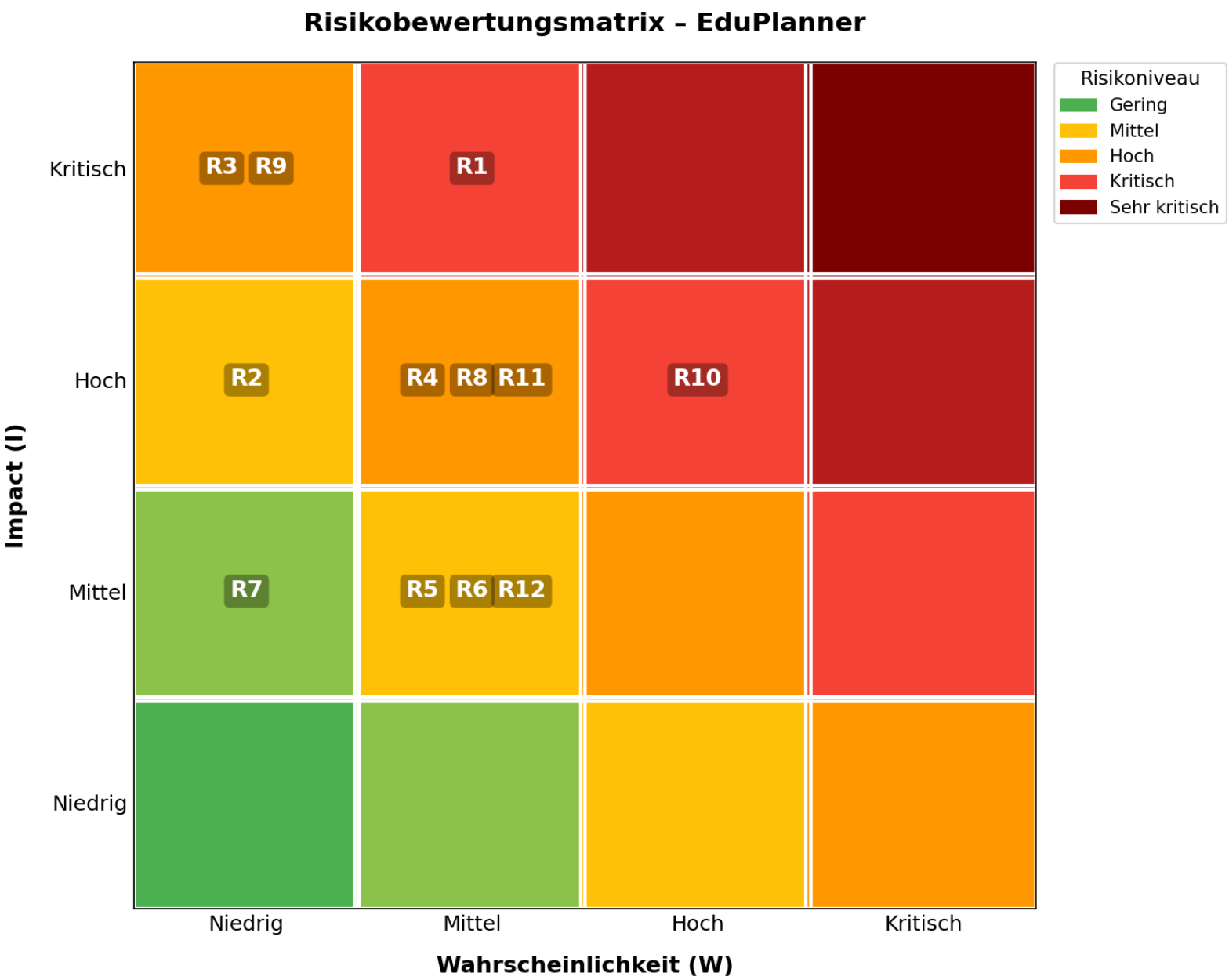


Abbildung 34: Grossansicht – *Risikobewertungsmatrix: Wahrscheinlichkeit vs. Impact (R1–R12)* (vgl. Kap. 11.1, Abb. 17)

C Anhang D: API-Referenz

Contract-First – Einzige Quelle der Wahrheit

Die Datei `openapi/eduplanner-api.yaml` *ist die verbindliche API-Spezifikation (OpenAPI 3.1.0). Server-Stubs (Spring-Boot-Interfaces) werden daraus generiert – nicht umgekehrt. Änderungen an der API* beginnen immer *mit einer Änderung an der YAML-Datei. Begründung: ADR-011. Workflow: Kap. 8.4.*

Contract-First Build-Pipeline

#	Schritt	Tool / Ergebnis
1	Spec schreiben / ändern	`openapi/eduplanner-api.yaml` in Git committen
2	Linting (CI)	`vacuum lint eduplanner-api.yaml` – blockiert Build bei Verstössen gegen API-Design-Regeln
3	Server-Stubs generieren (Build)	`openapi-generator-maven-plugin` erzeugt Spring-Boot-Interfaces (`CoursesApi`, `RegistrationsApi` etc.) mit `interfaceOnly=true`
4	Implementieren	Dev-Team implementiert generierte Interfaces; Compiler erzwingt Konformität
5	Swagger UI bereitstellen	`springdoc-openapi-starter-webmvc-ui` stellt `/swagger-ui/index.html` automatisch bereit
6	Contract Tests (ab Sprint 8)	Pact-Tests validieren Consumer/Provider-Konformität (Tilgung TS2)

Swagger UI – Zugriff je Umgebung

Umgebung	*URL*
Lokal (Docker)	*`http://localhost:{port}/swagger-ui/index.html`*
Staging	*`https://staging-api.eduplanner.skillbridge.ch/{service}/swagger-ui/index.html`*
Produktion	*Nur aus internem Netz (API-Gateway-Policy); nicht öffentlich erreichbar.*

Endpunktübersicht

Method	Pfad	Service	Mindestrolle
Catalog-Service (`/swagger-ui/index.html`, Port 8081)			
GET	`/courses`	catalog	Authentifiziert
POST	`/courses`	catalog	STAFF
GET	`/courses/{courseId}`	catalog	Authentifiziert
PUT	`/courses/{courseId}`	catalog	STAFF
DELETE	`/courses/{courseId}`	catalog	ADMIN
PUT	`/courses/{courseId}/publish`	catalog	ADMIN
PUT	`/courses/{courseId}/archive`	catalog	ADMIN
GET	`/trainers`	catalog	Authentifiziert
POST	`/trainers`	catalog	STAFF
GET	`/trainers/{trainerId}`	catalog	Authentifiziert
PUT	`/trainers/{trainerId}`	catalog	STAFF

Method	Pfad	Service	Mindestrolle
Scheduling-Service (`/swagger-ui/index.html`, Port 8082)			
GET	`/executions`	scheduling	Authentifiziert
POST	`/executions`	scheduling	STAFF
GET	`/executions/{executionId}`	scheduling	Authentifiziert
PUT	`/executions/{executionId}`	scheduling	STAFF
PUT	`/executions/{executionId}/publish`	scheduling	ADMIN
PUT	`/executions/{executionId}/cancel`	scheduling	ADMIN
GET	`/executions/{executionId}/participants`	scheduling	STAFF

Method	Pfad	Service	Mindestrolle
Registration-Service (`/swagger-ui/index.html`, Port 8083 – kritischster Service)			
GET	`/registrations`	registration	Authentifiziert
POST	`/registrations`	registration	PARTICIPANT
GET	`/registrations/{id}`	registration	Authentifiziert
DELETE	`/registrations/{id}`	registration	PARTICIPANT
GET	`/registrations/{id}/waitlist-position`	registration	Authentifiziert

Method	Pfad	Service	Mindestrolle
Identity-Service (`/swagger-ui/index.html`, Port 8084)			
GET	`/users`	identity	ADMIN
GET	`/users/{userId}`	identity	ADMIN / eigenes Profil
GET	`/users/{userId}/roles`	identity	ADMIN
PUT	`/users/{userId}/roles`	identity	ADMIN
DELETE	`/users/{userId}/delete-pii`	identity	ADMIN
GET	`/me`	identity	Authentifiziert

Designentscheide

Aspekt	Umsetzung
Authentifizierung	RS256-signierter JWT Bearer Token; 15 Min. Laufzeit; Validierung am API Gateway (Kap. 8.1).
Fehlerantworten	RFC 7807 `application/problem+json` mit `type`, `title`, `status`, `detail`, `instance`, `errors` (Kap. 8.4).
Pagination	Cursor-basiert (`?cursor=...&limit=20`, max. 100); kein Offset-Paging.
Idempotency	`Idempotency-Key`-Header (UUID) bei allen POST-Endpunkten (Kap. 5.4).
Kapazitätsprüfung	`POST /registrations`: 201 Created bei freiem Platz; 202 Accepted auf Warteliste (Kap. 6.1, 6.2).
DSGVO-Löschung	`DELETE /users/{id}/delete-pii`: 202 Accepted, Abschluss innerhalb 30 Tagen (Q8, Q12).
Versionierung	`/api/v1/...`; Breaking Changes → `/api/v2/...` (ADR-011).
PII-Schutz	`User`-Schema enthält keine E-Mail-Adresse; nur `userId` (UUID) in Events (Kap. 8.2, 4.3).
Optimistic Locking	`version`-Feld (int64) in `Course`, `CourseExecution`, `Registration`; Konflikt → 409 Conflict (Kap. 8.3).